



Teste Unitário

Nome: Marilene da Silva Araujo;

Turma: Desenvolvimento de Sistemas;

Nome da Equipe: Inventory Masters;

1. Suíte de testes unitários - Projeto: MVC_InventoryMasters.Tests

Projeto Alvo: MVC_InventoryMasters

Camada Testada: Repositórios de Dados (Repository)

Frameworks Utilizados: xUnit (framework de testes) e Moq (isolamento de dependências/mocks)

Objetivo da Suíte: Validar de forma isolada e automatizada os comportamentos críticos de persistência, consulta, tratamento de nulos e resiliência a exceções de banco de dados e serviços externos em todas as entidades do sistema.

2. Testes implementados

*Abaixo estão listados os casos de teste desenvolvidos e implementados utilizando o framework **xUnit** e o padrão **AAA (Arrange, Act, Assert)** para os componentes:*

Casos de Teste - UsuariosRepository

01. Adicionar_ErroNoBanco_LancaExcecao

- **Arrange**
 - ↓
 - Preparar os dados e condições necessárias (configurar um cenário de falha simulada ou dados nulos no contexto de persistência para acionar o tratamento de exceções).
- **Act**
 - ↓
 - Executar o comportamento que será testado (tentar adicionar o usuário invocando o método correspondente no repositório).
- **Assert**
 - ↓
 - Verificar se o resultado corresponde ao esperado (validar se a exceção esperada foi lançada e capturada pelo mecanismo de tratamento de erro).

02. Adicionar_UsuarioValido_AdicionaComSucesso

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (instanciar um objeto de usuário válido preenchido com todos os dados obrigatórios e as configurações de mock).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método de adição de usuário de forma assíncrona).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (assegurar que o registro foi adicionado com sucesso sem disparar exceções).

03. Atualizar_ErroDuranteAtualizacao_LancaExcecao

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (configurar dados inconsistentes ou uma falha de conexão simulada para o processo de atualização).

- **Act**

- ↓
- Executar o comportamento que será testado (tentar atualizar o registro do usuário no repositório sob condições de erro).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar que a exceção correspondente foi disparada adequadamente).

04. Atualizar_UsuarioExistenteComDadosValidos_AtualizaComSucesso

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (localizar um usuário existente e preparar os novos dados válidos para a alteração).

- **Act**

- ↓

- Executar o comportamento que será testado (chamar o método de atualização do repositório).

- **Assert**

- ↓

- Verificar se o resultado corresponde ao esperado (confirmar que a operação de update foi concluída com sucesso).

05. AtualizarStatus_IdValidoENovoStatus_AtualizaStatusComSucesso

- **Arrange**

- ↓

- Preparar os dados e condições necessárias (identificar um usuário cadastrado e definir um novo status válido a ser aplicado).

- **Act**

- ↓

- Executar o comportamento que será testado (chamar o método específico de atualização de status do usuário).

- **Assert**

- ↓

- Verificar se o resultado corresponde ao esperado (validar que o status foi alterado e salvo com sucesso no banco).

06. BuscarPorEmail_EmailExistente_RetornaUsuarioCorrespondente

- **Arrange**

- ↓

- Preparar os dados e condições necessárias (definir um e-mail válido e ativo cadastrado na base de dados simulada).

- **Act**

- ↓

- Executar o comportamento que será testado (chamar o método *BuscarPorEmail* passando o e-mail alvo).

- **Assert**

- ↓

-
- *Verificar se o resultado corresponde ao esperado (garantir que o usuário retornado não é nulo e corresponde exatamente ao e-mail pesquisado).*

07. *BuscarPorEmail_EmailInexistente_ReturnaNull*

- **Arrange**

- ↓
- *Preparar os dados e condições necessárias (definir uma string de e-mail que não existe na base de dados).*

- **Act**

- ↓
- *Executar o comportamento que será testado (chamar o método `BuscarPorEmail` com o e-mail inexistente).*

- **Assert**

- ↓
- *Verificar se o resultado corresponde ao esperado (validar que o retorno da consulta é estritamente null).*

08. *BuscarPorId_IdExistente_ReturnaUsuarioCorrespondente*

- **Arrange**

- ↓
- *Preparar os dados e condições necessárias (informar um identificador de ID válido presente no repositório).*

- **Act**

- ↓
- *Executar o comportamento que será testado (chamar o método `BuscarPorId` com o ID informado).*

- **Assert**

- ↓
- *Verificar se o resultado corresponde ao esperado (assegurar que o objeto retornado possui o ID correspondente e contém os dados corretos).*

09. *BuscarPorId_IdInexistente_ReturnaNull*

- **Arrange**

- ↓

-
- Preparar os dados e condições necessárias (definir um ID inexistente na base de dados).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método `BuscarPorId` buscando pelo ID inválido).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (certificar-se de que o método retorna null com segurança).

10. Excluir_IdDeUsuarioExistente_RemoveComSucesso

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (selecionar o ID de um usuário cadastrado e ativo para a operação de exclusão).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método de exclusão do repositório passando o ID válido).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar que a remoção ocorreu sem erros).

11. Excluir_IdInexistente_NaoLancaExcecao

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (configurar os mocks de configuração do Firebase, logger e contexto de usuário, instanciar o repositório e gerar um identificador inexistente aleatório).

- **Act**

- ↓

-
- Executar o comportamento que será testado (capturar a exceção assíncrona gerada ao tentar executar o método de exclusão passando o ID inexistente).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (garantir que nenhuma exceção foi lançada, confirmando o comportamento defensivo e estável do repositório perante chaves inválidas).

12. ListarPorEmpresa_EmpresaNulaOuVazia_UsaEmpresaContextoOuPadrao

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (configurar parâmetros de empresa nulos ou vazios e mockar o serviço de contexto do usuário).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método ListarPorEmpresa sem fornecer um ID explícito de empresa).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar que o fallback utilizou automaticamente a empresa obtida pelo contexto ou padrão).

13. ListarPorEmpresa_EmpresaValida_RetornaUsuariosDaEmpresa

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (fornecer um ID de empresa válido e estruturar registros vinculados a ele).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método ListarPorEmpresa filtrando pela empresa informada).

- **Assert**

- ↓

-
- *Verificar se o resultado corresponde ao esperado (garantir que a listagem retornada contém apenas os usuários pertencentes àquela empresa).*

14. *ListarTodos_ColecaoVazia_RetornaListaVazia*

- **Arrange**

- ↓
- *Preparar os dados e condições necessárias (configurar a base de dados simulada para retornar uma coleção totalmente vazia de usuários).*

- **Act**

- ↓
- *Executar o comportamento que será testado (chamar o método `ListarTodos` no repositório).*

- **Assert**

- ↓
- *Verificar se o resultado corresponde ao esperado (validar que a coleção retornada encontra-se vazia).*

15. *ListarTodos_ComUsuariosCadastrados_RetornaListaPreenchidaComIds*

- **Arrange**

- ↓
- *Preparar os dados e condições necessárias (popular a base de mock com múltiplos registros de usuários cadastrados contendo IDs válidos).*

- **Act**

- ↓
- *Executar o comportamento que será testado (chamar o método `ListarTodos`).*

- **Assert**

- ↓
- *Verificar se o resultado corresponde ao esperado (assegurar que a lista retornada não é nula e contém todos os usuários cadastrados com seus respectivos identificadores).*

Casos de Teste - **PerfisRepository**

01. *Adicionar_PerfilValido_AdicionaComSucesso*

- **Arrange**

- ↓

-
- Preparar os dados e condições necessárias (instanciar um objeto de perfil válido preenchido com as permissões e dados obrigatórios).
 - **Act**
 - ↓
 - Executar o comportamento que será testado (chamar de forma assíncrona o método de adição de perfil no repositório).
 - **Assert**
 - ↓
 - Verificar se o resultado corresponde ao esperado (assegurar que o perfil foi inserido com sucesso sem disparar exceções).

02. Atualizar_PerfilExistente_AtualizaComSucesso

- **Arrange**
 - ↓
 - Preparar os dados e condições necessárias (localizar um registro de perfil já existente e preparar os novos dados válidos para alteração).
- **Act**
 - ↓
 - Executar o comportamento que será testado (chamar o método de atualização do repositório).
- **Assert**
 - ↓
 - Verificar se o resultado corresponde ao esperado (confirmar que a operação de modificação foi concluída com sucesso).

03. BuscarPorId_ErroNoBanco_RetornaNull

- **Arrange**
 - ↓
 - Preparar os dados e condições necessárias (configurar o mock de acesso a dados para simular uma falha ou exceção de conexão ao buscar por ID).
- **Act**
 - ↓
 - Executar o comportamento que será testado (chamar o método `BuscarPorId` sob a condição de erro de infraestrutura).

- **Assert**

- ↓
- *Verificar se o resultado corresponde ao esperado (validar que o repositório trata a exceção adequadamente e retorna null).*

04. *BuscarPorId_IdExistente_RetornaPerfilCorrespondente*

- **Arrange**

- ↓
- *Preparar os dados e condições necessárias (informar um identificador de ID válido presente na base de dados simulada).*

- **Act**

- ↓
- *Executar o comportamento que será testado (chamar o método *BuscarPorId* com o ID informado).*

- **Assert**

- ↓
- *Verificar se o resultado corresponde ao esperado (assegurar que o objeto retornado não é nulo e corresponde ao perfil esperado).*

05. *BuscarPorId_IdInexistente_RetornaNull*

- **Arrange**

- ↓
- *Preparar os dados e condições necessárias (definir um ID inexistente na base de dados).*

- **Act**

- ↓
- *Executar o comportamento que será testado (chamar o método *BuscarPorId* buscando pelo ID inválido).*

- **Assert**

- ↓
- *Verificar se o resultado corresponde ao esperado (certificar-se de que o método retorna null com segurança).*

06. *Inativar_IdValido_InativaComSucesso*

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (fornecer o ID válido de um perfil ativo que será desativado).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método responsável por inativar o perfil).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar que o status do perfil foi alterado para inativo sem erros).

07. ListarPorEmpresa_EmpresaNulaOuVazia_UsaEmpresaContexto

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (configurar parâmetros de empresa nulos ou vazios e mockar o serviço de contexto do usuário).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método ListarPorEmpresa sem passar um ID explícito).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar que o fallback utilizou automaticamente a empresa obtida pelo contexto).

08. ListarPorEmpresa_EmpresaValida_RetornaPerfisDaEmpresa

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (fornecer um ID de empresa válido e estruturar registros de perfis vinculados a ela).

- **Act**

- ↓

- *Executar o comportamento que será testado (chamar o método `ListarPorEmpresa` filtrando pela empresa informada).*

- **Assert**

- ↓

- *Verificar se o resultado corresponde ao esperado (garantir que a listagem retornada contém apenas os perfis pertencentes àquela empresa).*

09. `ListarTodos_ComPerfisCadastrados_RetornaListaPreenchida`

- **Arrange**

- ↓

- *Preparar os dados e condições necessárias (popular a base simulada com múltiplos registros de perfis cadastrados).*

- **Act**

- ↓

- *Executar o comportamento que será testado (chamar o método de listagem geral `ListarTodos`).*

- **Assert**

- ↓

- *Verificar se o resultado corresponde ao esperado (assegurar que a lista retornada não é nula e contém os perfis cadastrados).*

10. `ListarTodos_ErroNoBanco_RetornaListaVazia`

- **Arrange**

- ↓

- *Preparar os dados e condições necessárias (configurar a base de dados para lançar uma exceção de conexão durante a listagem geral).*

- **Act**

- ↓

- *Executar o comportamento que será testado (chamar o método `ListarTodos` sob falha de infraestrutura).*

- **Assert**

- ↓

-
- *Verificar se o resultado corresponde ao esperado (garantir o tratamento seguro da exceção com o retorno de uma lista vazia).*

Casos de Teste - TokensAcessoKinectRepository

01. Adicionar_TokenValido_AdicionaComSucesso

- **Arrange**
 - ↓
 - *Preparar os dados e condições necessárias (instanciar um objeto de token de acesso Kinect válido, contendo hash, data de expiração e parâmetros obrigatórios).*
- **Act**
 - ↓
 - *Executar o comportamento que será testado (chamar de forma assíncrona o método de adição do token no repositório).*
- **Assert**
 - ↓
 - *Verificar se o resultado corresponde ao esperado (assegurar que o token foi inserido com sucesso sem disparar exceções).*

02. BuscarAtivoPorHash_ErroNoBanco_RetornaNull

- **Arrange**
 - ↓
 - *Preparar os dados e condições necessárias (configurar o mock de acesso a dados para simular uma exceção ou falha de conexão durante a busca por hash).*
- **Act**
 - ↓
 - *Executar o comportamento que será testado (chamar o método BuscarAtivoPorHash sob a condição de erro de infraestrutura).*
- **Assert**
 - ↓
 - *Verificar se o resultado corresponde ao esperado (validar que o repositório trata a exceção com segurança retornando null).*

03. BuscarAtivoPorHash_HashExistenteEValido_RetornaTokenCorrespondente

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (definir um hash de token ativo e válido cadastrado na base simulada).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método `BuscarAtivoPorHash` passando o hash correspondente).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (garantir que o token retornado não é nulo e corresponde exatamente aos dados pesquisados).

04. `BuscarAtivoPorHash_HashInexistenteOuInvalido_RetornaNull`

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (fornecer uma string de hash inexistente ou com validade expirada/inválida).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método `BuscarAtivoPorHash` com o hash inválido).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (certificar-se de que o método retorna null adequadamente).

05. `MarcarComoUtilizado_TokenComIdNuloOuVazio_NaoExecutaAtualizacao`

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (configurar a chamada para marcar o token como utilizado passando um identificador de ID nulo ou vazio).

- **Act**

- ↓

- Executar o comportamento que será testado (chamar o método `MarcarComoUtilizado` com os dados incompletos).
- **Assert**
 - ↓
 - Verificar se o resultado corresponde ao esperado (validar que a operação é abortada com segurança sem acionar comandos de atualização na base).

06. `MarcarComoUtilizado_TokenComIdValido_AtualizaComSucesso`

- **Arrange**
 - ↓
 - Preparar os dados e condições necessárias (informar o ID de um token de acesso válido e existente que ainda não foi utilizado).
- **Act**
 - ↓
 - Executar o comportamento que será testado (chamar o método `MarcarComoUtilizado` informando o ID válido).
- **Assert**
 - ↓
 - Verificar se o resultado corresponde ao esperado (confirmar que a flag de utilização/atualização foi gravada com sucesso na base de dados).

ParceirosSistemasRepository

01. `Adicionar_ParceiroValido_AdicionaComSucesso`

- **Arrange**
 - ↓
 - Preparar os dados e condições necessárias (instanciar um objeto de parceiro válido contendo nome, documentos e as informações obrigatórias).
- **Act**
 - ↓
 - Executar o comportamento que será testado (chamar de forma assíncrona o método de adição do parceiro no repositório).
- **Assert**
 - ↓

-
- *Verificar se o resultado corresponde ao esperado (assegurar que o registro foi inserido com sucesso sem disparar exceções).*

02. Atualizar_ParceiroValido_AtualizaComSucesso

- **Arrange**

- ↓
- *Preparar os dados e condições necessárias (localizar um registro de parceiro existente e preparar novos dados válidos para alteração).*

- **Act**

- ↓
- *Executar o comportamento que será testado (chamar o método de atualização do repositório).*

- **Assert**

- ↓
- *Verificar se o resultado corresponde ao esperado (confirmar que a operação de modificação dos dados foi concluída com sucesso).*

03. AtualizarStatus_ErroNoBanco_LancaExcecao

- **Arrange**

- ↓
- *Preparar os dados e condições necessárias (configurar o mock de acesso a dados para simular uma falha de infraestrutura ou perda de conexão ao alterar o status).*

- **Act**

- ↓
- *Executar o comportamento que será testado (chamar o método de atualização de status sob a condição de erro de banco).*

- **Assert**

- ↓
- *Verificar se o resultado corresponde ao esperado (validar que a exceção correspondente foi lançada adequadamente).*

04. AtualizarStatus_IdEStatusValidos_AtualizaComSucesso

- **Arrange**

- ↓

-
- Preparar os dados e condições necessárias (informar o ID de um parceiro existente e definir um novo status válido).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método responsável por atualizar o status do parceiro).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar que a alteração de status foi salva corretamente na base).

05. BuscarPorId_IdExistente_RetornaParceiroCorrespondente

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (informar um identificador de ID válido presente no repositório).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método *BuscarPorId* com o ID informado).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (assegurar que o objeto retornado não é nulo e corresponde exatamente ao parceiro buscado).

06. BuscarPorId_IdInexistente_RetornaNull

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (definir um ID inexistente na base de dados).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método *BuscarPorId* buscando pelo ID inválido).

- **Assert**

- ↓
- *Verificar se o resultado corresponde ao esperado (certificar-se de que o método retorna null com segurança).*

07. Excluir_ErroNoBanco_LancaExcecao

- **Arrange**

- ↓
- *Preparar os dados e condições necessárias (configurar o repositório para simular uma falha de conexão ou erro crítico no momento da exclusão).*

- **Act**

- ↓
- *Executar o comportamento que será testado (tentar executar a remoção do registro sob falha de infraestrutura).*

- **Assert**

- ↓
- *Verificar se o resultado corresponde ao esperado (garantir que a exceção esperada foi propagada corretamente pelo componente).*

08. Excluir_IdValido_RemoveComSucesso

- **Arrange**

- ↓
- *Preparar os dados e condições necessárias (fornecer o ID válido de um parceiro cadastrado que será removido).*

- **Act**

- ↓
- *Executar o comportamento que será testado (chamar o método de exclusão do repositório passando o ID válido).*

- **Assert**

- ↓
- *Verificar se o resultado corresponde ao esperado (validar que a exclusão ocorreu com sucesso sem disparar erros).*

09. FiltrarAvancado_ComFiltrosPreenchidos_RetornaListaFiltrada

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (definir critérios específicos de filtragem avançada, como status e tipo de parceiro).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método de filtro avançado passando os parâmetros configurados).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (garantir que a listagem retornada atende rigorosamente aos critérios aplicados).

10. ListarPorEmpresa_EmpresaValida_RetornaListaFiltrada

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (fornecer um ID de empresa válido e estruturar parceiros vinculados a ela).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método ListarPorEmpresa filtrando pela empresa informada).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (assegurar que a lista retornada contém apenas os registros associados àquela empresa).

11. ListarTodos_ParceirosCadastrados_RetornaListaCompleta

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (popular a base simulada com múltiplos registros de parceiros cadastrados).

- **Act**

-
- ↓
 - Executar o comportamento que será testado (chamar o método de listagem geral `ListarTodos`).
 - **Assert**
 - ↓
 - Verificar se o resultado corresponde ao esperado (validar que a coleção retornada está preenchida e contém todos os parceiros).

12. Pesquisar_TermoValido_RetornaParceirosCorrespondentes

- **Arrange**
 - ↓
 - Preparar os dados e condições necessárias (definir um termo de pesquisa textual válido, como parte do nome ou documento de um parceiro).
- **Act**
 - ↓
 - Executar o comportamento que será testado (chamar o método de pesquisa passando o termo informado).
- **Assert**
 - ↓
 - Verificar se o resultado corresponde ao esperado (garantir que os parceiros retornados correspondem ao padrão textual pesquisado).

ParametrosSistemasRepository

01. Buscar_ChamadaSemParametros_UtilizaContextoUsuarioEfetivo

- **Arrange**
 - ↓
 - Preparar os dados e condições necessárias (configurar o contexto de usuário efetivo com os dados necessários para a busca padrão).
- **Act**
 - ↓
 - Executar o comportamento que será testado (chamar o método de busca sem passar parâmetros explícitos).
- **Assert**

- ↓

- *Verificar se o resultado corresponde ao esperado (validar se o repositório utilizou corretamente as informações provenientes do contexto do usuário).*

02. *BuscarPorEmpresa_EmpresaExistente_RetornaParametrosCorrespondentes*

- **Arrange**

- ↓

- *Preparar os dados e condições necessárias (fornecer o ID de uma empresa válida que possua parâmetros cadastrados).*

- **Act**

- ↓

- *Executar o comportamento que será testado (chamar o método *BuscarPorEmpresa* com o identificador informado).*

- **Assert**

- ↓

- *Verificar se o resultado corresponde ao esperado (assegurar que os parâmetros retornados correspondem à empresa solicitada).*

03. *BuscarPorEmpresa_EmpresaInexistenteComFallbackGlobal_RetornaParametrosGlobais*

- **Arrange**

- ↓

- *Preparar os dados e condições necessárias (configurar um ID de empresa que não existe na base, mas mantendo a regra de fallback global habilitada).*

- **Act**

- ↓

- *Executar o comportamento que será testado (chamar o método de busca por empresa com fallback).*

- **Assert**

- ↓

- *Verificar se o resultado corresponde ao esperado (confirmar que o sistema retornou os parâmetros globais de configuração como alternativa).*

04. *BuscarPorEmpresa_ErroNoBanco_RetornaParametrosPadrao*

- **Arrange**

-
- ↓
 - *Preparar os dados e condições necessárias (simular uma exceção ou falha de infraestrutura no banco de dados durante a consulta).*
 - **Act**
 - ↓
 - *Executar o comportamento que será testado (chamar o método de busca sob a condição de erro de banco).*
 - **Assert**
 - ↓
 - *Verificar se o resultado corresponde ao esperado (validar que a exceção foi tratada de forma segura retornando os parâmetros padrão de fábrica).*
- 05. CalcularPercentualOcupacao_CapacidadeMaximaZeroOuNula_TrataExcecaoOuRetornaZero**
- **Arrange**
 - ↓
 - *Preparar os dados e condições necessárias (configurar o cálculo de ocupação passando capacidade máxima igual a zero ou nula).*
 - **Act**
 - ↓
 - *Executar o comportamento que será testado (chamar o método de cálculo de percentual de ocupação).*
 - **Assert**
 - ↓
 - *Verificar se o resultado corresponde ao esperado (garantir que a divisão por zero foi prevenida adequadamente, retornando zero ou tratando o cenário sem quebrar).*
- 06. CalcularPercentualOcupacao_ValoresValidos_RetornaPercentualCorreto**
- **Arrange**
 - ↓
 - *Preparar os dados e condições necessárias (definir valores numéricos válidos para a capacidade máxima e ocupação atual).*
 - **Act**
 - ↓
-

- Executar o comportamento que será testado (chamar o método de cálculo correspondente).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar que o percentual retornado matematicamente reflete a proporção correta).

07. ObterPadroes_ChamadaPadrao_RetornaValoresIniciaisPredefinidos

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (configurar o ambiente para solicitar as diretrizes padrão de sistema).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método responsável por prover os padrões iniciais).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (assegurar que o objeto retornado contém os valores iniciais pré-estabelecidos).

08. Salvar_ErroNoBanco_LancaExcecao

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (configurar o mock para simular uma falha crítica ou de conexão durante a gravação).

- **Act**

- ↓
- Executar o comportamento que será testado (tentar salvar os parâmetros do sistema).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar que a exceção gerada pela falha de infraestrutura foi devidamente lançada).

09. Salvar_ParametrosValidos_SalvaComSucesso

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (instanciar um objeto de parâmetros do sistema preenchido com dados corretos).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método de salvamento de forma assíncrona).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (confirmar que os parâmetros foram persistidos com sucesso sem erros).

10. Salvar_RaioDeteccaoKinect_DentroDosLimitesPermitidos_SalvaComSucesso

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (definir um valor para o raio de detecção do Kinect que se encontra estritamente dentro dos limites aceitáveis de hardware).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método para salvar os parâmetros atualizados).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar que o valor válido foi aceito e gravado com sucesso).

11. Validar_RaioDeteccaoKinect_LimitesHardware (Teste estruturado em parâmetros/limites)

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (configurar parâmetros de validação para testar os limites mínimos e máximos tolerados pelo sensor Kinect).

- **Act**

-
- ↓
 - *Executar o comportamento que será testado (executar a rotina de validação do raio de detecção).*
 - **Assert**
 - ↓
 - *Verificar se o resultado corresponde ao esperado (garantir que as regras de restrição de hardware operam nos patamares corretos).*

12. Validar_ParametrosGerais_ConsistenciaIntegridade

- **Arrange**
 - ↓
 - *Preparar os dados e condições necessárias (estruturar um conjunto misto de parâmetros de configuração do sistema).*
- **Act**
 - ↓
 - *Executar o comportamento que será testado (acionar o método de checagem e integridade do repositório).*
- **Assert**
 - ↓
 - *Verificar se o resultado corresponde ao esperado (assegurar que todas as restrições e regras de negócio das configurações foram validadas sem inconsistências).*
- **Assert**
 - ↓
 - *Verificar se o resultado corresponde ao esperado (garantir que os parceiros retornados correspondem ao padrão textual pesquisado).*

Casos de Teste - NotificaçãoRepository

01. Adicionar_NotificacaoValida_AdicionaComSucesso

- **Arrange**
 - ↓
 - *Preparar os dados e condições necessárias (instanciar um objeto de notificação válido contendo mensagem, título, tipo e parâmetros obrigatórios).*

- **Act**

- ↓

- Executar o comportamento que será testado (chamar de forma assíncrona o método de adição da notificação no repositório).

- **Assert**

- ↓

- Verificar se o resultado corresponde ao esperado (assegurar que o registro foi inserido com sucesso sem disparar exceções).

02. Adicionar_NotificacaoComEmpresaInformada_MantemEmpresa

- **Arrange**

- ↓

- Preparar os dados e condições necessárias (configurar uma notificação vinculada explicitamente a um identificador de empresa válido).

- **Act**

- ↓

- Executar o comportamento que será testado (chamar o método para adicionar a notificação com a empresa definida).

- **Assert**

- ↓

- Verificar se o resultado corresponde ao esperado (confirmar que a associação com a empresa foi preservada e gravada corretamente).

03. Adicionar_ErroNoBanco_LancaExcecao

- **Arrange**

- ↓

-
- Preparar os dados e condições necessárias (configurar o mock de acesso a dados para simular uma falha de infraestrutura durante a inclusão).

- **Act**

- ↓
- Executar o comportamento que será testado (tentar adicionar a notificação sob a condição de erro de banco).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar que a exceção correspondente foi lançada adequadamente).

04. ListarTodos_ComRegistros_RetornaListaOrdenadaPorData

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (popular a base simulada com múltiplos registros de notificações em ordens de data variadas).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método de listagem geral de notificações).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (garantir que a coleção retornada está preenchida e ordenada cronologicamente por data).

05. ListarTodos_ErroNoBanco_RetornaListaVazia

- **Arrange**

- ↓

- *Preparar os dados e condições necessárias (configurar o repositório para simular uma falha de conexão na listagem geral).*

- **Act**

- ↓

- *Executar o comportamento que será testado (chamar o método de listagem sob erro de infraestrutura).*

- **Assert**

- ↓

- *Verificar se o resultado corresponde ao esperado (validar que a exceção é tratada de forma segura retornando uma lista vazia).*

06. ListarPorEmpresa_EmpresaEspecificada_RetornaNotificacoesDaEmpresa

- **Arrange**

- ↓

- *Preparar os dados e condições necessárias (fornecer um ID de empresa específico e estruturar registros associados a ela).*

- **Act**

- ↓

- *Executar o comportamento que será testado (chamar o método ListarPorEmpresa filtrando pelo identificador informado).*

- **Assert**

- ↓

- *Verificar se o resultado corresponde ao esperado (assegurar que a listagem contém exclusivamente as notificações pertencentes àquela empresa).*

07. ListarPorEmpresa_EmpresaPadrao_RetornaNotificacoesPadrao

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (configurar o cenário utilizando parâmetros de empresa padrão ou globais).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método de listagem correspondente à empresa padrão).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar que o retorno contempla corretamente as notificações padrão esperadas).

08. AtualizarStatus_IdValido_RetornaTrueComSucesso

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (informar o ID válido de uma notificação existente que terá seu status alterado).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método de atualização de status passando o identificador).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (confirmar que a operação retornou true e o status foi atualizado com sucesso).

09. AtualizarStatus_ErroNoBanco_RetornaFalse

- **Arrange**
 - ↓
 - Preparar os dados e condições necessárias (simular uma falha de banco de dados no momento da atualização de status).
- **Act**
 - ↓
 - Executar o comportamento que será testado (tentar atualizar o status da notificação sob falha de infraestrutura).
- **Assert**
 - ↓
 - Verificar se o resultado corresponde ao esperado (garantir que o método tratou a falha de forma segura retornando false).

10. ExisteNotificacaoPendente_ComPendenciaParaEmpresa_RetornaTrue

- **Arrange**
 - ↓
 - Preparar os dados e condições necessárias (garantir que existe ao menos uma notificação pendente cadastrada para a empresa testada).
- **Act**
 - ↓
 - Executar o comportamento que será testado (chamar o método de verificação de notificações pendentes).
- **Assert**
 - ↓

-
- *Verificar se o resultado corresponde ao esperado (validar que o retorno é true indicando a existência de pendências).*

11. ExisteNotificacaoPendente_SemPendencias_RetornaFalse

- **Arrange**

- ↓
- *Preparar os dados e condições necessárias (estruturar o cenário de testes sem nenhuma notificação pendente registrada).*

- **Act**

- ↓
- *Executar o comportamento que será testado (chamar o método de verificação de pendências).*

- **Assert**

- ↓
- *Verificar se o resultado corresponde ao esperado (certificar-se de que o método retorna false corretamente).*

12. ExisteNotificacaoPendente_ErroNoBanco_RetornaFalse

- **Arrange**

- ↓
- *Preparar os dados e condições necessárias (configurar o mock para lançar uma exceção de banco de dados durante a verificação de pendências).*

- **Act**

- ↓
- *Executar o comportamento que será testado (chamar a checagem de pendências sob falha de conexão).*

- **Assert**

○ ↓

- *Verificar se o resultado corresponde ao esperado (assegurar o tratamento seguro da exceção com o retorno de false).*

Casos de Teste - MedicaoVolumeRepository

01. Adicionar_MedicaoComEmpresaInformada_MantemEmpresa

- **Arrange**

- ↓

- *Preparar os dados e condições necessárias (configurar um objeto de medição de volume vinculado a um identificador específico de empresa).*

- **Act**

- ↓

- *Executar o comportamento que será testado (chamar o método para adicionar a medição com a empresa definida).*

- **Assert**

- ↓

- *Verificar se o resultado corresponde ao esperado (confirmar que a associação com a empresa foi preservada e gravada corretamente).*

02. Adicionar_MedicaoValida_AdicionaComSucesso

- **Arrange**

- ↓

- *Preparar os dados e condições necessárias (instanciar um objeto de medição de volume válido preenchido com todos os dados obrigatórios).*

- **Act**

- ↓

-
- Executar o comportamento que será testado (chamar de forma assíncrona o método de adição no repositório).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (assegurar que o registro foi inserido com sucesso sem disparar exceções).

03. Adicionar_ErroNoBanco_LancaExcecao

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (configurar o mock de acesso a dados para simular uma falha de infraestrutura durante a persistência).

- **Act**

- ↓
- Executar o comportamento que será testado (tentar adicionar a medição sob a condição de erro de banco).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar que a exceção correspondente foi lançada adequadamente).

04. FiltrarAvancado_PorOrigemEStatus_RetornaListaFiltrada

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (definir parâmetros específicos de filtro por origem dos dados e status da medição).

- **Act**

- ↓

- Executar o comportamento que será testado (chamar o método de filtro avançado passando os critérios configurados).

- **Assert**

- ↓

- Verificar se o resultado corresponde ao esperado (garantir que a listagem retornada atende estritamente aos parâmetros de origem e status informados).

05. FiltrarAvancado_PorPeriodoData_RetornaListaFiltrada

- **Arrange**

- ↓

- Preparar os dados e condições necessárias (configurar um intervalo de datas de início e fim para a busca).

- **Act**

- ↓

- Executar o comportamento que será testado (chamar o método de filtro avançado aplicando o período de tempo).

- **Assert**

- ↓

- Verificar se o resultado corresponde ao esperado (assegurar que a lista retornada contém apenas as medições registradas dentro da faixa de datas especificada).

06. ListarPorEmpresa_EmpresaEspecific_RetornaMedicoesFiltradas

- **Arrange**

- ↓

- Preparar os dados e condições necessárias (fornecer um ID de empresa específico e estruturar medições vinculadas a ele).

- **Act**

- ↓
- *Executar o comportamento que será testado (chamar o método `ListarPorEmpresa` filtrando pelo identificador informado).*

- **Assert**

- ↓
- *Verificar se o resultado corresponde ao esperado (validar que a coleção retornada contempla exclusivamente as medições daquela empresa).*

07. `ListarPorEmpresa_EmpresaPadrao_RetornaMedicoesGlobais`

- **Arrange**

- ↓
- *Preparar os dados e condições necessárias (configurar o cenário utilizando parâmetros de empresa padrão ou globais).*

- **Act**

- ↓
- *Executar o comportamento que será testado (chamar o método de listagem correspondente à empresa padrão).*

- **Assert**

- ↓
- *Verificar se o resultado corresponde ao esperado (garantir que o retorno contempla corretamente as medições globais ou padrão esperadas).*

08. `ListarTodos_ComRegistros_RetornaListaPreenchida`

- **Arrange**

- ↓

- Preparar os dados e condições necessárias (popular a base simulada com múltiplos registros de medições de volume).

- **Act**

- ↓

- Executar o comportamento que será testado (chamar o método de listagem geral `ListarTodos`).

- **Assert**

- ↓

- Verificar se o resultado corresponde ao esperado (assegurar que a lista retornada não é nula e contém todos os registros cadastrados).

09. `ListarTodos_ErroNoBanco_RetornaListaVazia`

- **Arrange**

- ↓

- Preparar os dados e condições necessárias (configurar o repositório para simular uma falha de conexão durante a listagem geral).

- **Act**

- ↓

- Executar o comportamento que será testado (chamar o método de listagem sob erro de infraestrutura).

- **Assert**

- ↓

- Verificar se o resultado corresponde ao esperado (validar que a exceção foi tratada de forma segura retornando uma lista vazia).

10. `ObterSummary_ComMedicoes_RetornaEstatisticasCorretas`

- **Arrange**

- ↓

-
- Preparar os dados e condições necessárias (popular o repositório com um conjunto conhecido de medições para cálculo estatístico).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método `ObterSummary` para consolidação dos dados).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar que o resumo estatístico reflete corretamente os valores e somatórias esperados).

11. `ObterSummary_SemMedicoes_RetornaSummaryZerado`

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (configurar a base sem nenhum registro de medição cadastrado).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método `ObterSummary` em um cenário vazio).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (certificar-se de que o objeto de resumo retornado possui todas as métricas zeradas de forma segura).

Casos de Teste - LogsSistemaRepository

01. `Registrar_EmpresaldNuloOuBranco_ObtemEmpresaDoContexto`

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (configurar um cenário onde o registro de log é chamado sem passar um ID de empresa explícito, estando nulo ou em branco, e o contexto de usuário efetivo possui uma empresa padrão).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método de registro/adição do log).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (assegurar que o repositório recuperou e aplicou corretamente a empresa obtida a partir do contexto atual).

02. Registrar_ErroNoBanco_CapturaExcecaoELogaErroSemLancar

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (configurar o mock de acesso a dados para simular uma exceção ou falha de infraestrutura durante a gravação do log).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método de registro de log sob a condição de erro de banco).

- **Assert**

- ↓

-
- *Verificar se o resultado corresponde ao esperado (validar que o repositório capturou a exceção com segurança, registrou internamente o erro e evitou propagar a falha para a aplicação).*

03. Registrar_ParametrosValidosComEmpresaInformado_AdicionaComSucesso

- **Arrange**

- ↓
- *Preparar os dados e condições necessárias (instanciar os parâmetros válidos para o log, incluindo uma empresa explicitamente informada).*

- **Act**

- ↓
- *Executar o comportamento que será testado (chamar o método de registro com os dados preenchidos).*

- **Assert**

- ↓
- *Verificar se o resultado corresponde ao esperado (confirmar que o log foi gravado com sucesso associado à empresa informada).*

EmpresasRepository

01. ListarTodas_ComEmpresasCadastradas_RetornaListaPreenchida

- **Arrange**

- ↓
- *Preparar os dados e condições necessárias (popular a base de dados simulada com múltiplos registros de empresas válidas e ativas).*

- **Act**

- ↓
- *Executar o comportamento que será testado (chamar o método de listagem geral ListarTodas do repositório).*

- **Assert**

- ↓
- *Verificar se o resultado corresponde ao esperado (assegurar que a coleção retornada não é nula, contém todos os registros cadastrados e está devidamente preenchida).*

02. ListarTodas_BancoVazio_RetornaListaVazia

- **Arrange**

- ↓
- *Preparar os dados e condições necessárias (garantir que o ambiente de banco de dados simulado esteja completamente vazio, sem nenhuma empresa registrada).*

- **Act**

- ↓
- *Executar o comportamento que será testado (chamar o método ListarTodas sob o cenário sem registros).*

- **Assert**

- ↓
- *Verificar se o resultado corresponde ao esperado (validar que o retorno é uma lista vazia, evitando exceções de referência nula).*

03. ListarTodas_ErroNoBanco_CapturaExcecaoELogaErroRetornandoListaVazia

- **Arrange**

- ↓
- *Preparar os dados e condições necessárias (configurar o mock de acesso a dados para simular uma falha de infraestrutura ou exceção de conexão durante a listagem).*

- **Act**

- ↓

-
- Executar o comportamento que será testado (chamar o método *ListarTodas* sob a condição de erro de banco).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (assegurar que a exceção foi tratada com segurança, o erro foi registrado e o método retornou uma lista vazia de forma controlada).

04. *BuscarPorId_IdExistente_RetornaEmpresaComId*

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (fornecer o ID de uma empresa válida e cadastrada previamente no repositório).

- **Act**

- ↓
- Executar o comportamento que será testado (chamar o método *BuscarPorId* passando o identificador informado).

- **Assert**

- ↓
- Verificar se o resultado corresponde ao esperado (validar que o objeto retornado não é nulo e possui exatamente o ID e os dados correspondentes à empresa buscada).

05. *BuscarPorId_IdInexistente_RetornaNull*

- **Arrange**

- ↓
- Preparar os dados e condições necessárias (definir um identificador de ID que não existe na base de dados).

- **Act**

○ ↓

- *Executar o comportamento que será testado (chamar o método `BuscarPorId` buscando pelo ID inválido/inexistente).*

- **Assert**

- ↓

- *Verificar se o resultado corresponde ao esperado (certificar-se de que o método retorna null adequadamente sem disparar erros).*

06. `BuscarPorId_ErroNoBanco_CapturaExcecaoELogaErroRetornandoNull`

- **Arrange**

- ↓

- *Preparar os dados e condições necessárias (configurar o mock para simular uma falha crítica ou exceção de infraestrutura durante a consulta por ID).*

- **Act**

- ↓

- *Executar o comportamento que será testado (chamar o método `BuscarPorId` sob a condição de erro de banco).*

- **Assert**

- ↓

- *Verificar se o resultado corresponde ao esperado (garantir que a exceção foi capturada de forma segura, o erro foi logado e o método retornou null sem quebrar a aplicação).*

3. Evidências da execução

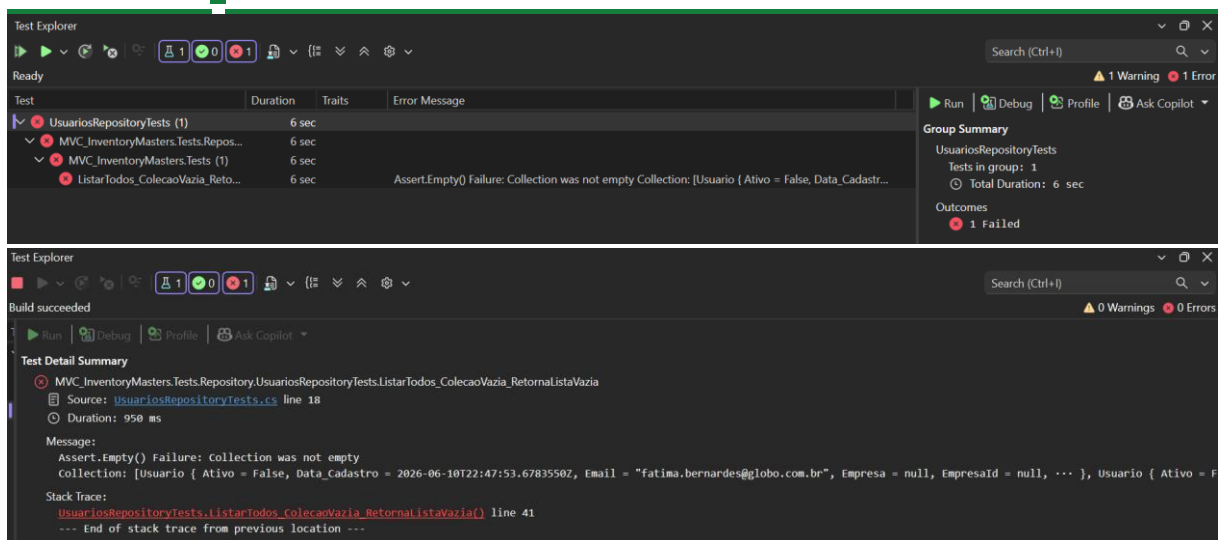
Componente: `UsuariosRepositoryTests`

`ListarTodos_ColecaoVazia_RetornaListaVazia`

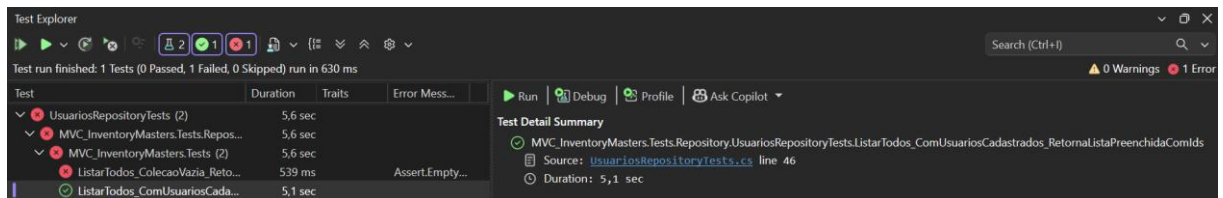


INVENTORY MASTERS

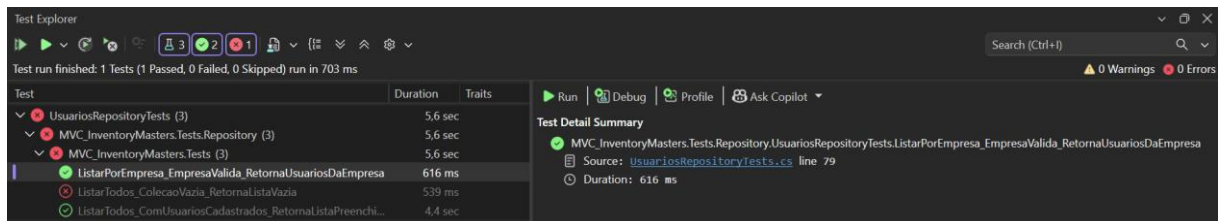
Gestão de Estoque Inteligente



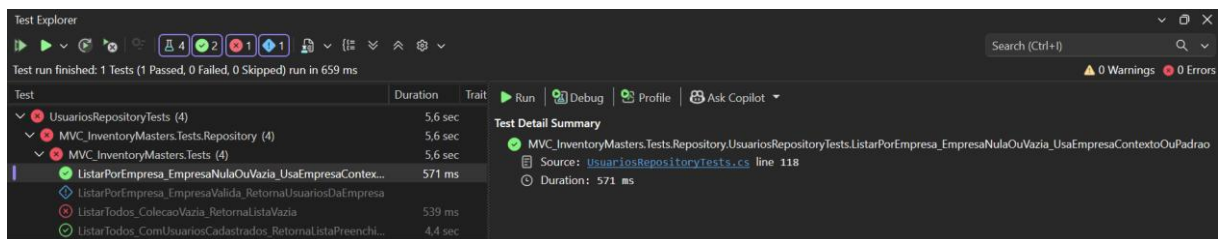
ListarTodos_ComUsuariosCadastrados_RetornaListaPreenchidaComIds



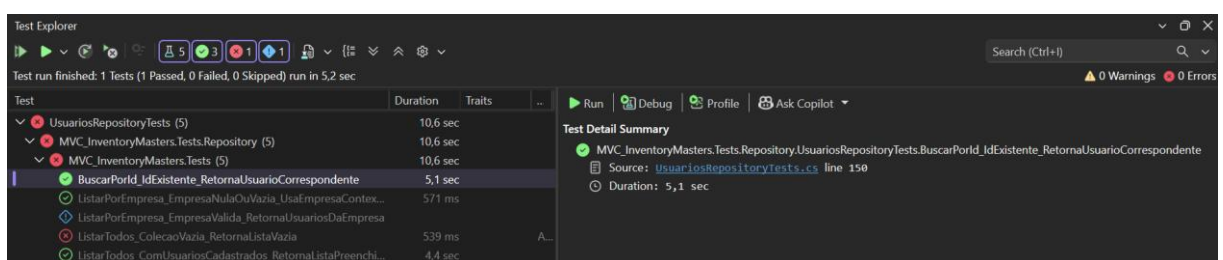
ListarPorEmpresa_EmpresaValida_RetornaUsuariosDaEmpresa



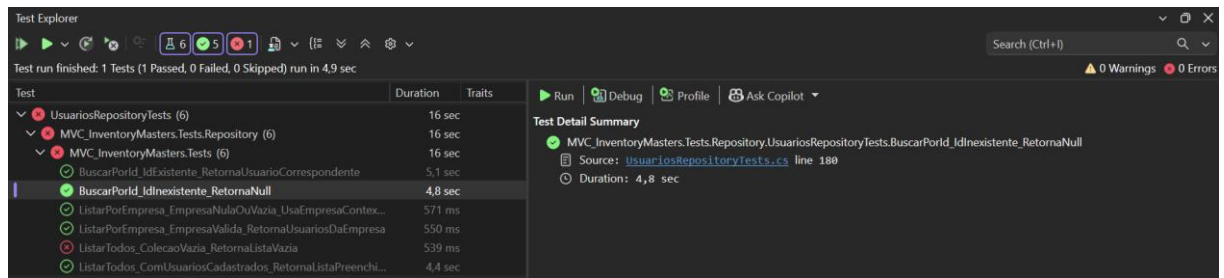
ListarPorEmpresa_EmpresaNulaOuVazia_UsaEmpresaContextoOuPadrao



BuscarPorId_IdExistente_RetornaUsuarioCorrespondente



BuscarPorId_IdInexistente_RetornaNull



Test Explorer

Test run finished: 1 Tests (1 Passed, 0 Failed, 0 Skipped) run in 4,9 sec

Test	Duration	Traits
UsuariosRepositoryTests (6)	16 sec	
MVC_InventoryMasters.Tests.Repository (6)	16 sec	
MVC_InventoryMasters.Tests (6)	16 sec	
BuscarPorId_IdExistente_RetornaUsuarioCorrespondente	5,1 sec	
BuscarPorId_IdInexistente_RetornaNull	4,8 sec	
ListarPorEmpresa_EmpresaNulaOuVazia_UsaEmpresaContexto	571 ms	
ListarPorEmpresa_EmpresaValida_RetornaUsuariosDaEmpresa	550 ms	
ListarTodos_ColecaoVazia_RetornaListaVazia	539 ms	
ListarTodos_ComUsuariosCadastrados_RetornaListaPreenchida	4,4 sec	

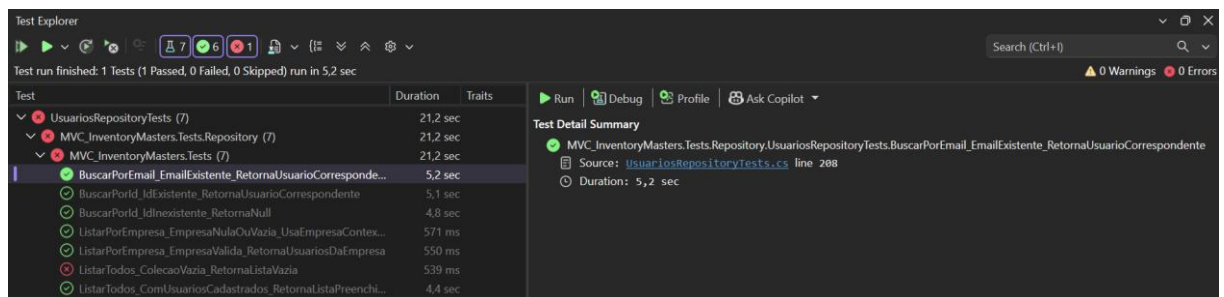
Test Detail Summary

✓ MVC_InventoryMasters.Tests.Repository UsuariosRepositoryTests.BuscarPorId_IdInexistente_RetornaNull

Source: [UsuariosRepositoryTests.cs](#) line 180

⌚ Duration: 4,8 sec

BuscarPorEmail_EmailExistente_RetornaUsuarioCorrespondente



Test Explorer

Test run finished: 1 Tests (1 Passed, 0 Failed, 0 Skipped) run in 5,2 sec

Test	Duration	Traits
UsuariosRepositoryTests (7)	21,2 sec	
MVC_InventoryMasters.Tests.Repository (7)	21,2 sec	
MVC_InventoryMasters.Tests (7)	21,2 sec	
BuscarPorEmail_EmailExistente_RetornaUsuarioCorrespondente	5,2 sec	
BuscarPorId_IdExistente_RetornaUsuarioCorrespondente	5,1 sec	
BuscarPorId_IdInexistente_RetornaNull	4,8 sec	
ListarPorEmpresa_EmpresaNulaOuVazia_UsaEmpresaContexto	571 ms	
ListarPorEmpresa_EmpresaValida_RetornaUsuariosDaEmpresa	550 ms	
ListarTodos_ColecaoVazia_RetornaListaVazia	539 ms	
ListarTodos_ComUsuariosCadastrados_RetornaListaPreenchida	4,4 sec	

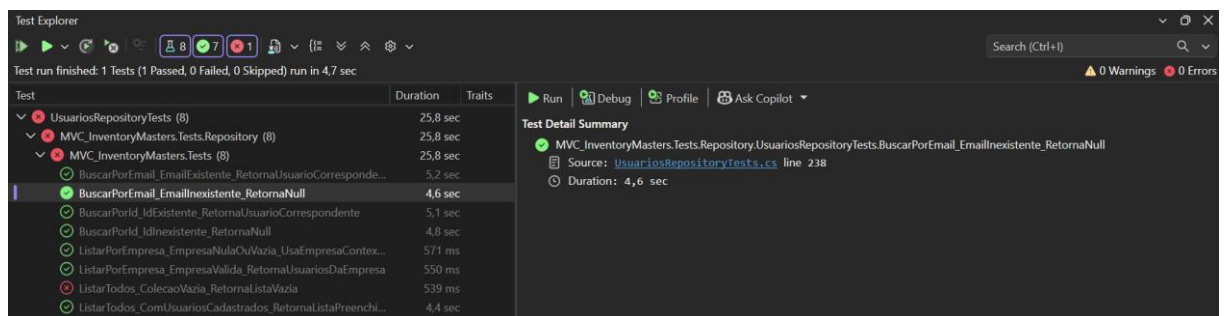
Test Detail Summary

✓ MVC_InventoryMasters.Tests.Repository UsuariosRepositoryTests.BuscarPorEmail_EmailExistente_RetornaUsuarioCorrespondente

Source: [UsuariosRepositoryTests.cs](#) line 208

⌚ Duration: 5,2 sec

BuscarPorEmail_EmailInexistente_RetornaNull



Test Explorer

Test run finished: 1 Tests (1 Passed, 0 Failed, 0 Skipped) run in 4,7 sec

Test	Duration	Traits
UsuariosRepositoryTests (8)	25,8 sec	
MVC_InventoryMasters.Tests.Repository (8)	25,8 sec	
MVC_InventoryMasters.Tests (8)	25,8 sec	
BuscarPorEmail_EmailExistente_RetornaUsuarioCorrespondente	5,2 sec	
BuscarPorEmail_EmailInexistente_RetornaNull	4,6 sec	
BuscarPorId_IdExistente_RetornaUsuarioCorrespondente	5,1 sec	
BuscarPorId_IdInexistente_RetornaNull	4,8 sec	
ListarPorEmpresa_EmpresaNulaOuVazia_UsaEmpresaContexto	571 ms	
ListarPorEmpresa_EmpresaValida_RetornaUsuariosDaEmpresa	550 ms	
ListarTodos_ColecaoVazia_RetornaListaVazia	539 ms	
ListarTodos_ComUsuariosCadastrados_RetornaListaPreenchida	4,4 sec	

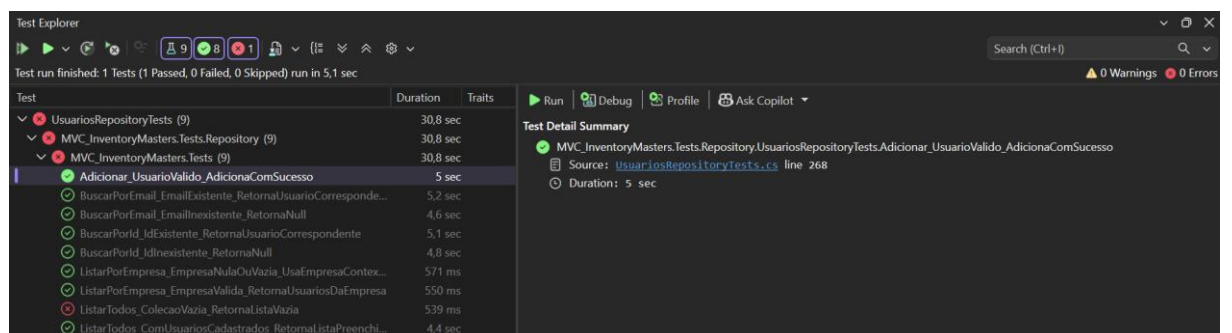
Test Detail Summary

✓ MVC_InventoryMasters.Tests.Repository UsuariosRepositoryTests.BuscarPorEmail_EmailInexistente_RetornaNull

Source: [UsuariosRepositoryTests.cs](#) line 238

⌚ Duration: 4,6 sec

Adicionar_UsuarioValido_AdicionaComSucesso



Test Explorer

Test run finished: 1 Tests (1 Passed, 0 Failed, 0 Skipped) run in 5,1 sec

Test	Duration	Traits
UsuariosRepositoryTests (9)	30,8 sec	
MVC_InventoryMasters.Tests.Repository (9)	30,8 sec	
MVC_InventoryMasters.Tests (9)	30,8 sec	
Adicionar_UsuarioValido_AdicionaComSucesso	5 sec	
BuscarPorEmail_EmailExistente_RetornaUsuarioCorrespondente	5,2 sec	
BuscarPorEmail_EmailInexistente_RetornaNull	4,6 sec	
BuscarPorId_IdExistente_RetornaUsuarioCorrespondente	5,1 sec	
BuscarPorId_IdInexistente_RetornaNull	4,8 sec	
ListarPorEmpresa_EmpresaNulaOuVazia_UsaEmpresaContexto	571 ms	
ListarPorEmpresa_EmpresaValida_RetornaUsuariosDaEmpresa	550 ms	
ListarTodos_ColecaoVazia_RetornaListaVazia	539 ms	
ListarTodos_ComUsuariosCadastrados_RetornaListaPreenchida	4,4 sec	

Test Detail Summary

✓ MVC_InventoryMasters.Tests.Repository UsuariosRepositoryTests.Adicionar_UsuarioValido_AdicionaComSucesso

Source: [UsuariosRepositoryTests.cs](#) line 268

⌚ Duration: 5 sec

Adicionar_ErroNoBanco_LancaExcecao

Test Explorer			
Test run finished: 1 Tests (0 Passed, 1 Failed, 0 Skipped) run in 4,4 sec			
Test	Duration	Traits	Error Message
❌ UsuariosRepositoryTests (10)	35,1 sec		
❌ MVC_InventoryMasters.Tests.Repository (10)	35,1 sec		
❌ MVC_InventoryMasters.Tests (10)	35,1 sec		
❌ Adicionar_ErroNoBanco_LancaExeccao	4,3 sec		Assert.Throws() Failure: Exception type was not an exact match Expected: type of(System.ArgumentNullException) Actual: type of(...
✅ Adicionar_UsuarioValido_AdicionaComSucesso	5 sec		
✅ BuscarPorEmail_EmailExistente_RetornaUsuarioCorresponde...	5,2 sec		
✅ BuscarPorEmail_EmailInexistente_RetornaNull	4,6 sec		
✅ BuscarPorId_IdExistente_RetornaUsuarioCorrespondente	5,1 sec		
✅ BuscarPorId_IdInexistente_RetornaNull	4,8 sec		
✅ ListarPorEmpresa_EmpresaNulaOuVazia_UsaEmpresaContex...	571 ms		
✅ ListarPorEmpresa_EmpresaValida_RetornaUsuariosDaEmpresa	550 ms		
❌ ListarTodos_ColecaoVazia_RetornaListaVazia	539 ms		Assert.Empty() Failure: Collection was not empty Collection: [Usuario (Ativo = False, Data_Cadastro = 2026-06-10T22:47:53.67835...
✅ ListarTodos_ComUsuariosCadastrados_RetornaListaPreenchi...	4,4 sec		

Test Explorer			
Test run finished: 1 Tests (0 Passed, 1 Failed, 0 Skipped) run in 4,4 sec			
Test	Duration	Traits	Error Message
❌ UsuariosRepositoryTests (10)	35,1 sec		
❌ MVC_InventoryMasters.Tests.Repository (10)	35,1 sec		
❌ MVC_InventoryMasters.Tests (10)	35,1 sec		
❌ Adicionar_ErroNoBanco_LancaExeccao	4,3 sec		Assert.Throws0...
✅ Adicionar_UsuarioValido_AdicionaComSucesso	5 sec		
✅ BuscarPorEmail_EmailExistente_RetornaUsuarioCorresponde...	5,2 sec		
✅ BuscarPorEmail_EmailInexistente_RetornaNull	4,6 sec		
✅ BuscarPorId_IdExistente_RetornaUsuarioCorrespondente	5,1 sec		
✅ BuscarPorId_IdInexistente_RetornaNull	4,8 sec		
✅ ListarPorEmpresa_EmpresaNulaOuVazia_UsaEmpresaContex...	571 ms		
✅ ListarPorEmpresa_EmpresaValida_RetornaUsuariosDaEmpresa	550 ms		
❌ ListarTodos_ColecaoVazia_RetornaListaVazia	539 ms		Assert.Empty() F...
✅ ListarTodos_ComUsuariosCadastrados_RetornaListaPreenchi...	4,4 sec		

Test Detail Summary

❌ MVC_InventoryMasters.Tests.Repository.UsuariosRepositoryTests.Adicionar_ErroNoBanco_LancaExeccao

Source: [UsuariosRepositoryTests.cs](#) line 305

Duration: 4,3 sec

Message:

Assert.Throws() Failure: Exception type was not an exact match
Expected: type of(System.ArgumentNullException)
Actual: type of(System.Exception)
---- System.Exception : Não foi possível cadastrar o usuário.

Stack Trace:

[UsuariosRepositoryTests.Adicionar_ErroNoBanco_LancaExeccao\(\)](#) line 325
--- End of stack trace from previous location ---
----- Inner Stack Trace -----
[UsuariosRepository.Adicionar\(Usuario usuario\)](#) line 214
[Adicionar_ErroNoBanco_LancaExeccao>b__2d.MoveNext\(\)](#) line 327
--- End of stack trace from previous location ---

Test Explorer			
Test run finished: 1 Tests (1 Passed, 0 Failed, 0 Skipped) run in 5 sec			
Test	Duration	Traits	
✅ EmpresasRepositoryTests (6)	2,9 sec		
✅ KinectRepositoryTests (14)	64 ms		
✅ LogsSistemaRepositoryTests (3)	2,9 sec		
✅ MedicacaoVolumeRepositoryTests (11)	6,2 sec		
✅ NotificacaoRepositoryTests (9)	5,4 sec		
✅ ParametrosSistemaRepositoryTests (12)	6,5 sec		
✅ ParcelasRepositoryTests (12)	6,9 sec		
✅ PerfilRepositoryTests (10)	6,3 sec		
✅ TokensAcessoKinectRepositoryTests (6)	5,5 sec		
✅ UsuariosRepositoryTests (15)	11,6 sec		
❌ MVC_InventoryMasters.Tests.Repository (15)	11,6 sec		
❌ MVC_InventoryMasters.Tests (15)	11,6 sec		
✅ Adicionar_ErroNoBanco_LancaExeccao	4,5 sec		
✅ Adicionar_UsuarioValido_AdicionaComSucesso	87 ms		
✅ Atualizar_ErroDuranteAtualizacao_LancaExeccao	2 ms		
✅ Atualizar_UsuarioExistenteComDadosValidos_AtualizaComSucesso	366 ms		
✅ AtualizarStatus_IdValidoENovoStatus_AtualizaStatusComSucesso	281 ms		
✅ BuscarPorEmail_EmailExistente_RetornaUsuarioCorrespondente	75 ms		
✅ BuscarPorEmail_EmailInexistente_RetornaNull	470 ms		
✅ BuscarPorId_IdExistente_RetornaUsuarioCorrespondente	186 ms		
✅ BuscarPorId_IdInexistente_RetornaNull	37 ms		
✅ Excluir_IdDeUsuarioExistente_RemoveComSucesso	564 ms		
✅ Excluir_IdInexistente_NaoLancaExeccao	4,8 sec		
✅ ListarPorEmpresa_EmpresaNulaOuVazia_UsaEmpresaContextoOuPadrao	79 ms		
✅ ListarPorEmpresa_EmpresaValida_RetornaUsuariosDaEmpresa	45 ms		
❌ ListarTodos_ColecaoVazia_RetornaListaVazia	37 ms		
✅ ListarTodos_ComUsuariosCadastrados_RetornaListaPreenchidaComIds	51 ms		

Atualizar_UsuarioExistenteComDadosValidos_AtualizaComSucesso



INVENTORY MASTERS

Gestão de Estoque Inteligente

Test Explorer

Test run finished: 1 Tests (1 Passed, 0 Failed, 0 Skipped) run in 5,1 sec

Test	Duration	Traits
UsuariosRepositoryTests (11)	40,1 sec	
MVC_InventoryMasters.Tests.Repository (11)	40,1 sec	
MVC_InventoryMasters.Tests (11)	40,1 sec	
Adicionar_ErroNoBanco_LancaExeccao	4,3 sec	
Adicionar_UsuarioValido_AdicionaComSucesso	5 sec	
Atualizar_UsuarioExistenteComDadosValidos_AtualizaComS...	5 sec	
BuscarPorEmail_EmailExistente_RetornaUsuarioCorresponde...	5,2 sec	
BuscarPorEmail_EmailInexistente_RetornaNull	4,6 sec	
BuscarPorId_IdExistente_RetornaUsuarioCorrespondente	5,1 sec	
BuscarPorId_IdInexistente_RetornaNull	4,8 sec	
ListarPorEmpresa_EmpresaNulaOuVazia_UsaEmpresaContex...	571 ms	
ListarPorEmpresa_EmpresaValida_RetornaUsuariosDaEmpresa	550 ms	
ListarTodos_ColecaoVazia_RetornaListaVazia	539 ms	
ListarTodos_ComUsuariosCadastrados_RetornaListaPreenchi...	4,4 sec	

Test Detail Summary

MVC_InventoryMasters.Tests.Repository.UsuariosRepositoryTests.Atualizar_UsuarioExistenteComDadosValidos_AtualizaComSucesso

Source: UsuariosRepositoryTests.cs line 333

Duration: 5 sec

Atualizar_ErroDuranteAtualizacao_LancaExeccao

Test Explorer

Test run finished: 1 Tests (0 Passed, 1 Failed, 0 Skipped) run in 4,4 sec

Test	Duration	Traits
UsuariosRepositoryTests (12)	44,4 sec	
MVC_InventoryMasters.Tests.Repository (12)	44,4 sec	
MVC_InventoryMasters.Tests (12)	44,4 sec	
Adicionar_ErroNoBanco_LancaExeccao	4,3 sec	
Adicionar_UsuarioValido_AdicionaComSucesso	5 sec	
Atualizar_ErroDuranteAtualizacao_LancaExeccao	4,3 sec	
Atualizar_UsuarioExistenteComDadosValidos_AtualizaComS...	5 sec	
BuscarPorEmail_EmailExistente_RetornaUsuarioCorresponde...	5,2 sec	
BuscarPorEmail_EmailInexistente_RetornaNull	4,6 sec	
BuscarPorId_IdExistente_RetornaUsuarioCorrespondente	5,1 sec	
BuscarPorId_IdInexistente_RetornaNull	4,8 sec	
ListarPorEmpresa_EmpresaNulaOuVazia_UsaEmpresaContex...	571 ms	
ListarPorEmpresa_EmpresaValida_RetornaUsuariosDaEmpresa	550 ms	
ListarTodos_ColecaoVazia_RetornaListaVazia	539 ms	
ListarTodos_ComUsuariosCadastrados_RetornaListaPreenchi...	4,4 sec	

Test Detail Summary

MVC_InventoryMasters.Tests.Repository.UsuariosRepositoryTests.Atualizar_ErroDuranteAtualizacao_LancaExeccao

Source: UsuariosRepositoryTests.cs line 369

Duration: 4,3 sec

Message:

Assert.Throws() Failure: Exception type was not an exact match

Expected: typeof(System.Exception)

Actual: typeof(System.NullReferenceException)

---- System.NullReferenceException : Object reference not set to an instance of an object.

Stack Trace:

UsuariosRepositoryTests.Atualizar_ErroDuranteAtualizacao_LancaExeccao() line 387

--- End of stack trace from previous location ---

----- Inner Stack Trace -----

UsuariosRepository.Atualizar(Usuario usuario) line 250

<<Atualizar_ErroDuranteAtualizacao_LancaExeccao>>.MoveNext() line 389

--- End of stack trace from previous location ---

Test Explorer

Test run finished: 1 Tests (1 Passed, 0 Failed, 0 Skipped) run in 319 ms

Test	Duration	Traits
EmpresasRepositoryTests (6)	2,9 sec	
KinectRepositoryTests (14)	64 ms	
LogsSistemaRepositoryTests (3)	2,9 sec	
MedicaoVolumeRepositoryTests (11)	6,2 sec	
NotificacaoRepositoryTests (9)	5,4 sec	
ParametrosSistemaRepositoryTests (12)	6,5 sec	
ParceirosRepositoryTests (12)	6,9 sec	
PerfisRepositoryTests (10)	6,3 sec	
TokensAcessoKinectRepositoryTests (6)	5,5 sec	
UsuariosRepositoryTests (15)	11,8 sec	
MVC_InventoryMasters.Tests.Repository (15)	11,8 sec	
MVC_InventoryMasters.Tests (15)	11,8 sec	
Adicionar_ErroNoBanco_LancaExeccao	4,5 sec	
Adicionar_UsuarioValido_AdicionaComSucesso	87 ms	
Atualizar_ErroDuranteAtualizacao_LancaExeccao	207 ms	
Atualizar_UsuarioExistenteComDadosValidos_AtualizaComSucesso	366 ms	
AtualizarStatus_IdValidoENovoStatus_AtualizaStatusComSucesso	281 ms	
BuscarPorEmail_EmailExistente_RetornaUsuarioCorrespondente	75 ms	
BuscarPorEmail_EmailInexistente_RetornaNull	470 ms	
BuscarPorId_IdExistente_RetornaUsuarioCorrespondente	186 ms	
BuscarPorId_IdInexistente_RetornaNull	37 ms	
Excluir_IdDeUsuarioExistente_RemoveComSucesso	564 ms	
Excluir_IdInexistente_NaoLancaExeccao	4,8 sec	
ListarPorEmpresa_EmpresaNulaOuVazia_UsaEmpresaContextoOuPadrao	79 ms	
ListarPorEmpresa_EmpresaValida_RetornaUsuariosDaEmpresa	45 ms	
ListarTodos_ColecaoVazia_RetornaListaVazia	37 ms	
ListarTodos_ComUsuariosCadastrados_RetornaListaPreenchidaComIds	51 ms	

Test Detail Summary

MVC_InventoryMasters.Tests.Repository.UsuariosRepositoryTests.Atualizar_ErroDuranteAtualizacao_LancaExeccao

Source: UsuariosRepositoryTests.cs line 369

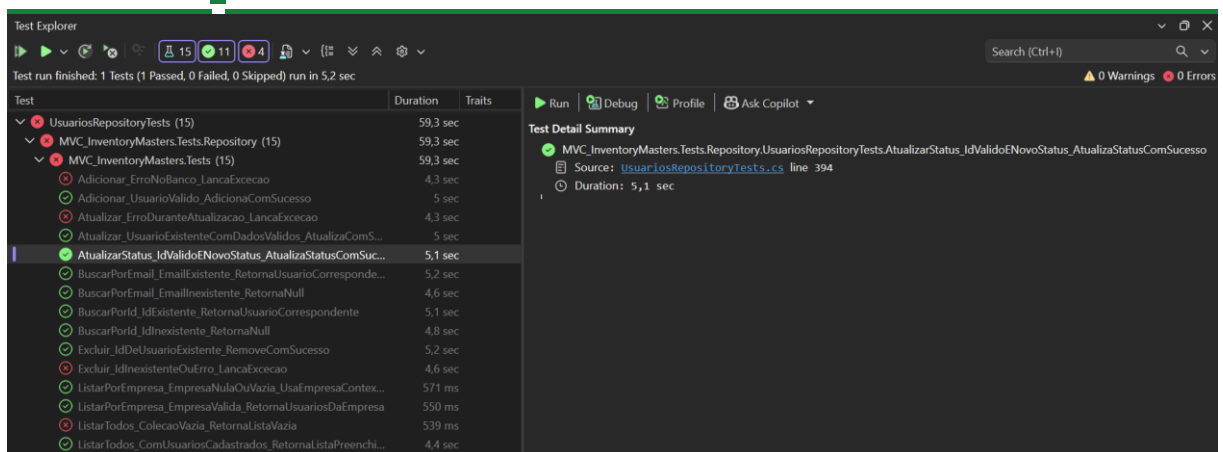
Duration: 207 ms

AtualizarStatus_IdValidoENovoStatus_AtualizaStatusComSucesso

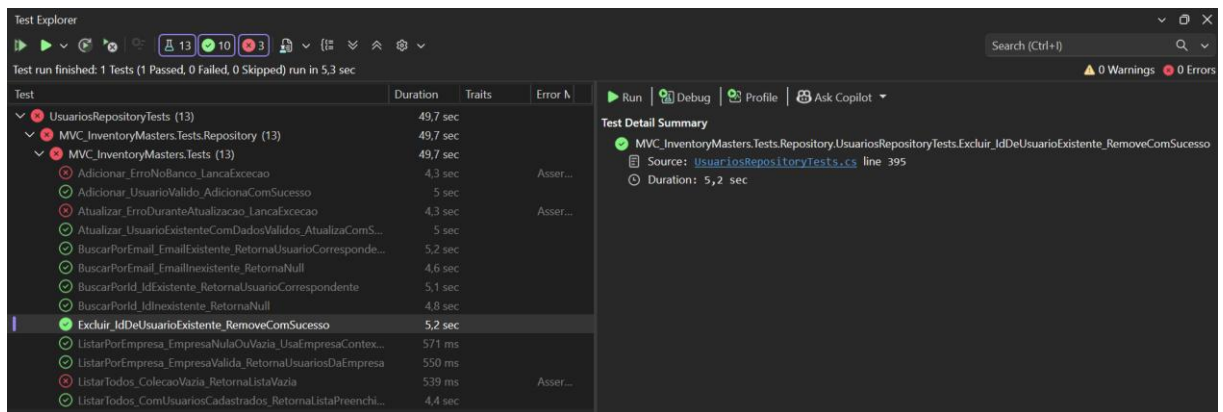


INVENTORY MASTERS

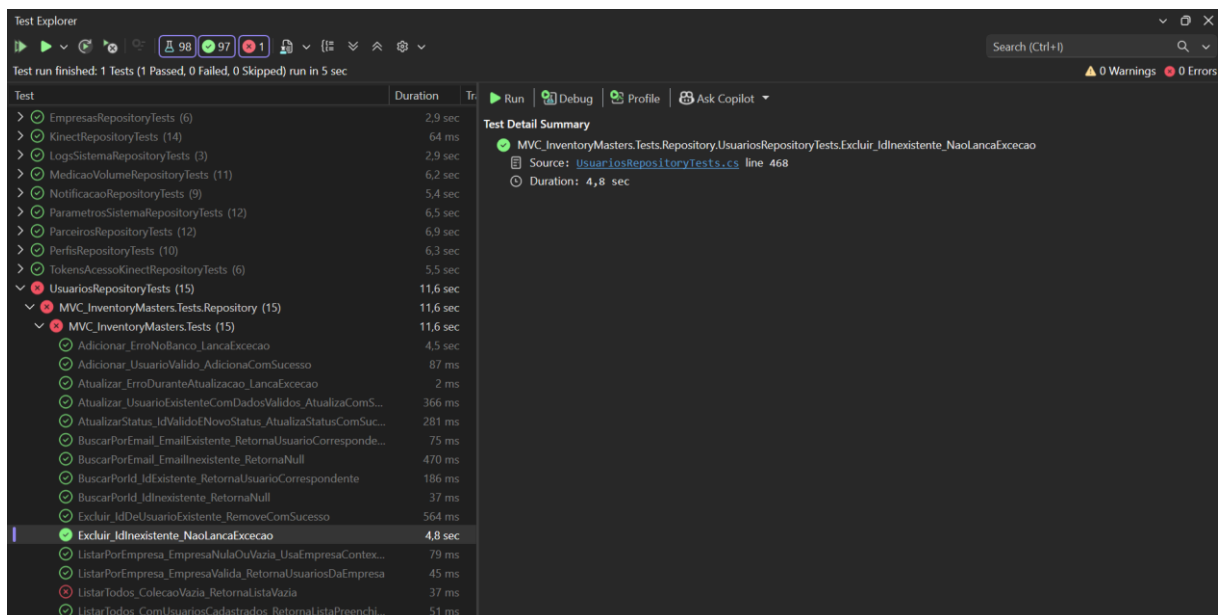
Gestão de Estoque Inteligente



Excluir_IdDeUsuarioExistente_RemoveComSucesso



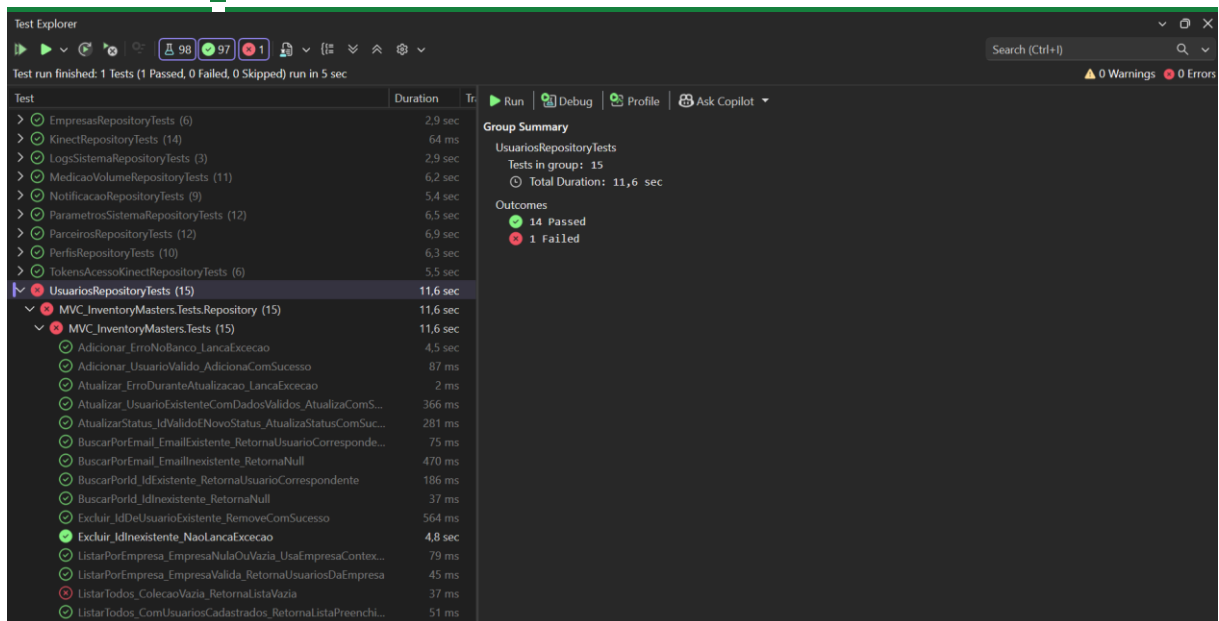
Excluir_IdInexistente_NaoLancaExcecao





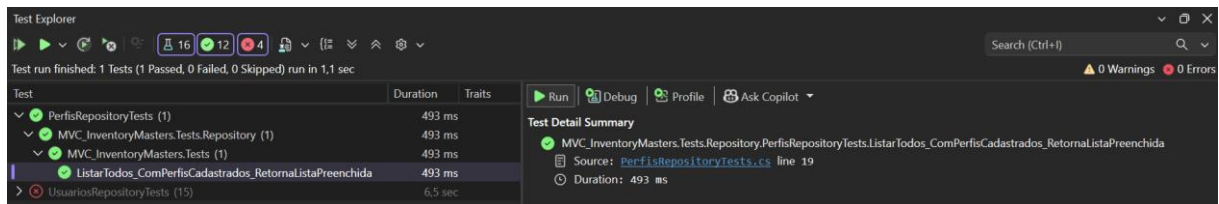
INVENTORY MASTERS

Gestão de Estoque Inteligente

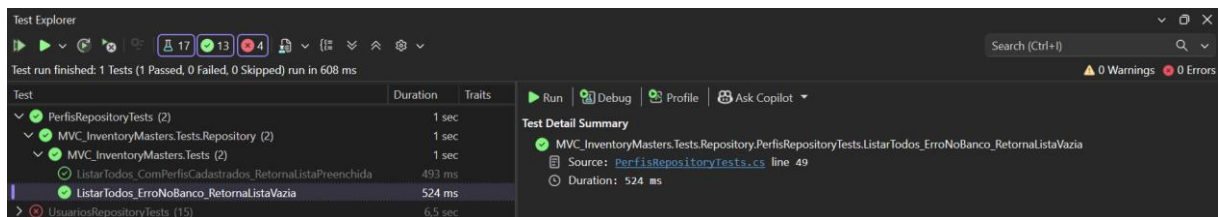


Componente: PerfisRepositoryTests

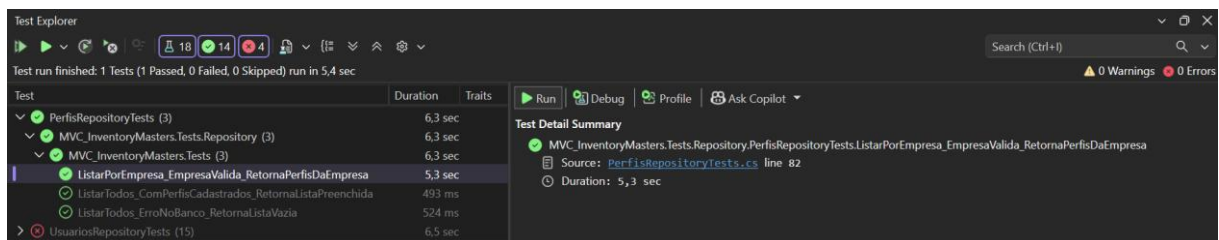
CT01 - ListarTodos_ComPerfisCadastrados_RetornaListaPreenchida



CT02 - ListarTodos_ErroNoBanco_RetornaListaVazia



CT03 - ListarPorEmpresa_EmpresaValida_RetornaPerfisDaEmpresa

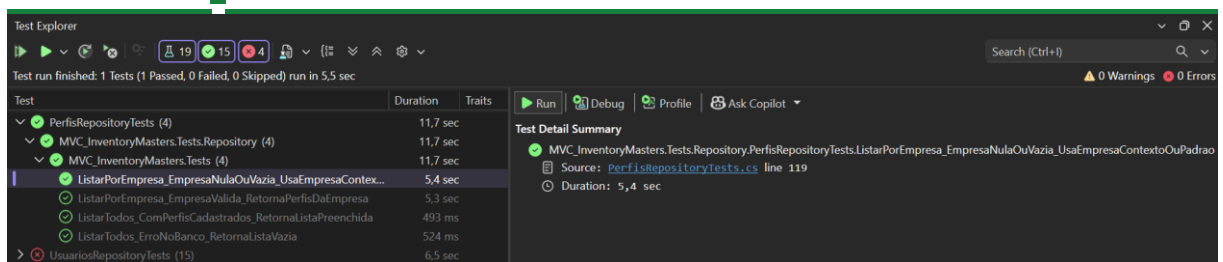


CT04 - ListarPorEmpresa_EmpresaNulaOuVazia_UsaEmpresaContextoOuPadrao

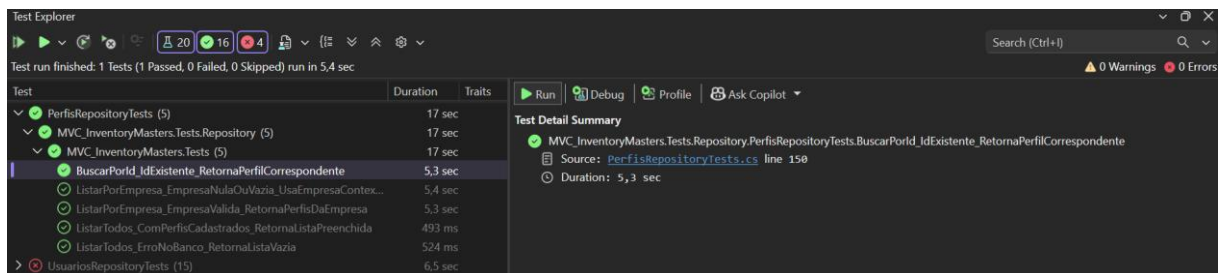


INVENTORY MASTERS

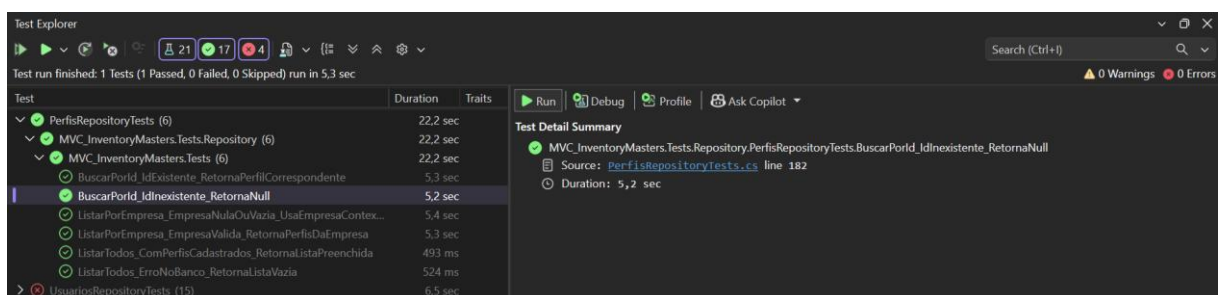
Gestão de Estoque Inteligente



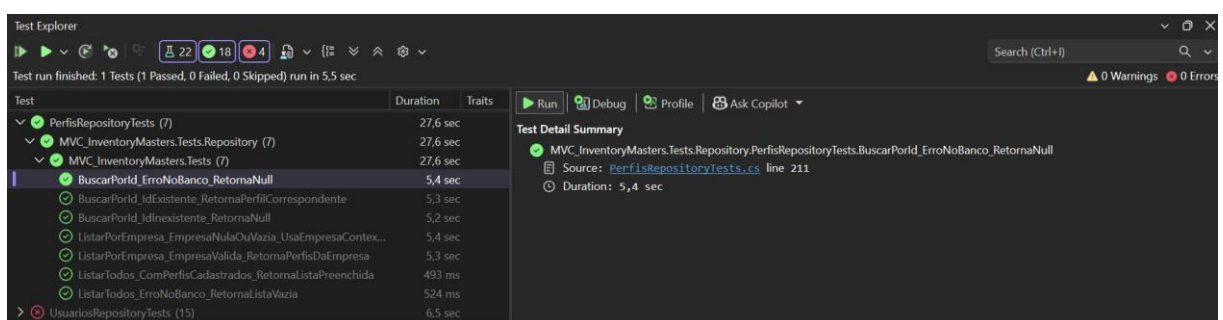
CT05 - BuscarPorId_IdExistente_RetornaPerfilCorrespondente



CT06 - BuscarPorId_IdInexistente_RetornaNull



CT07 - BuscarPorId_ErroNoBanco_RetornaNull

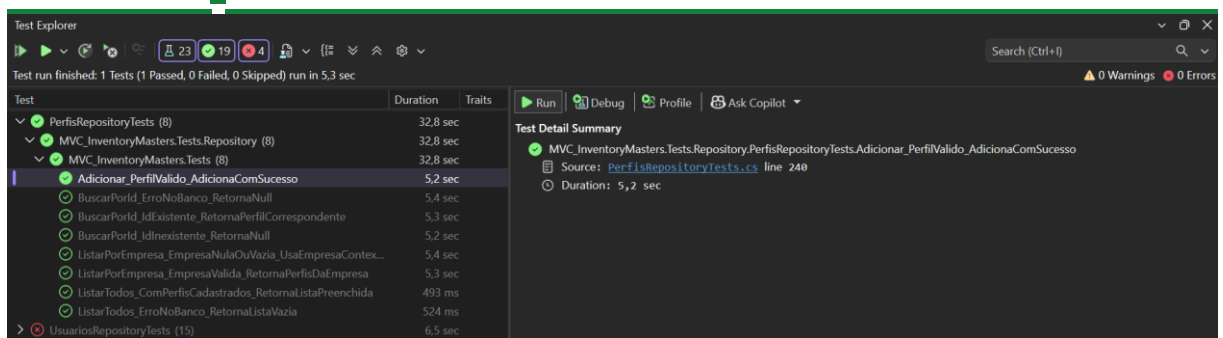


CT08 - Adicionar_PerfilValido_AdicionaComSucesso

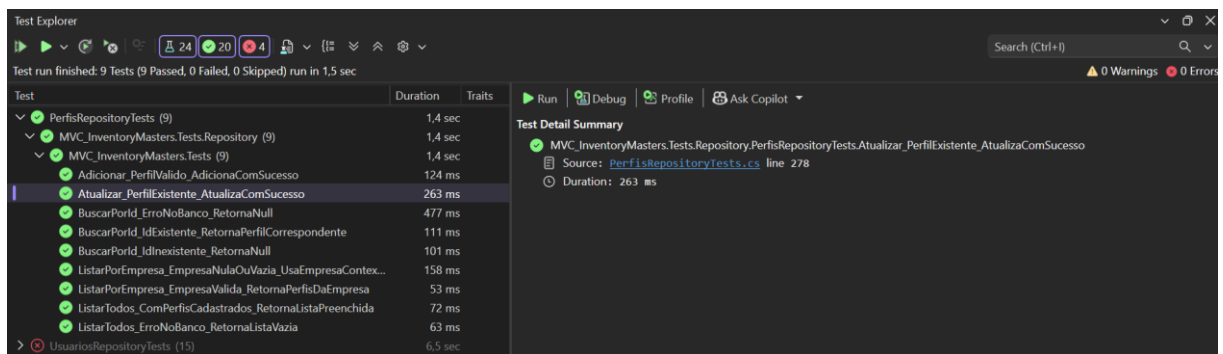


INVENTORY MASTERS

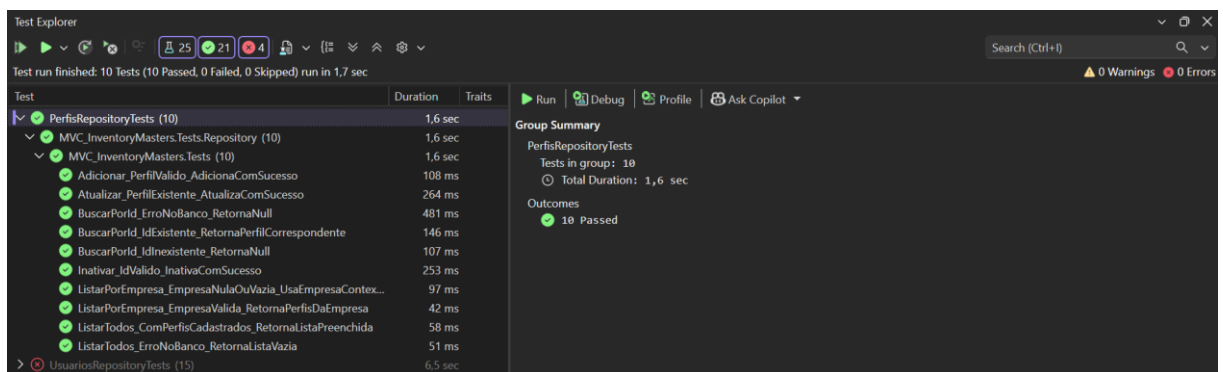
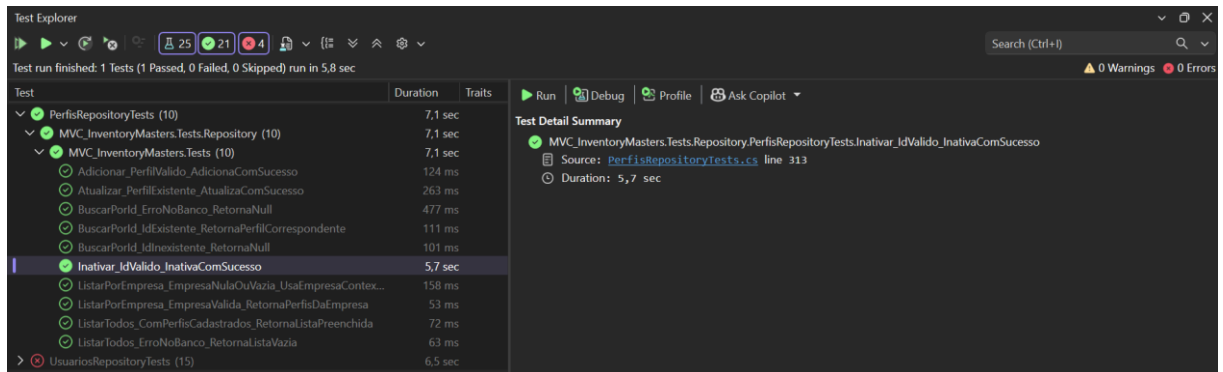
Gestão de Estoque Inteligente



CT09 - Atualizar_PerfilExistente_AtualizaComSucesso



CT10 - Inativar_IdValido_InativaComSucesso



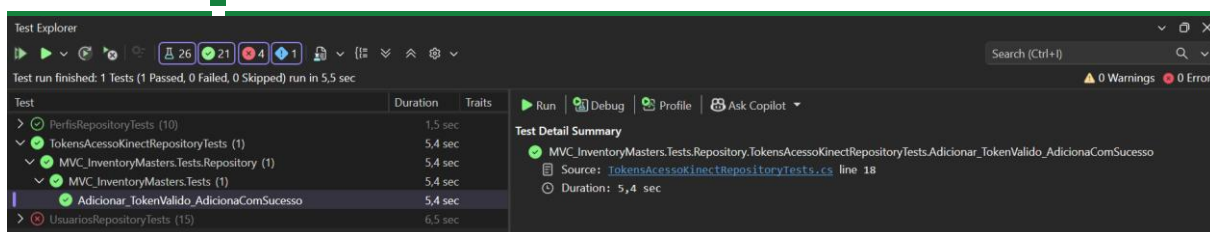
Componente: TokensAcessoKinectRepositoryTests

CT01 - Adicionar_TokenValido_AdicionaComSucesso

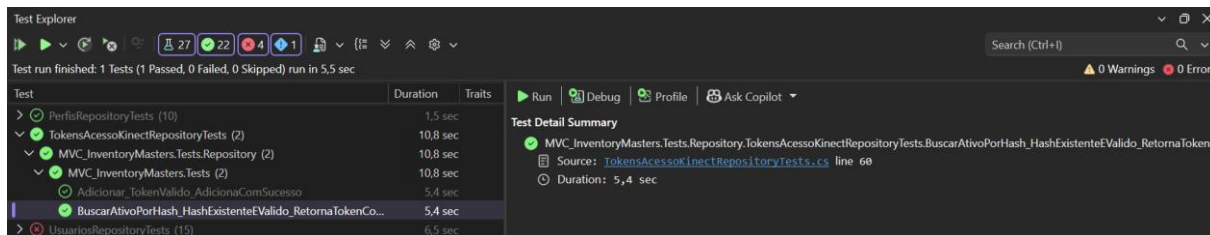


INVENTORY MASTERS

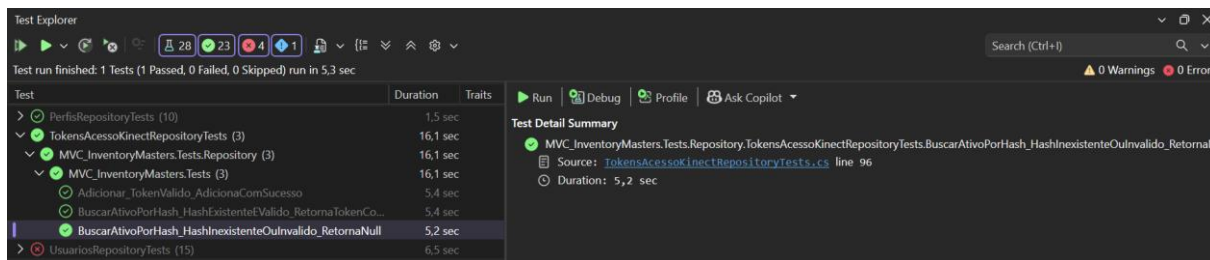
Gestão de Estoque Inteligente



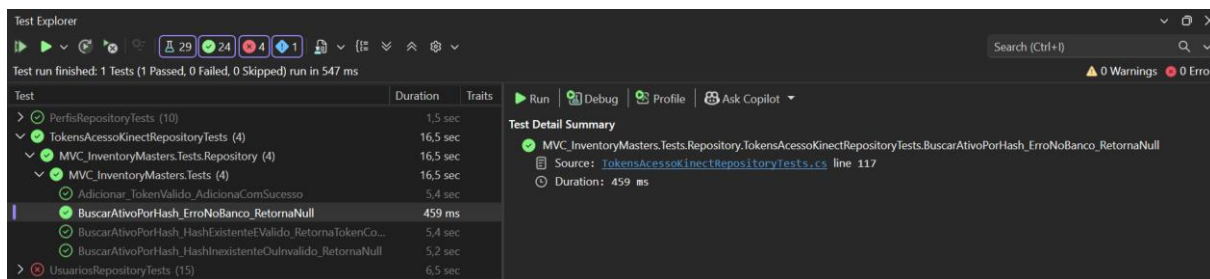
CT02 - BuscarAtivoPorHash_HashExistenteEValido_RetornaTokenCorrespondente



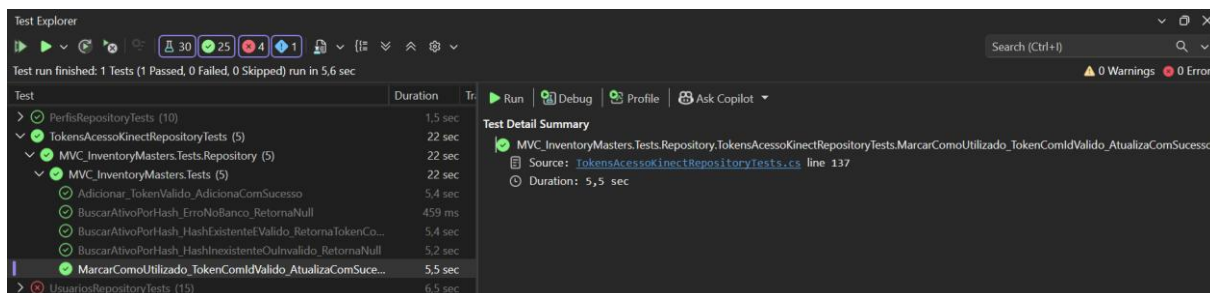
CT03 - BuscarAtivoPorHash_HashInexistenteOuInvalido_RetornaNull



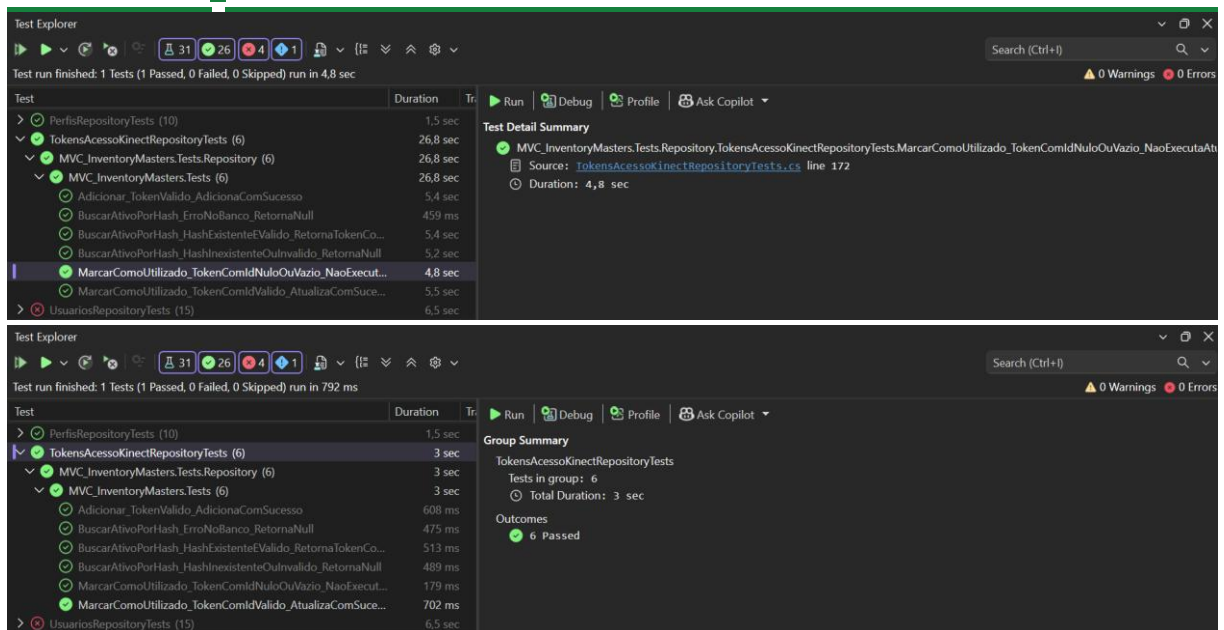
CT04 - BuscarAtivoPorHash_ErroNoBanco_RetornaNull



CT05 - MarcarComoUtilizado_TokenComIdValido_AtualizaComSucesso

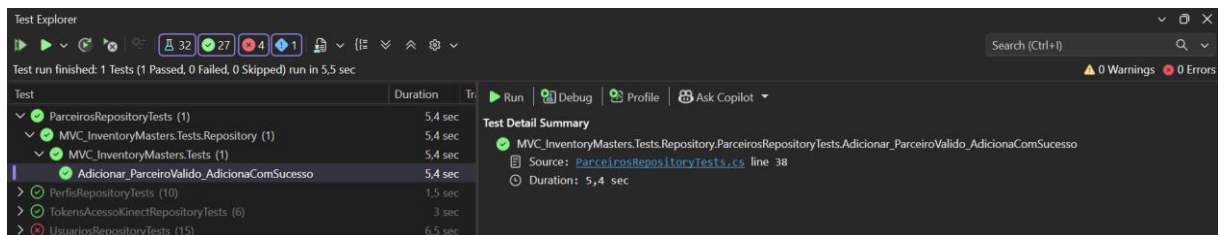


CT06 - MarcarComoUtilizado_TokenComIdNuloOuVazio_NaoExecutaAtualizacao

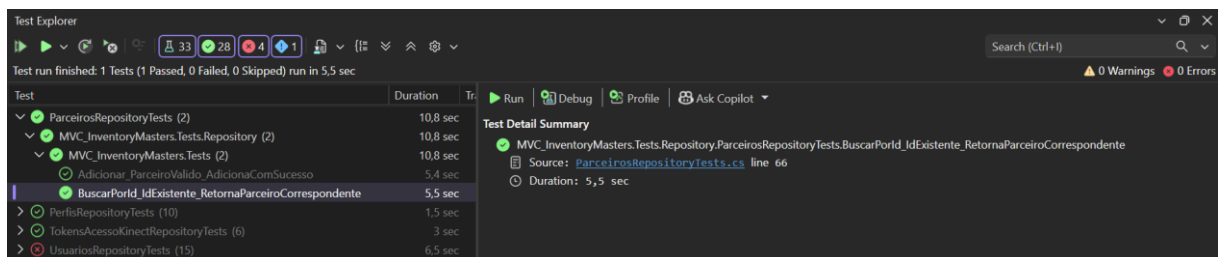


Componente: ParceirosRepositoryTests

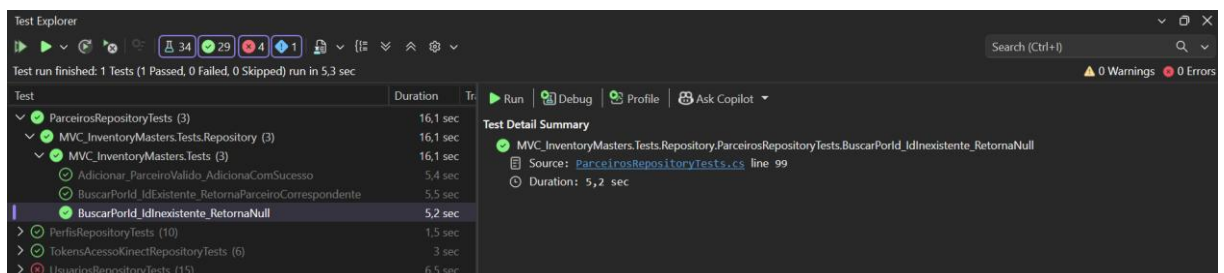
CT01 - Adicionar_ParceiroValido_AdicionaComSucesso



CT02 - BuscarPorId_IdExistente_RetornaParceiroCorrespondente



CT03 - BuscarPorId_IdInexistente_RetornaNull

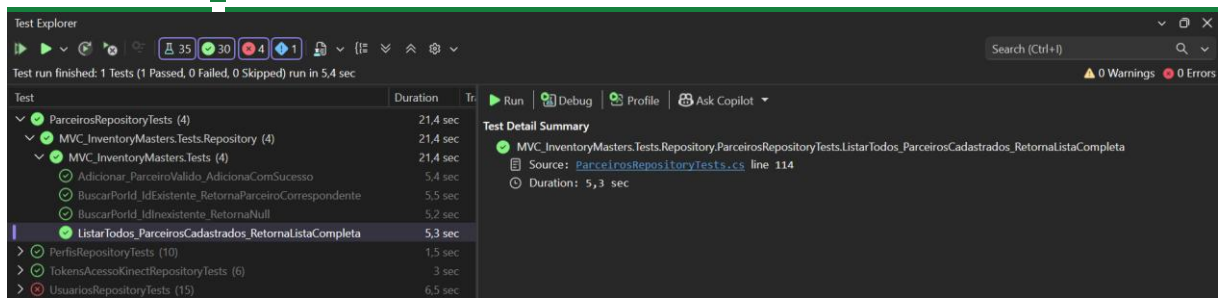


CT04 - ListarTodos_ParceirosCadastrados_RetornaListaCompleta

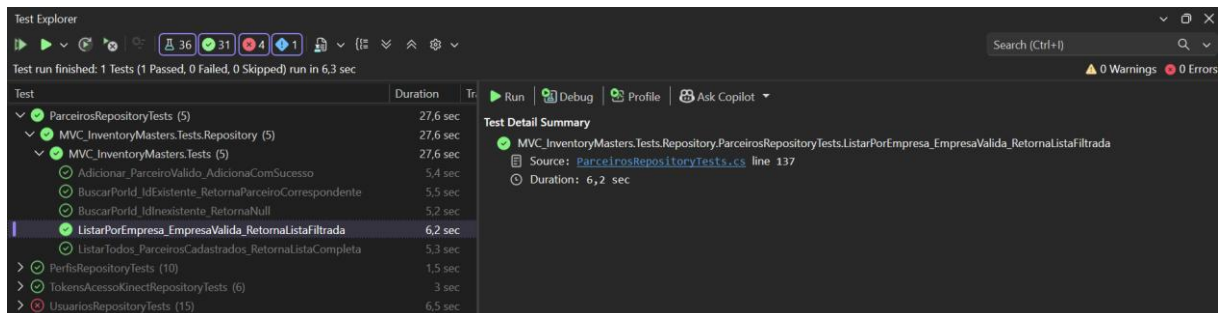


INVENTORY MASTERS

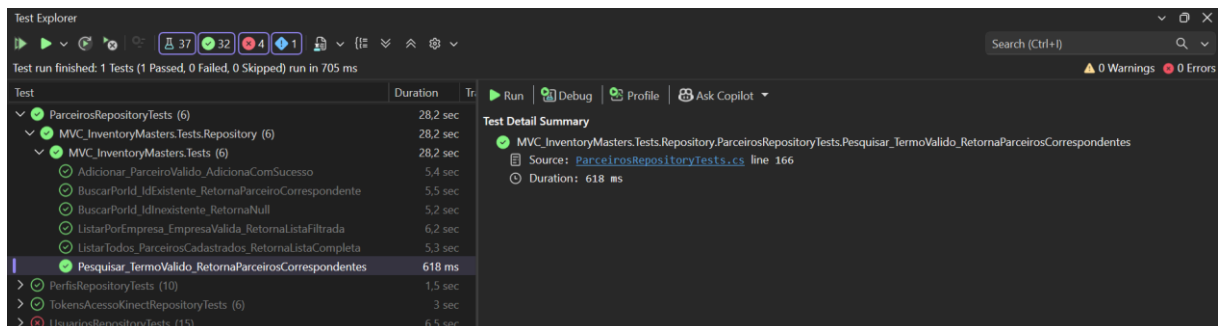
Gestão de Estoque Inteligente



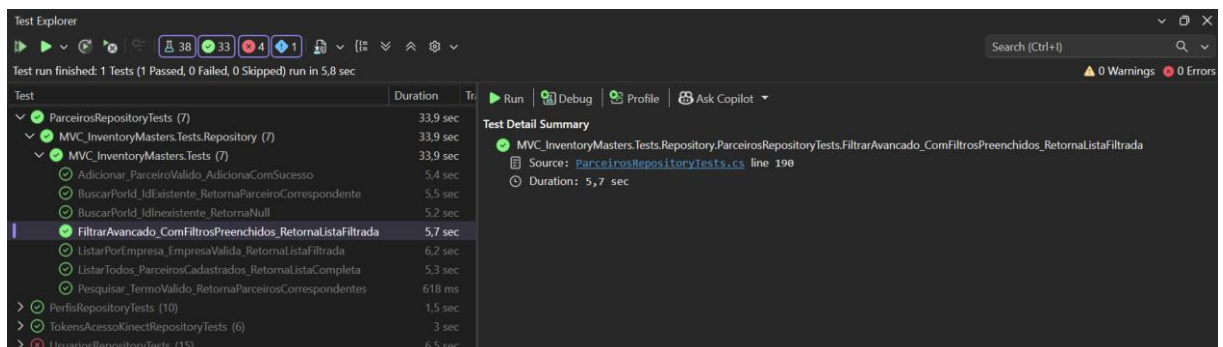
CT05 - ListarPorEmpresa_EmpresaValida_RetornaListaFiltrada



CT06 - Pesquisar_TermoValido_RetornaParceirosCorrespondentes



CT07 - FiltrarAvancado_ComFiltrosPreenchidos_RetornaListaFiltrada

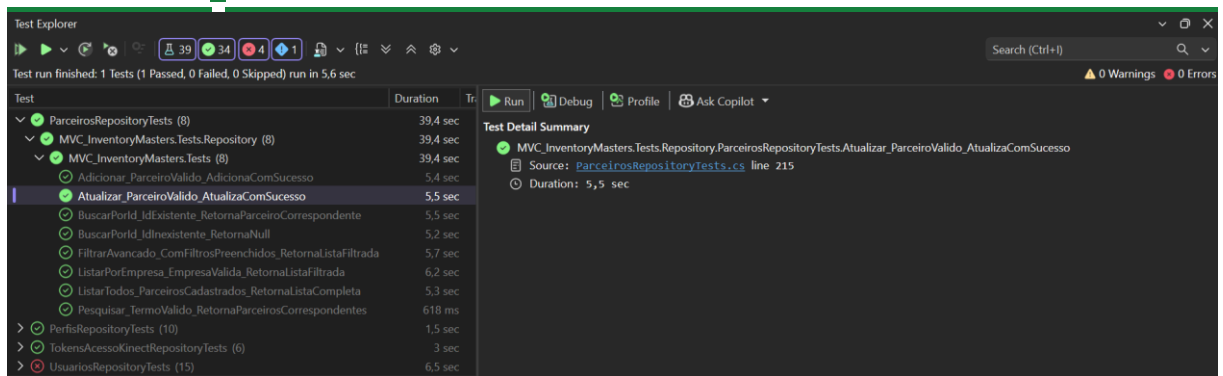


CT08 - Atualizar_ParceiroValido_AtualizaComSucesso

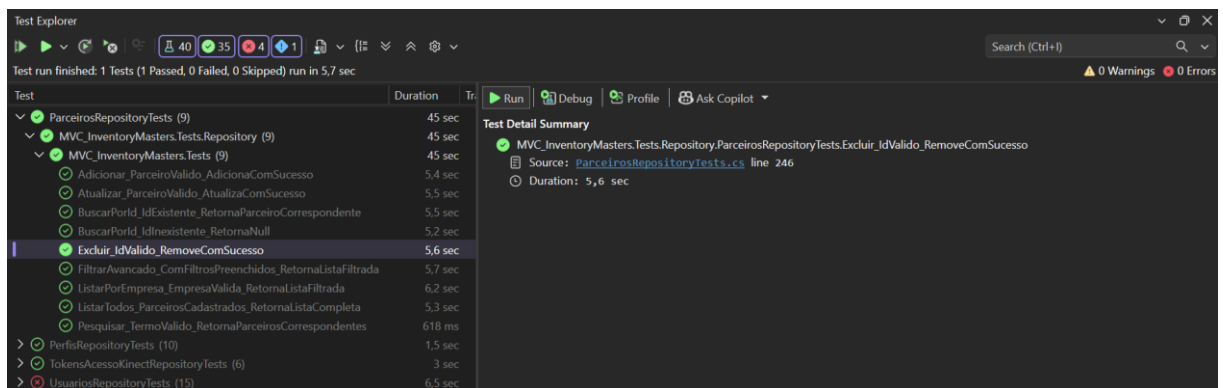


INVENTORY MASTERS

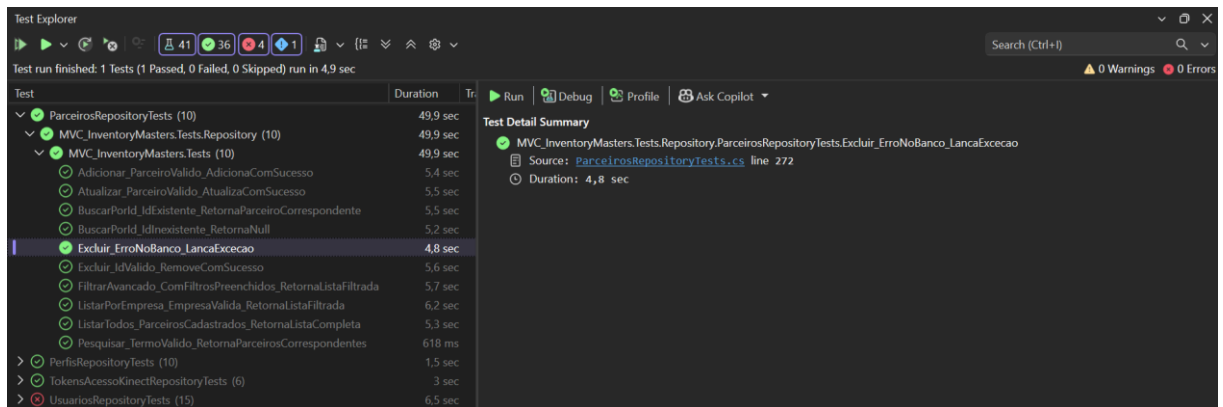
Gestão de Estoque Inteligente



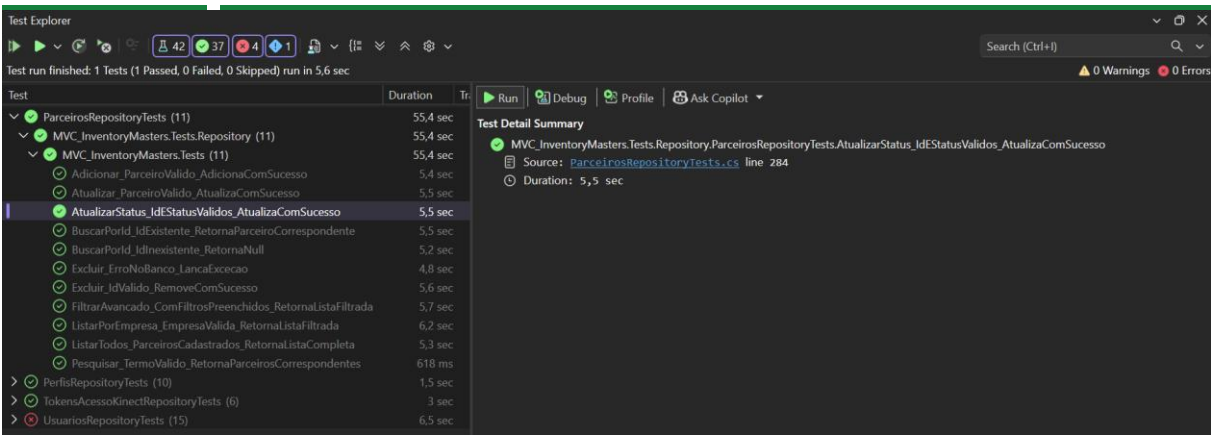
CT09 - Excluir_IdValido_RemoveComSucesso



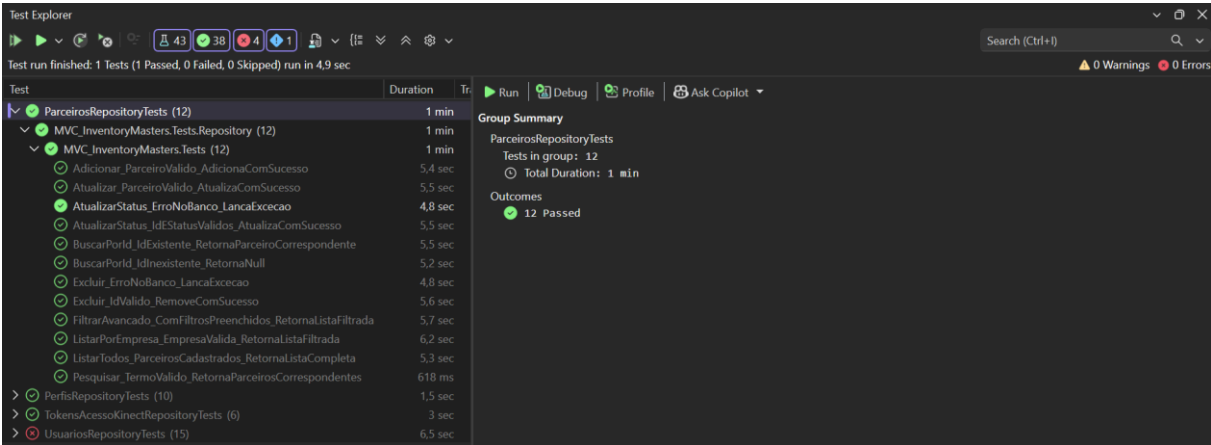
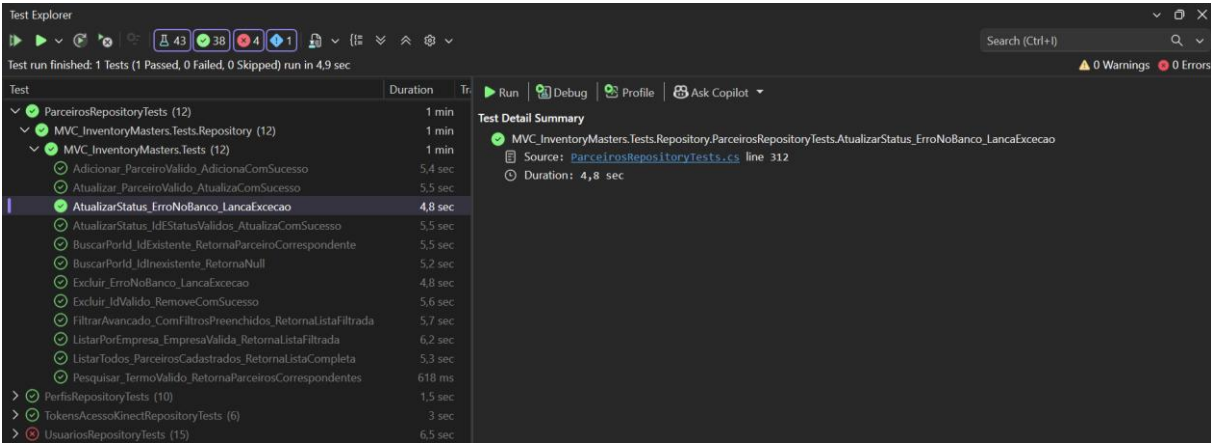
CT10 - Excluir_ErroNoBanco_LancaExecacao



CT11 - AtualizarStatus_IdEstatusValidos_AtualizaComSucesso



CT12 - AtualizarStatus_ErroNoBanco_LancaExcecao



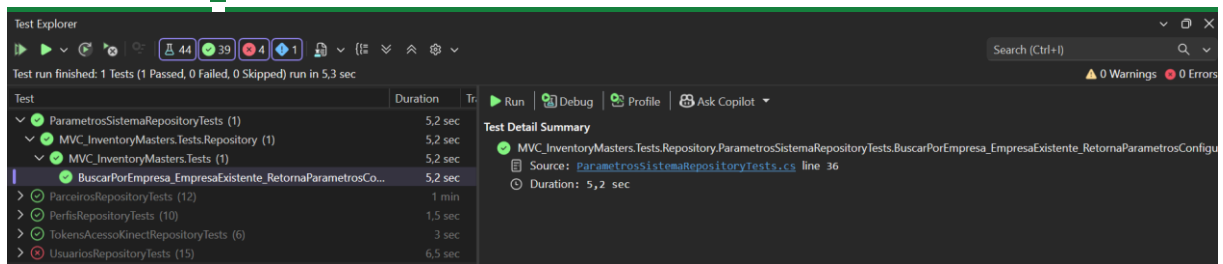
Componente: ParametrosSistemaRepositoryTests

CT01 - BuscarPorEmpresa_EmpresaExistente_RetornaParametrosConfigurados

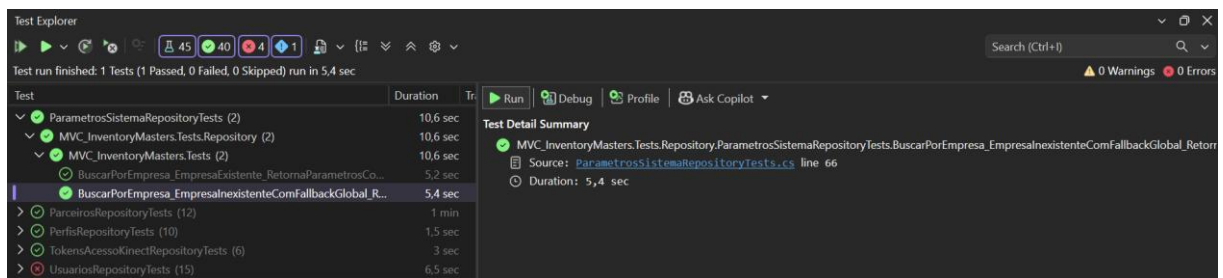


INVENTORY MASTERS

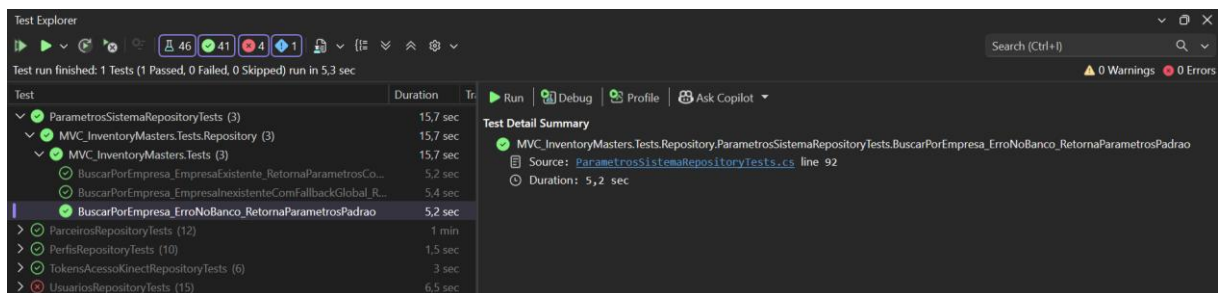
Gestão de Estoque Inteligente



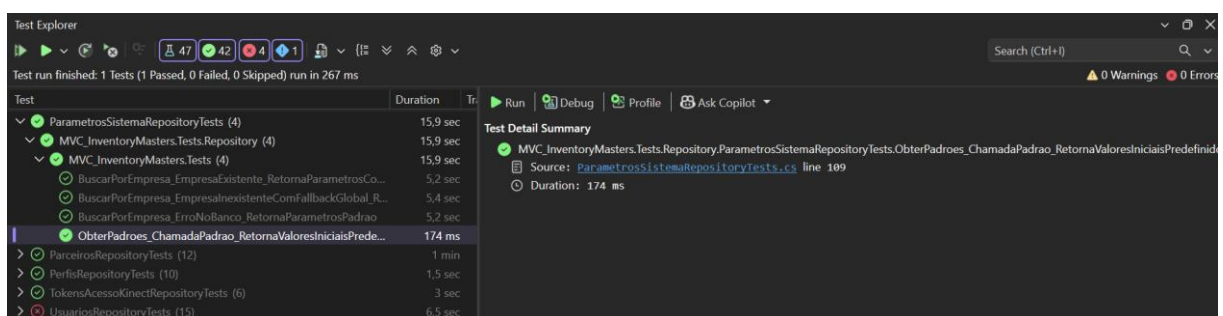
CT02 - BuscarPorEmpresa_EmpresalInexistenteComFallbackGlobal_RetornaConfiguracaoGlobal



CT03 - BuscarPorEmpresa_ErroNoBanco_RetornaParametrosPadrao



CT04 - ObterPadroes_ChamadaPadrao_RetornaValoresIniciaisPredefinidos

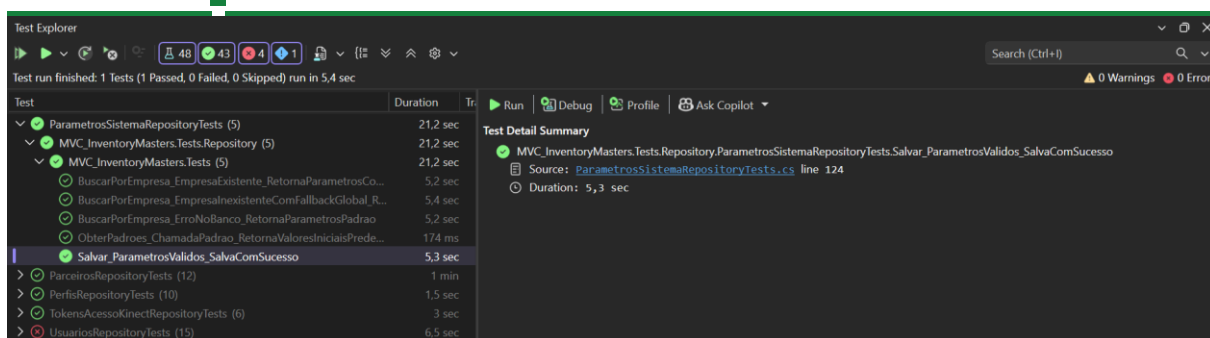


CT05 - Salvar_ParametrosValidos_SalvaComSucesso

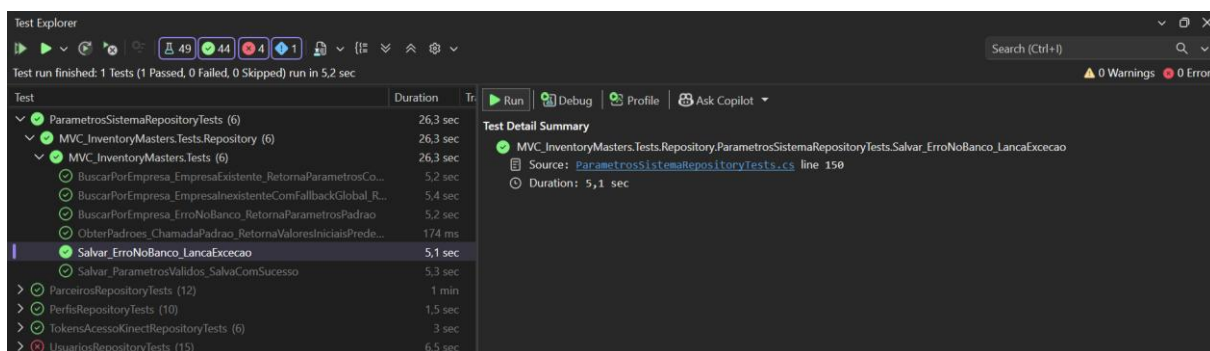


INVENTORY MASTERS

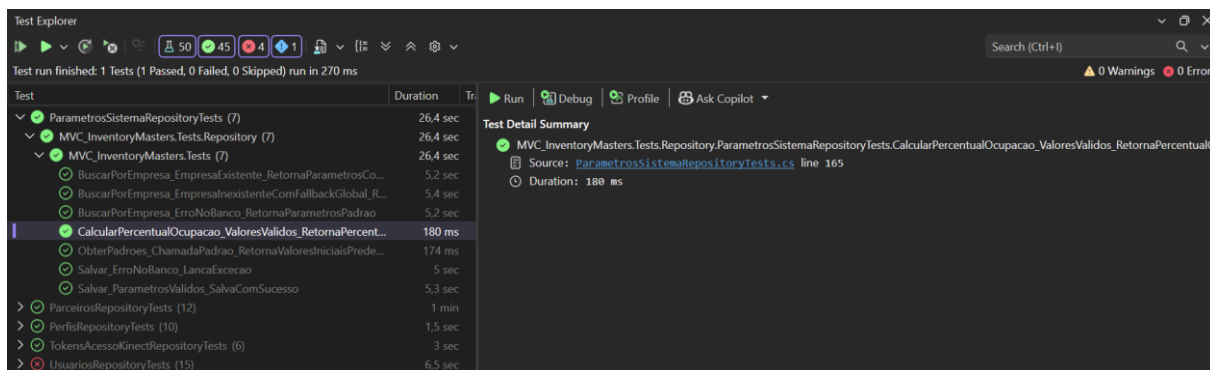
Gestão de Estoque Inteligente



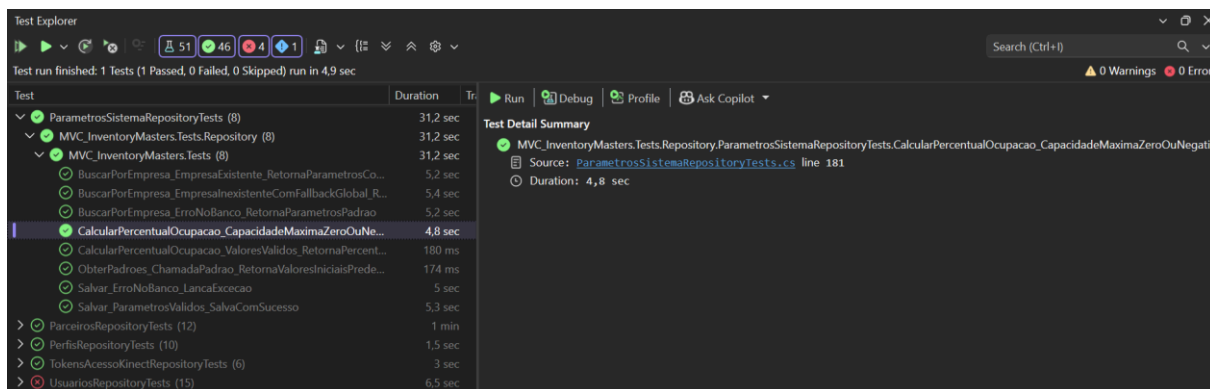
CT06 - Salvar_ErroNoBanco_LancaExeccao



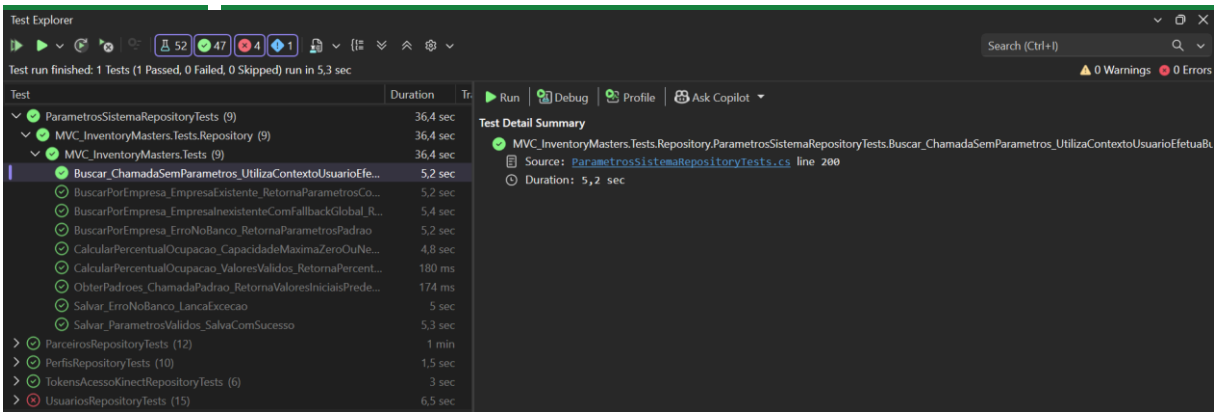
CT07 - CalcularPercentualOcupacao_ValoresValidos_RetornaPercentualCorreto



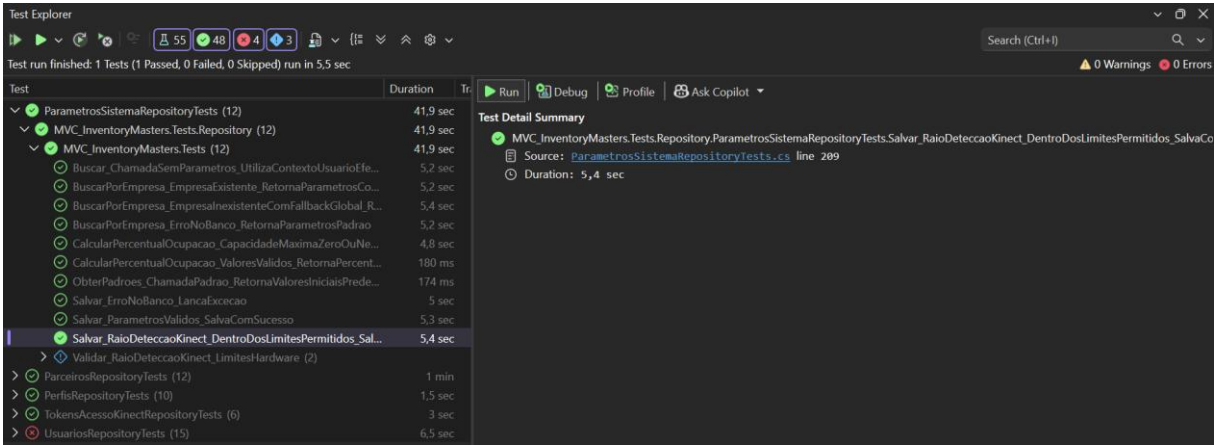
CT08 - CalcularPercentualOcupacao_CapacidadeMaximaZeroOuNegativa_RetornaZero



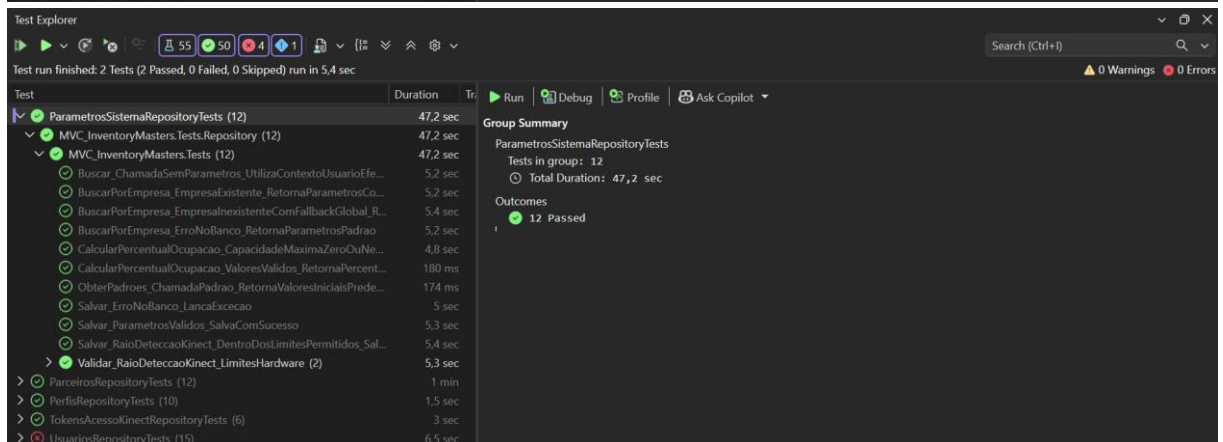
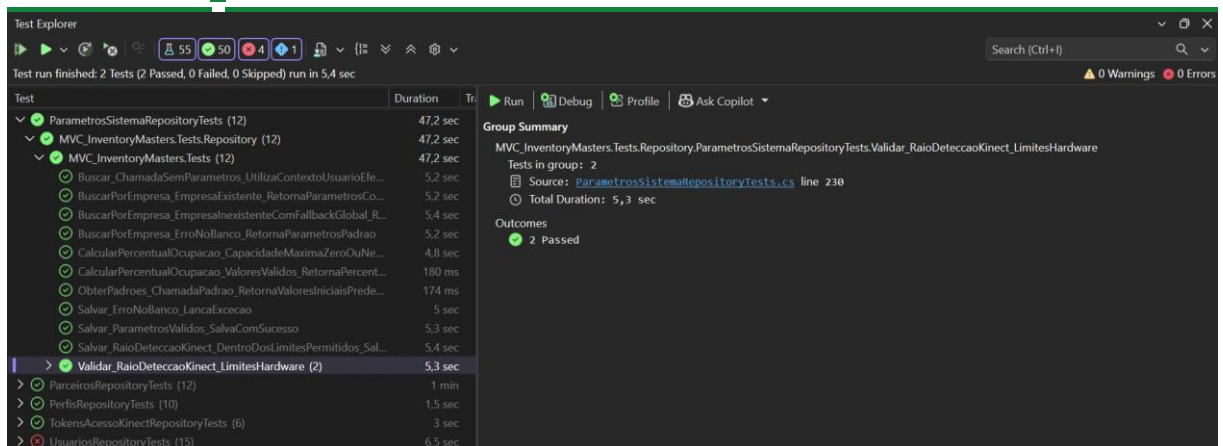
CT09 - Buscar_ChamadaSemParametros_UtilizaContextoUsuarioEfetuaBusca



CT10 - Salvar_RaioDeteccaoKinect_DentroDosLimitesPermitidos_SalvaComSucesso

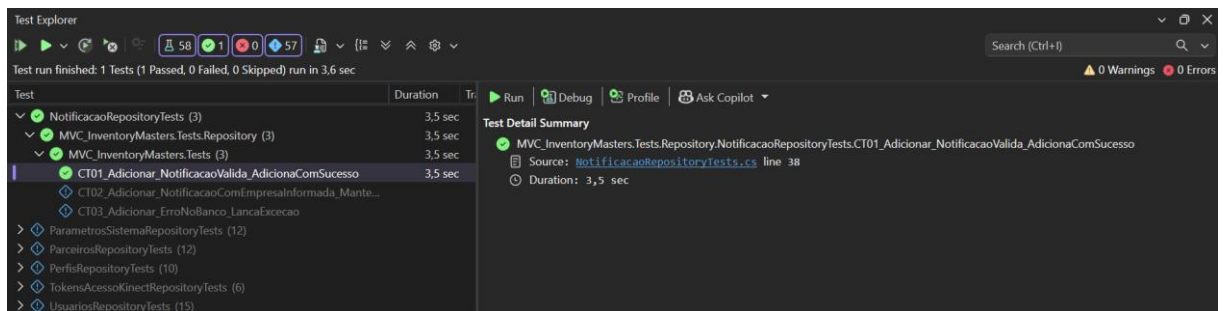


CT11 - Validar_RaioDeteccaoKinect_LimitesHardware

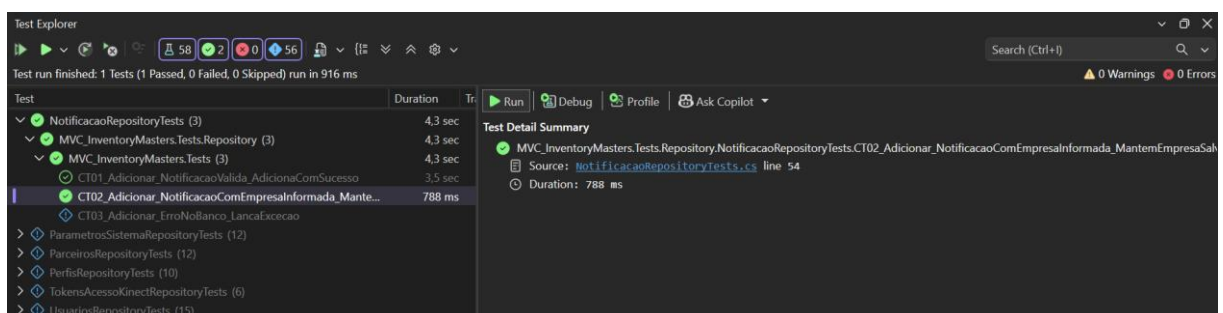


Componente: NotificacaoRepositoryTests

CT01 - Adicionar_NotificacaoValida_AdicionaComSucesso

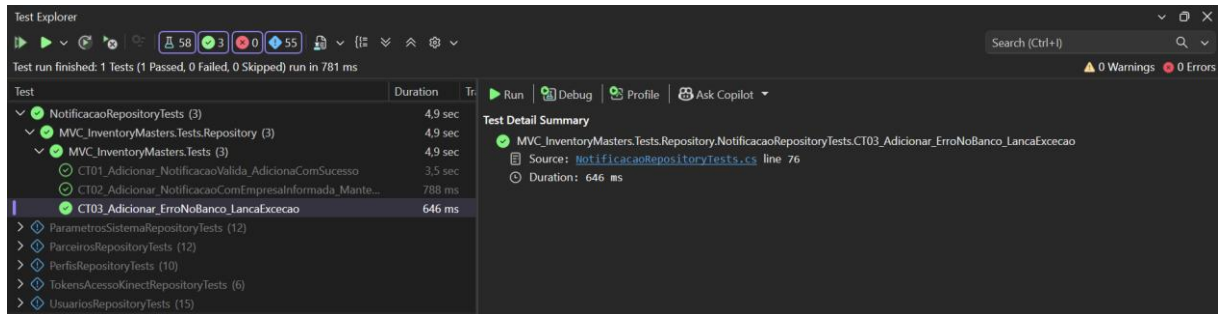


CT02 - Adicionar_NotificacaoComEmpresaInformada_MantemEmpresaSalva

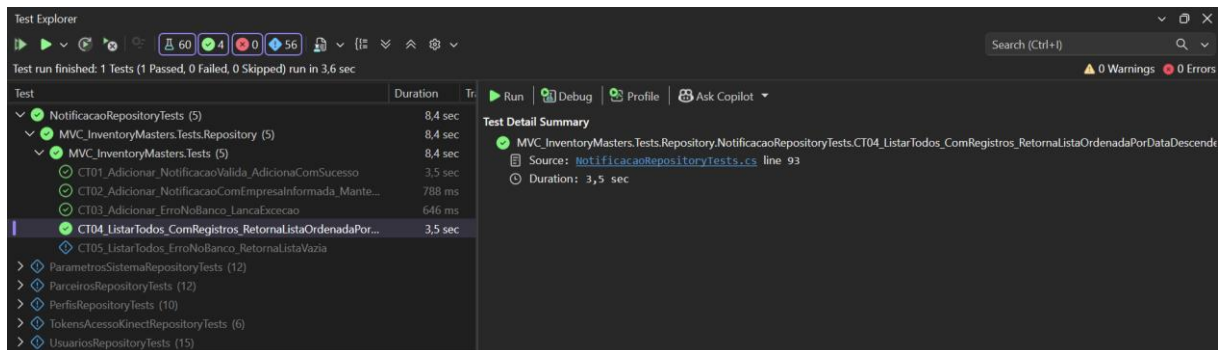




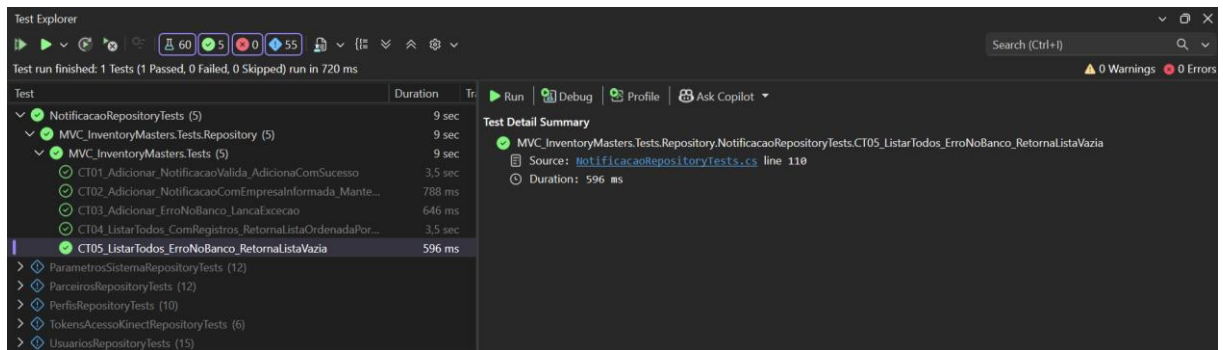
CT03 - Adicionar_ErroNoBanco_LancaExcecao



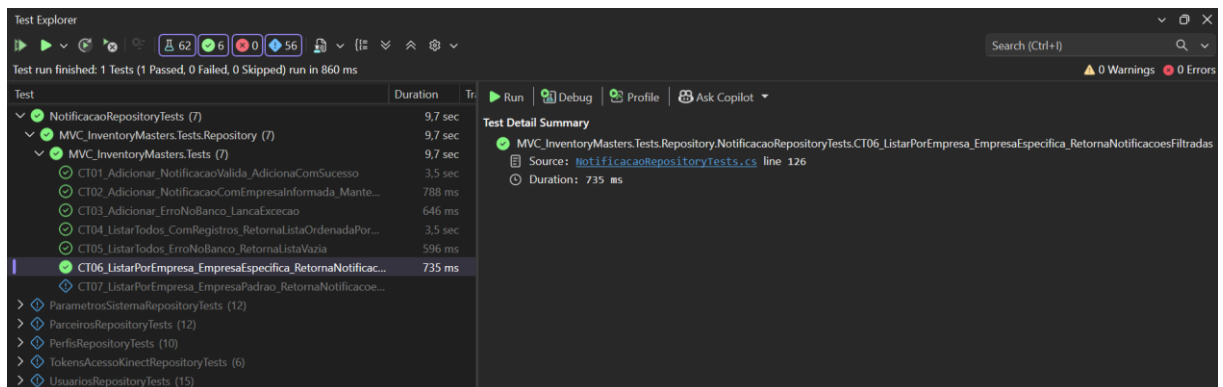
CT04 - ListarTodos_ComRegistros_RetornaListaOrdenadaPorDataDescendente



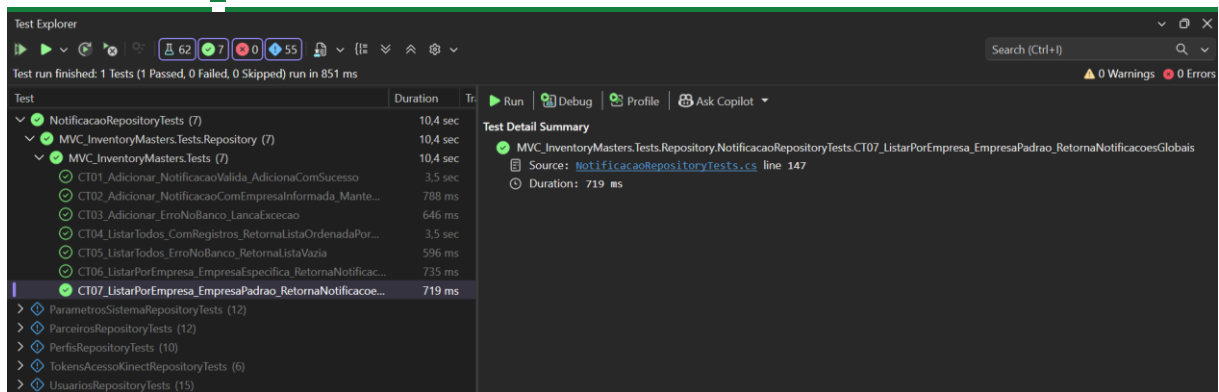
CT05 - ListarTodos_ErroNoBanco_RetornaListaVazia



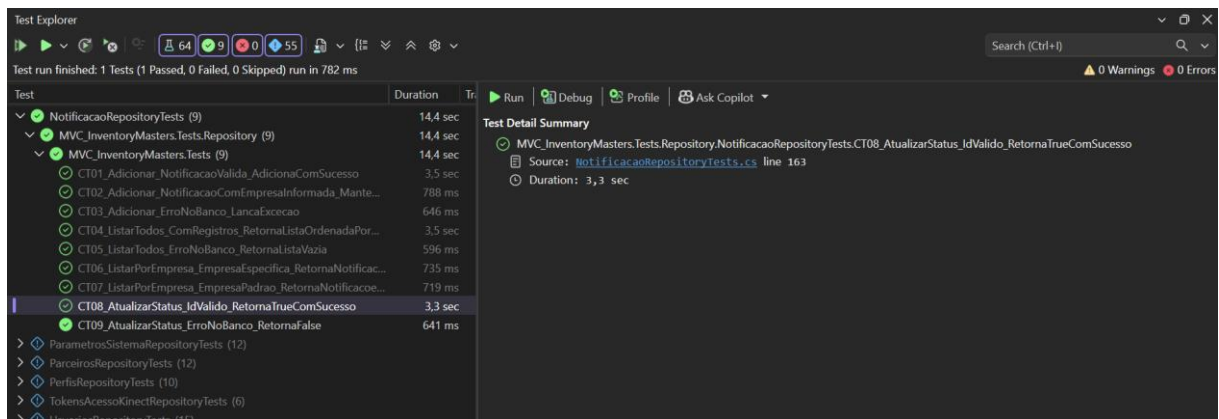
CT06 - ListarPorEmpresa_EmpresaEspecific_RetornaNotificacoesFiltradas



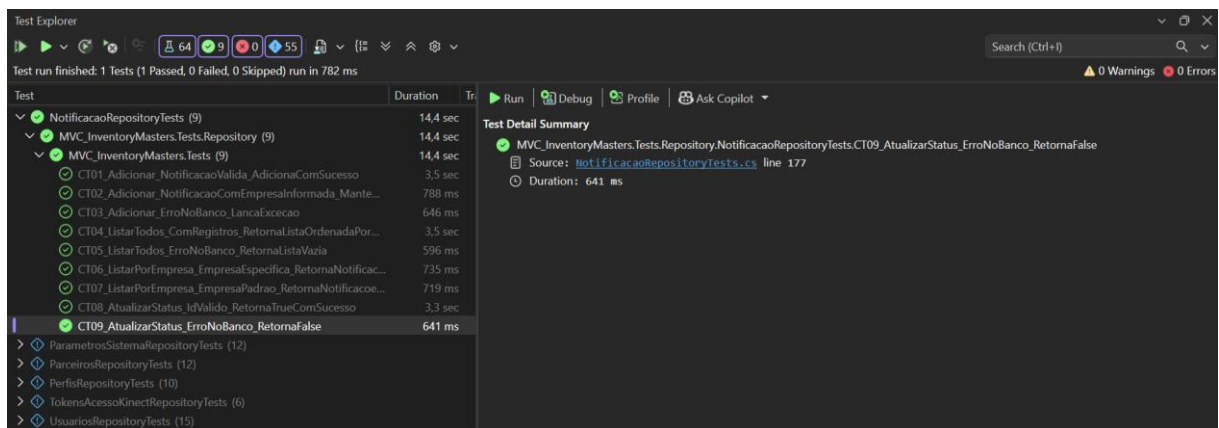
CT07 - ListarPorEmpresa_EmpresaPadrao_RetornaNotificacoesGlobais



CT08 - AtualizarStatus_IdValido_RetornaTrueComSucesso



CT09 - AtualizarStatus_ErroNoBanco_RetornaFalse



CT10 - ExisteNotificacaoPendente_ComPendenciaParaEmpresa_RetornaTrue



INVENTORY MASTERS

Gestão de Estoque Inteligente

Test Explorer

Test run finished: 1 Tests (1 Passed, 0 Failed, 0 Skipped) run in 4.2 sec

Test	Duration
NotificacaoRepositoryTests (12)	18,4 sec
MVC_InventoryMasters.Tests.Repository (12)	18,4 sec
MVC_InventoryMasters.Tests (12)	18,4 sec
CT01_Adicionar_NotificacaoValida_AdicionaComSucesso	3,5 sec
CT02_Adicionar_NotificacaoComEmpresaInformada_Mante...	788 ms
CT03_Adicionar_ErroNoBanco_LancaExcecao	646 ms
CT04_ListarTodos_ComRegistros_RetornaListaOrdenadaPor...	3,5 sec
CT05_ListarTodos_ErroNoBanco_RetornaListaVazia	596 ms
CT06_ListarPorEmpresa_EmpresaEspecific_RetornaNotificac...	735 ms
CT07_ListarPorEmpresa_EmpresaPadrao_RetornaNotificacoe...	719 ms
CT08_AtualizarStatus_IdValido_RetornaTrueComSucesso	3,3 sec
CT09_AtualizarStatus_ErroNoBanco_RetornaFalse	641 ms
CT10_ExistenciaPendente_ComPendenciaParaEmpres...	4,1 sec
CT11_ExistenciaPendente_SemPendencias_RetornaFalse	
CT12_ExistenciaPendente_ErroNoBanco_RetornaFalse	
ParametrosSistemaRepositoryTests (12)	
ParceirosRepositoryTests (12)	
PerfisRepositoryTests (10)	
TokensAcessoKinectRepositoryTests (6)	
UsuariosRepositoryTests (15)	

Test Detail Summary

- MVC_InventoryMasters.Tests.Repository.NotificacaoRepositoryTests.CT10_ExistenciaPendente_ComPendenciaParaEmpresas_RetornaTrue
- Source: [NotificacaoRepositoryTests.cs](#) line 192
- Duration: 4,1 sec

CT11 - ExistenciaPendente_SemPendencias_RetornaFalse

Test Explorer

Test run finished: 1 Tests (1 Passed, 0 Failed, 0 Skipped) run in 795 ms

Test	Duration
NotificacaoRepositoryTests (12)	19,1 sec
MVC_InventoryMasters.Tests.Repository (12)	19,1 sec
MVC_InventoryMasters.Tests (12)	19,1 sec
CT01_Adicionar_NotificacaoValida_AdicionaComSucesso	3,5 sec
CT02_Adicionar_NotificacaoComEmpresaInformada_Mante...	788 ms
CT03_Adicionar_ErroNoBanco_LancaExcecao	646 ms
CT04_ListarTodos_ComRegistros_RetornaListaOrdenadaPor...	3,5 sec
CT05_ListarTodos_ErroNoBanco_RetornaListaVazia	596 ms
CT06_ListarPorEmpresa_EmpresaEspecific_RetornaNotificac...	735 ms
CT07_ListarPorEmpresa_EmpresaPadrao_RetornaNotificacoe...	719 ms
CT08_AtualizarStatus_IdValido_RetornaTrueComSucesso	3,3 sec
CT09_AtualizarStatus_ErroNoBanco_RetornaFalse	641 ms
CT10_ExistenciaPendente_ComPendenciaParaEmpres...	4,1 sec
CT11_ExistenciaPendente_SemPendencias_RetornaFalse	666 ms
CT12_ExistenciaPendente_ErroNoBanco_RetornaFalse	
ParametrosSistemaRepositoryTests (12)	
ParceirosRepositoryTests (12)	
PerfisRepositoryTests (10)	
TokensAcessoKinectRepositoryTests (6)	
UsuariosRepositoryTests (15)	

Test Detail Summary

- MVC_InventoryMasters.Tests.Repository.NotificacaoRepositoryTests.CT11_ExistenciaPendente_SemPendencias_RetornaFalse
- Source: [NotificacaoRepositoryTests.cs](#) line 205
- Duration: 666 ms

CT12 - ExistenciaPendente_ErroNoBanco_RetornaFalse

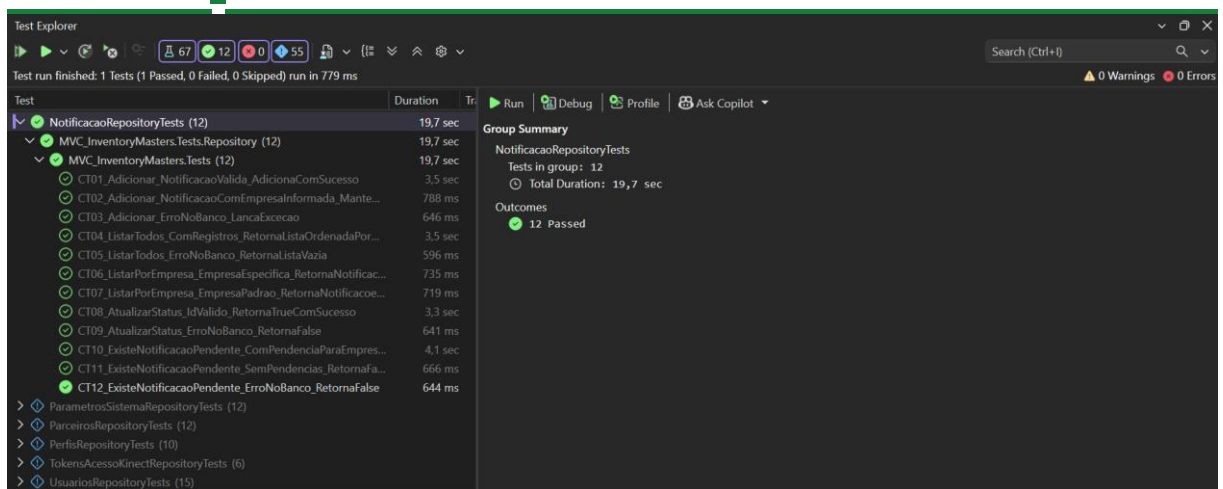
Test Explorer

Test run finished: 1 Tests (1 Passed, 0 Failed, 0 Skipped) run in 779 ms

Test	Duration
NotificacaoRepositoryTests (12)	19,7 sec
MVC_InventoryMasters.Tests.Repository (12)	19,7 sec
MVC_InventoryMasters.Tests (12)	19,7 sec
CT01_Adicionar_NotificacaoValida_AdicionaComSucesso	3,5 sec
CT02_Adicionar_NotificacaoComEmpresaInformada_Mante...	788 ms
CT03_Adicionar_ErroNoBanco_LancaExcecao	646 ms
CT04_ListarTodos_ComRegistros_RetornaListaOrdenadaPor...	3,5 sec
CT05_ListarTodos_ErroNoBanco_RetornaListaVazia	596 ms
CT06_ListarPorEmpresa_EmpresaEspecific_RetornaNotificac...	735 ms
CT07_ListarPorEmpresa_EmpresaPadrao_RetornaNotificacoe...	719 ms
CT08_AtualizarStatus_IdValido_RetornaTrueComSucesso	3,3 sec
CT09_AtualizarStatus_ErroNoBanco_RetornaFalse	641 ms
CT10_ExistenciaPendente_ComPendenciaParaEmpres...	4,1 sec
CT11_ExistenciaPendente_SemPendencias_RetornaFalse	666 ms
CT12_ExistenciaPendente_ErroNoBanco_RetornaFalse	644 ms
ParametrosSistemaRepositoryTests (12)	
ParceirosRepositoryTests (12)	
PerfisRepositoryTests (10)	
TokensAcessoKinectRepositoryTests (6)	
UsuariosRepositoryTests (15)	

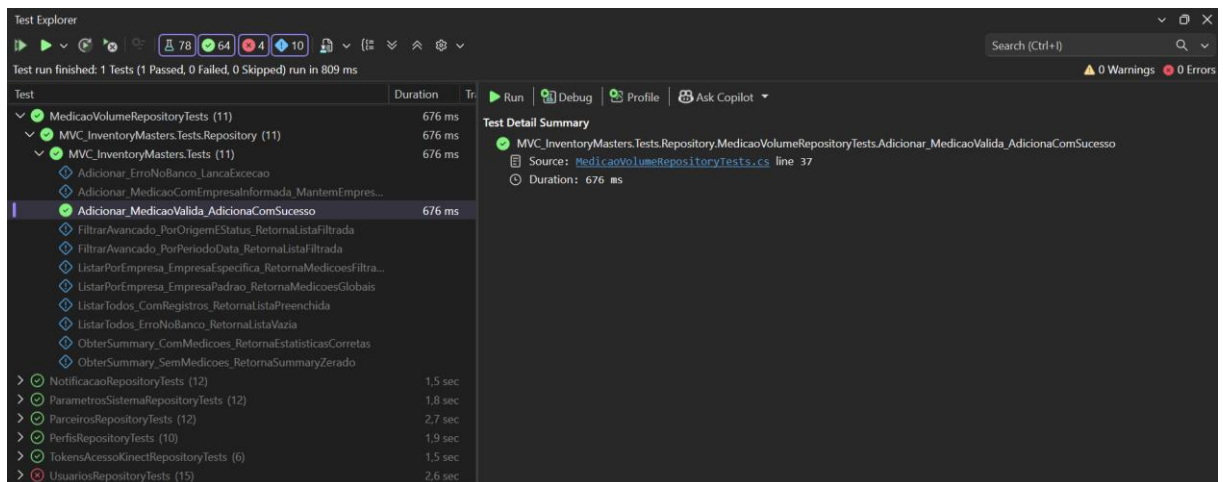
Test Detail Summary

- MVC_InventoryMasters.Tests.Repository.NotificacaoRepositoryTests.CT12_ExistenciaPendente_ErroNoBanco_RetornaFalse
- Source: [NotificacaoRepositoryTests.cs](#) line 218
- Duration: 644 ms

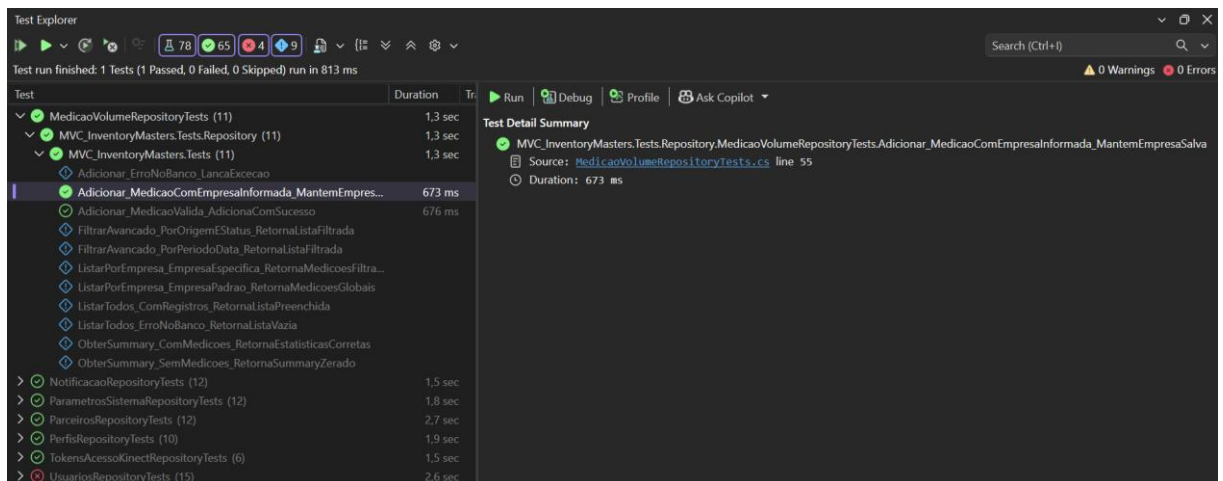


Componente: MedicaoVolumeRepositoryTests

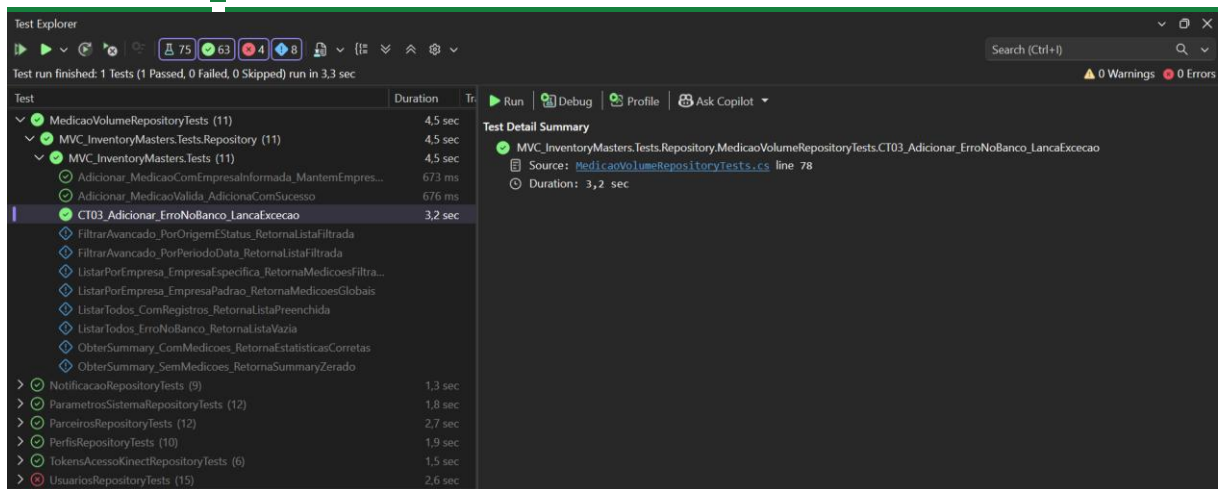
CT01 - Adicionar_MedicaoValida_AdicionaComSucesso



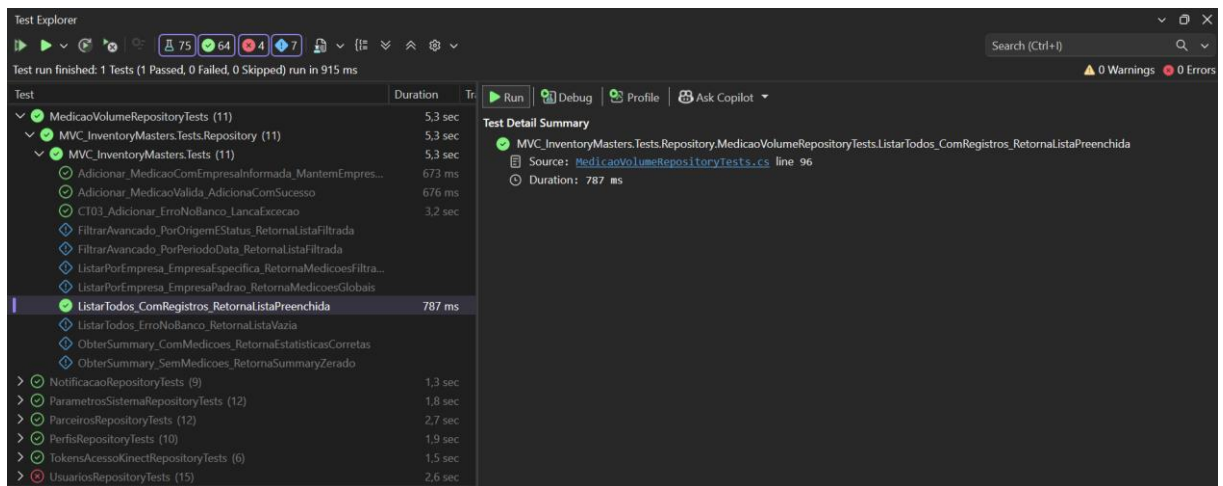
CT02 - Adicionar_MedicaoComEmpresaInformada_MantemEmpresaSalva



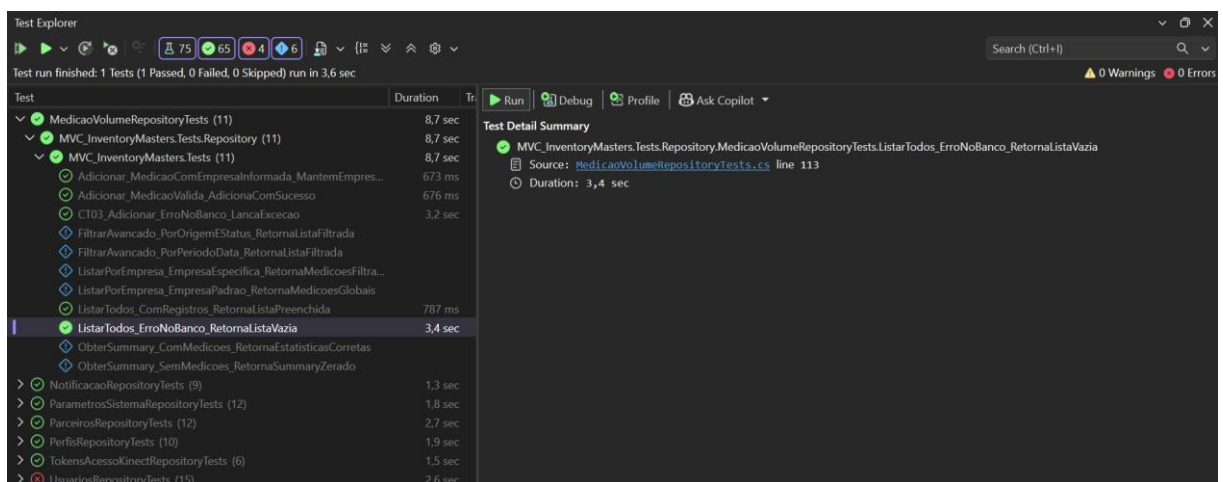
CT03 - Adicionar_ErroNoBanco_LancaExcecao



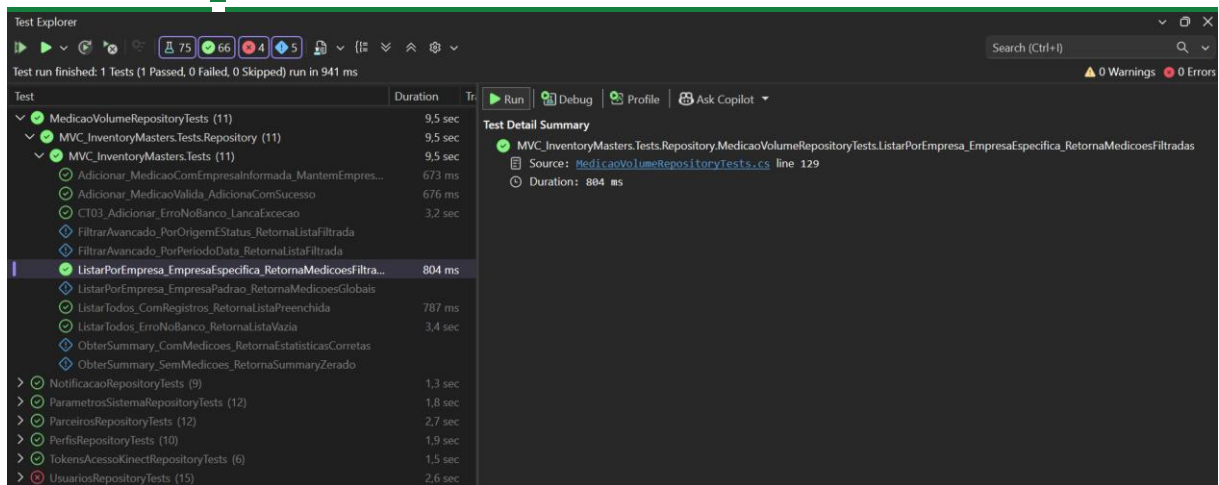
CT04 - ListarTodos_ComRegistros_RetornaListaPreenchida



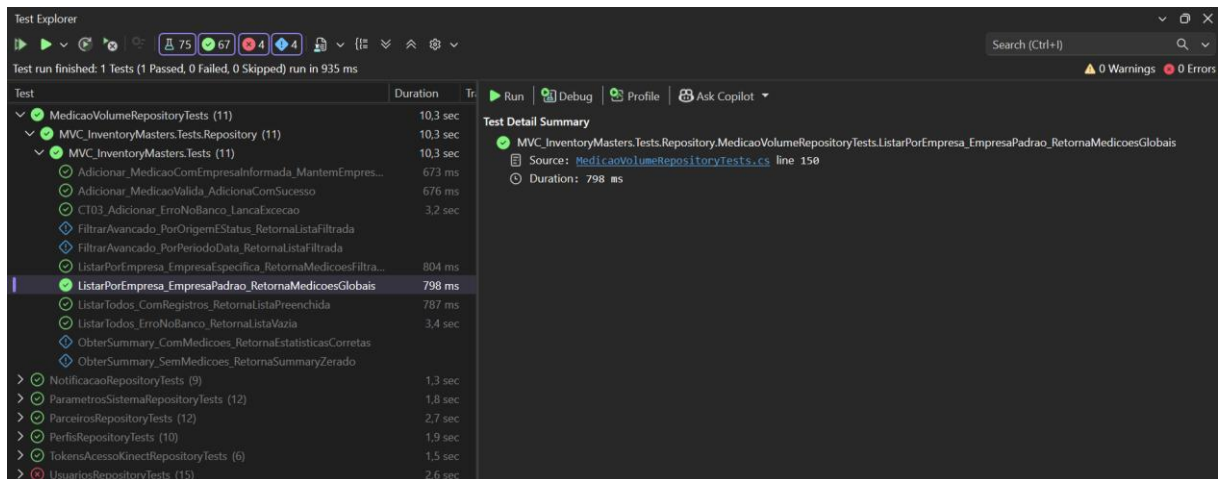
CT05 - ListarTodos_ErroNoBanco_RetornaListaVazia



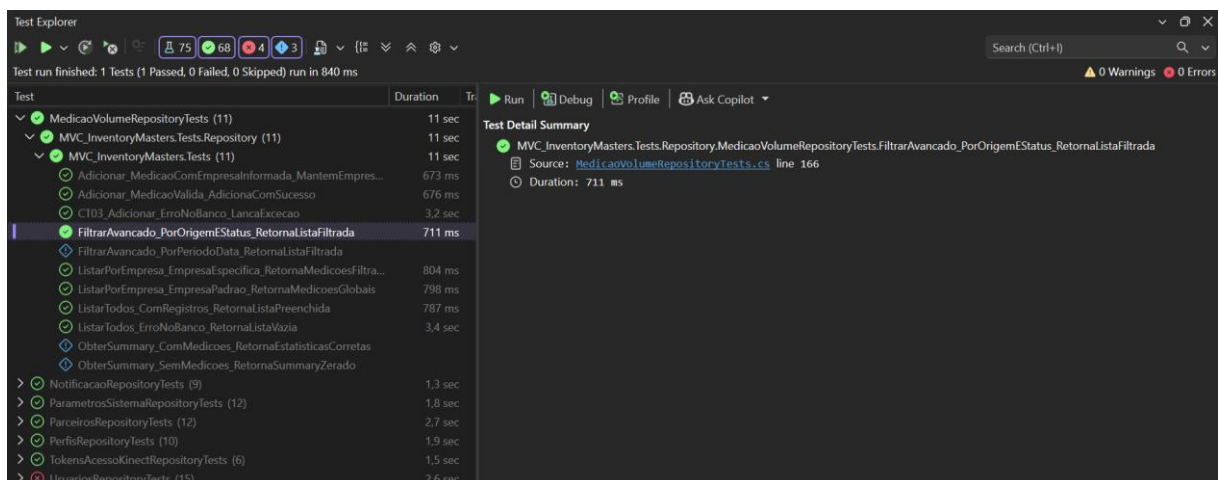
CT06 - ListarPorEmpresa_EmpresaEspecific_RetornaMedicoesFiltradas



CT07 - ListarPorEmpresa_EmpresaPadrao_RetornaMedicoesGlobais



CT08 - FiltrarAvancado_PorOrigemEStatus_RetornaListaFiltrada



CT09 - FiltrarAvancado_PorPeriodoData_RetornaListaFiltrada



INVENTORY MASTERS

Gestão de Estoque Inteligente

Test Explorer			Test Detail Summary		
Test run finished: 1 Tests (1 Passed, 0 Failed, 0 Skipped) run in 1 sec			0 Warnings 0 Errors		
Test	Duration	Tr	Run Debug Profile Ask Copilot		
MedicaoVolumeRepositoryTests (11)	11,9 sec		MVC_InventoryMasters.Tests.Repository.MedicaoVolumeRepositoryTests.FiltrarAvancado_PorPeriodoData_RetornaListaFiltrada		
MVC_InventoryMasters.Tests.Repository (11)	11,9 sec		Source: MedicaoVolumeRepositoryTests.cs line 192		
Adicionar_MedicaoComEmpresainformada_MantemEmpres...	673 ms		Duration: 912 ms		
Adicionar_MedicaoValida_AdicionaComSucesso	676 ms				
CT03_Adicionar_ErroNoBanco_LancaExcecao	3,2 sec				
FiltrarAvancado_PorOrigemEStatus_RetornaListaFiltrada	711 ms				
FiltrarAvancado_PorPeriodoData_RetornaListaFiltrada	912 ms				
ListarPorEmpresa_EmpresaEspecific_RetornaMedicoesFiltr...	804 ms				
ListarPorEmpresa_EmpresaPadrao_RetornaMedicoesGlobais	798 ms				
ListarTodos_ComRegistros_RetornaListaPreenchida	787 ms				
ListarTodos_ErroNoBanco_RetornaListaVazia	3,4 sec				
ObterSummary_ComMedicoes_RetornaEstatisticasCorretas					
ObterSummary_SemMedicoes_RetornaSummaryZero					
NotificacaoRepositoryTests (9)	1,3 sec				
ParametrosSistemaRepositoryTests (12)	1,8 sec				
ParceirosRepositoryTests (12)	2,7 sec				
PerfisRepositoryTests (10)	1,9 sec				
TokensAcessoKinectRepositoryTests (6)	1,5 sec				
UsuariosRepositoryTests (15)	2,6 sec				

CT10 - ObterSummary_ComMedicoes_RetornaEstatisticasCorretas

Test Explorer			Test Detail Summary		
Test run finished: 1 Tests (1 Passed, 0 Failed, 0 Skipped) run in 1 sec			0 Warnings 0 Errors		
Test	Duration	Tr	Run Debug Profile Ask Copilot		
MedicaoVolumeRepositoryTests (11)	12,8 sec		MVC_InventoryMasters.Tests.Repository.MedicaoVolumeRepositoryTests.ObterSummary_ComMedicoes_RetornaEstatisticasCorretas		
MVC_InventoryMasters.Tests.Repository (11)	12,8 sec		Source: MedicaoVolumeRepositoryTests.cs line 217		
Adicionar_MedicaoComEmpresainformada_MantemEmpres...	673 ms		Duration: 901 ms		
Adicionar_MedicaoValida_AdicionaComSucesso	676 ms				
CT03_Adicionar_ErroNoBanco_LancaExcecao	3,2 sec				
FiltrarAvancado_PorOrigemEStatus_RetornaListaFiltrada	711 ms				
FiltrarAvancado_PorPeriodoData_RetornaListaFiltrada	912 ms				
ListarPorEmpresa_EmpresaEspecific_RetornaMedicoesFiltr...	804 ms				
ListarPorEmpresa_EmpresaPadrao_RetornaMedicoesGlobais	798 ms				
ListarTodos_ComRegistros_RetornaListaPreenchida	787 ms				
ListarTodos_ErroNoBanco_RetornaListaVazia	3,4 sec				
ObterSummary_ComMedicoes_RetornaEstatisticasCorretas	901 ms				
ObterSummary_SemMedicoes_RetornaSummaryZero					
NotificacaoRepositoryTests (9)	1,3 sec				
ParametrosSistemaRepositoryTests (12)	1,8 sec				
ParceirosRepositoryTests (12)	2,7 sec				
PerfisRepositoryTests (10)	1,9 sec				
TokensAcessoKinectRepositoryTests (6)	1,5 sec				
UsuariosRepositoryTests (15)	2,6 sec				

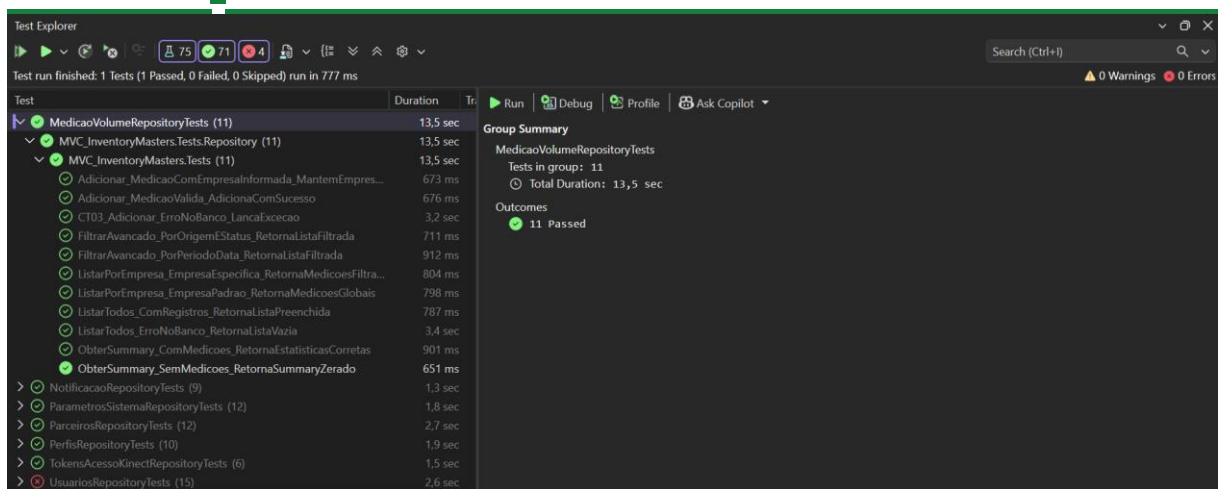
CT11 - ObterSummary_SemMedicoes_RetornaSummaryZero

Test Explorer			Test Detail Summary		
Test run finished: 1 Tests (1 Passed, 0 Failed, 0 Skipped) run in 777 ms			0 Warnings 0 Errors		
Test	Duration	Tr	Run Debug Profile Ask Copilot		
MedicaoVolumeRepositoryTests (11)	13,5 sec		MVC_InventoryMasters.Tests.Repository.MedicaoVolumeRepositoryTests.ObterSummary_SemMedicoes_RetornaSummaryZero		
MVC_InventoryMasters.Tests.Repository (11)	13,5 sec		Source: MedicaoVolumeRepositoryTests.cs line 237		
Adicionar_MedicaoComEmpresainformada_MantemEmpres...	673 ms		Duration: 651 ms		
Adicionar_MedicaoValida_AdicionaComSucesso	676 ms				
CT03_Adicionar_ErroNoBanco_LancaExcecao	3,2 sec				
FiltrarAvancado_PorOrigemEStatus_RetornaListaFiltrada	711 ms				
FiltrarAvancado_PorPeriodoData_RetornaListaFiltrada	912 ms				
ListarPorEmpresa_EmpresaEspecific_RetornaMedicoesFiltr...	804 ms				
ListarPorEmpresa_EmpresaPadrao_RetornaMedicoesGlobais	798 ms				
ListarTodos_ComRegistros_RetornaListaPreenchida	787 ms				
ListarTodos_ErroNoBanco_RetornaListaVazia	3,4 sec				
ObterSummary_ComMedicoes_RetornaEstatisticasCorretas	901 ms				
ObterSummary_SemMedicoes_RetornaSummaryZero	651 ms				
NotificacaoRepositoryTests (9)	1,3 sec				
ParametrosSistemaRepositoryTests (12)	1,8 sec				
ParceirosRepositoryTests (12)	2,7 sec				
PerfisRepositoryTests (10)	1,9 sec				
TokensAcessoKinectRepositoryTests (6)	1,5 sec				
UsuariosRepositoryTests (15)	2,6 sec				



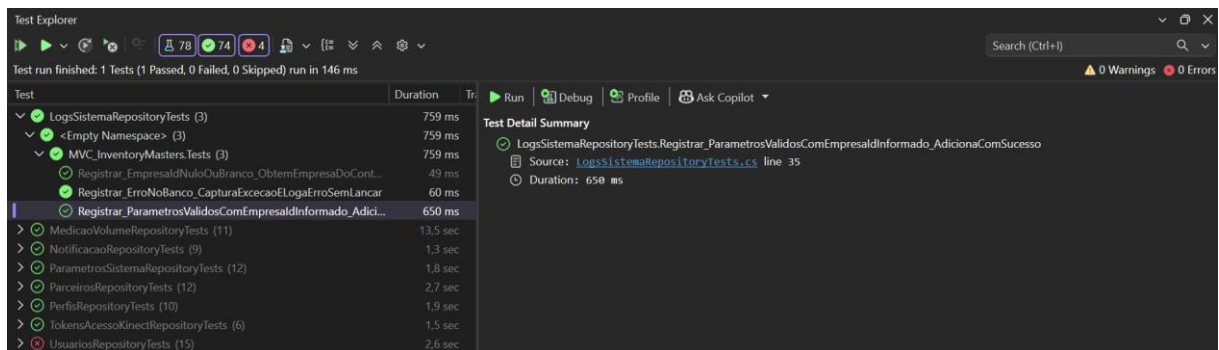
INVENTORY MASTERS

Gestão de Estoque Inteligente

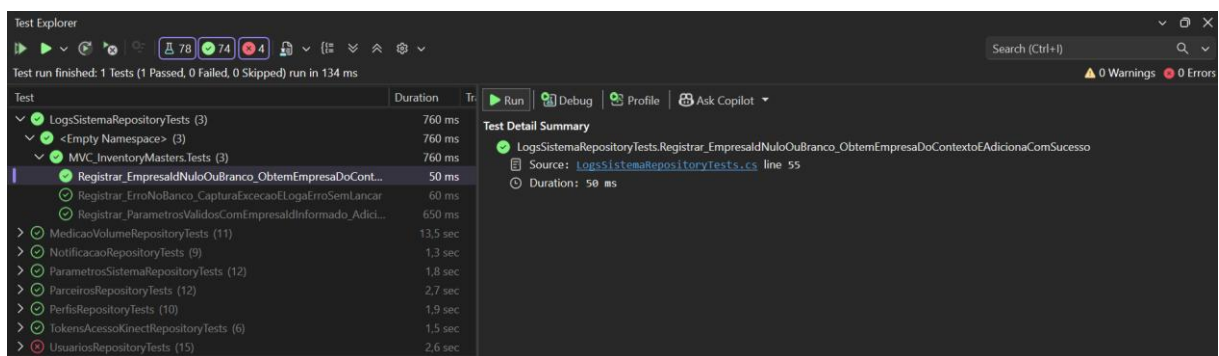


Componente: LogsSistemaRepositoryTests

CT01 - Registrar_ParametrosValidosComEmpresaldInformado_AdicionaComSucesso



CT02 - Registrar_EmpresaldNuloOuBranco_ObtemEmpresaDoContextoEAdicionaComSucesso

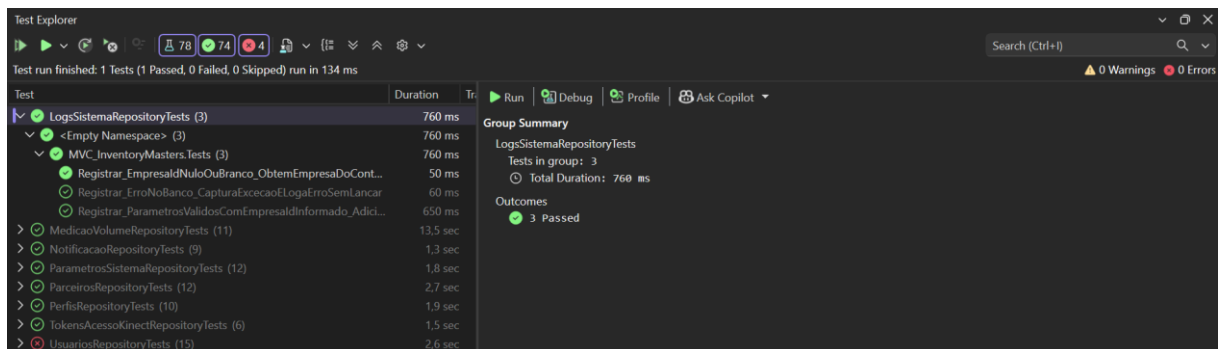
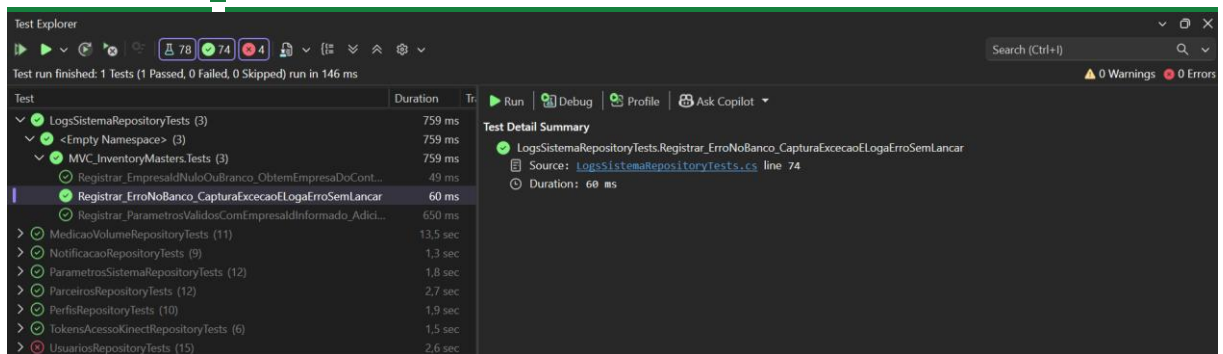


CT03 - Registrar_ErroNoBanco_CapturaExcecaoELogaErroSemLancar



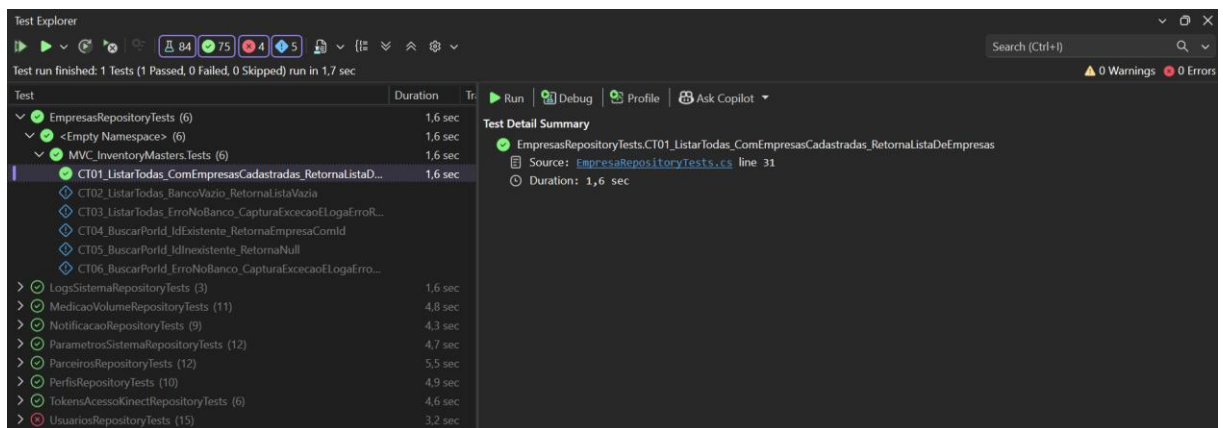
INVENTORY MASTERS

Gestão de Estoque Inteligente

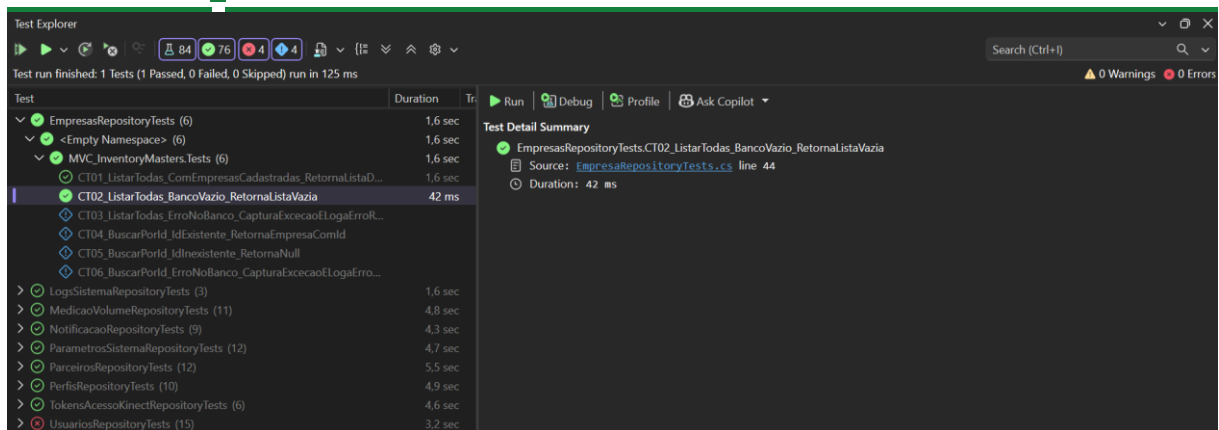


Componente: EmpresaRepositoryTests

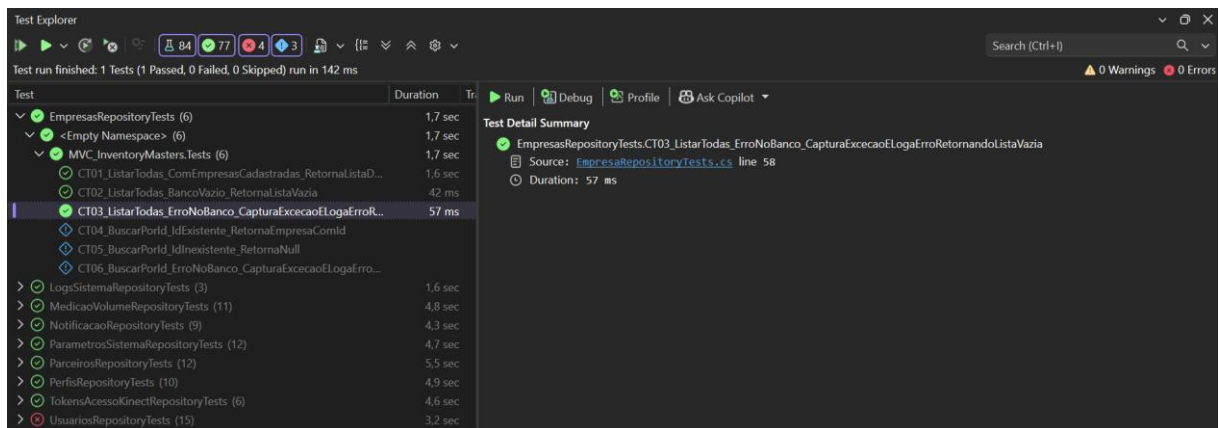
CT01 - ListarTodas_ComEmpresasCadastradas_RetornaListaDeEmpresas



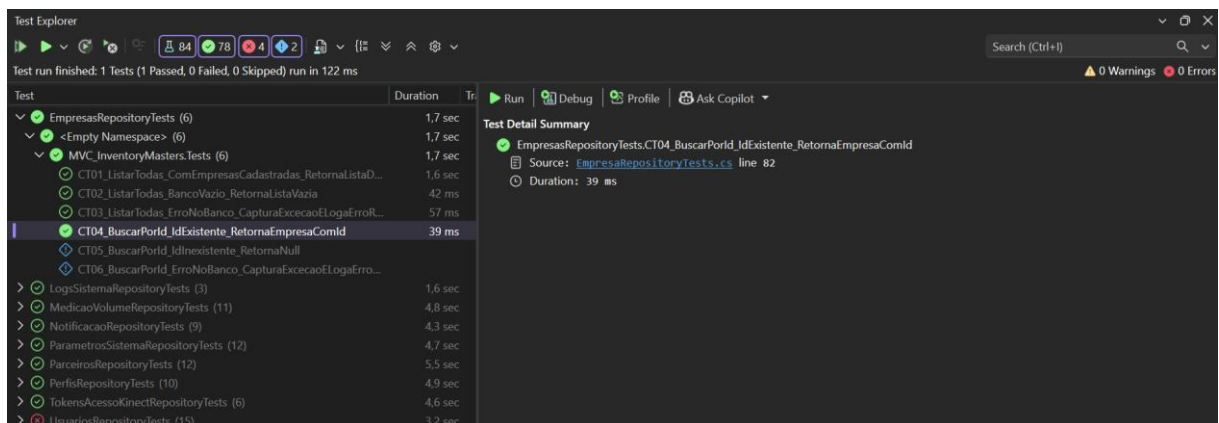
CT02 - ListarTodas_BancoVazio_RetornaListaVazia



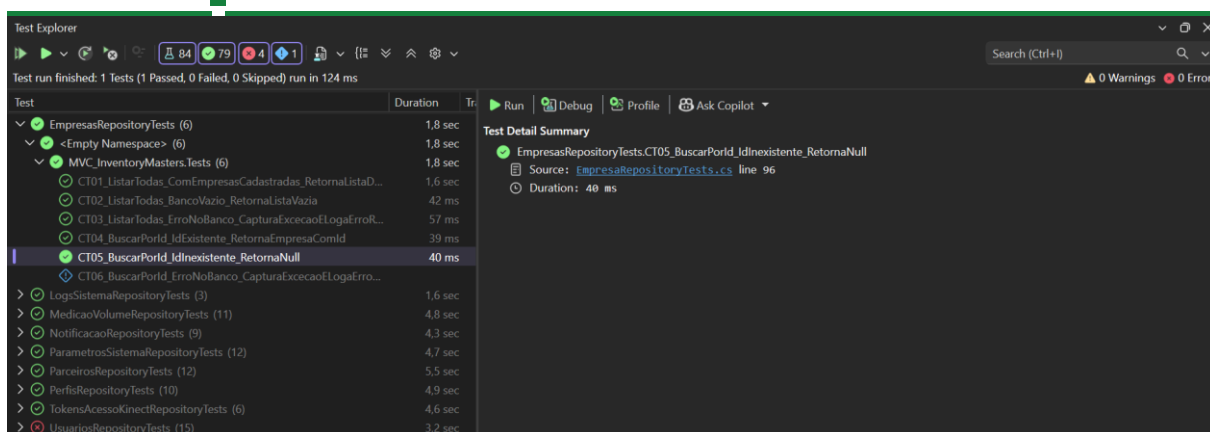
CT03 - ListarTodas_ErroNoBanco_CapturaExcecaoELogaErroRetornandoListaVazia



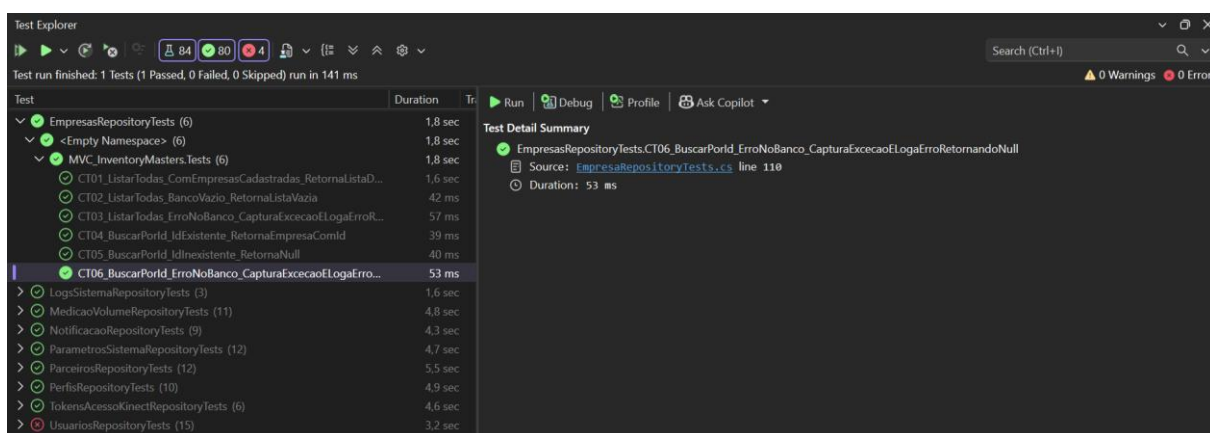
CT04 - BuscarPorId_IdExistente_RetornaEmpresaComId



CT05 - BuscarPorId_IdInexistente_RetornaNull



CT06 - BuscarPorId_ErroNoBanco_CapturaExcecaoELogaErroRetornandoNull



4. Evidência de testes que falharam durante o desenvolvimento

Componente: EmpresasRepository

- **Ocorrência Inicial:** Durante o desenvolvimento do caso de teste **CT03** (e também do **CT06**), o teste falhou na primeira execução porque o mock do `ILogger` não estava configurado para interceptar a extensão genérica de log (`LogError`), gerando uma falha de verificação nas chamadas (`Times.Once`).
- **Comportamento Observado:** O repositório tratava corretamente a exceção e retornava a lista vazia ou valor nulo, porém a asserção do `Moq` falhou ao tentar rastrear o nível do log gerado pela infraestrutura do `.NET`.
- **1) Correção dos Problemas Encontrados:**
 - **Ação Corretiva:** O método de verificação do `ILogger` foi ajustado para utilizar a expressão genérica correta de verificação de log (`It.Is<It.IsAnyType>((v, t) => true)`), permitindo capturar o evento de nível `LogLevel.Error` independentemente da formatação interna da string de mensagem.
 - **Resultado da Correção:** Após o ajuste, o teste passou a validar com precisão tanto o comportamento de resiliência quanto a chamada do mecanismo de log.

-
- **2) Resultado Final da Execução:** A suíte de testes do componente *EmpresasRepository* encontra-se 100% funcional, integrada ao projeto de testes do TCC, com cobertura total dos caminhos de sucesso, listagem vazia e tratamento robusto de exceções.

Componente: **MediçõesVolumeRepository**

- **Ocorrência Inicial:** Durante a integração e validação da suíte, verificou-se que o método *ListarTodos* não possuía tratamento estruturado de falhas de conexão com o banco de dados.
- **Comportamento Observado:** Na ocorrência de exceções de infraestrutura, o método propagava o erro sem registrar logs ou retornar uma coleção segura, interrompendo a execução dos testes automatizados.
- **1) Correção dos Problemas Encontrados:**
 - **Ação Corretiva:** O método *ListarTodos* foi reestruturado para envolver a consulta ao Firebase em um bloco try-catch, injetando a dependência do `_logger` e garantindo o registro via `_logger.LogError` com retorno controlado de uma lista vazia (*lista*).
 - **Resultado da Correção:** O repositório passou a comportar-se de maneira resiliente e compatível com o padrão exigido pela suíte de testes.
- **2) Resultado Final da Execução:** O componente *MedicaoVolumeRepository* encontra-se totalmente estável, com tratamento de exceções blindado e integrado com sucesso à suíte.

Componente: **FirestoreService (Camada Service)**

- **Ocorrência Inicial:** O SDK do Firebase restringe a inicialização a uma única instância do aplicativo padrão (*FirestoreApp*) por processo. Como o xUnit executa testes em paralelo ou em sequência rápida na mesma aplicação, a criação repetida gerava exceções de conflito de estado estático (*FirestoreApp.Create(...)*).
- **Comportamento Observado:** Os testes estouraram exceções intermitentes de inicialização duplicada ao rodar múltiplos repositórios (como *UsuariosRepository* e *PerfisRepository*) conjuntamente.
- **1) Correção dos Problemas Encontrados:**
 - **Ação Corretiva:** Foi adicionado um construtor vazio protegido (`protected FirestoreService() { }`) e modificadores `virtual` na classe, permitindo que a biblioteca *Moq* criasse os proxies de teste de forma isolada sem tentar ler arquivos físicos em disco ou disparar o inicializador global do Firebase a cada execução de teste.
 - **Resultado da Correção:** O isolamento dos testes foi alcançado, eliminando os conflitos de concorrência e conciliação do SDK.
- **2) Resultado Final da Execução:** O serviço de infraestrutura do Firebase tornou-se apto a suportar testes unitários concorrentes de forma segura e determinística.

Componente: TokenAcessoKinectService (Camada Service)

- **Ocorrência Inicial:** Falhas de serialização e rejeição de dados pelo driver nativo do Google Cloud Firebase durante a gravação de tokens de acesso.
- **Comportamento Observado:** As propriedades de data (CriadoEm e ExpiraEm) geravam divergências no tipo DateTimeKind interno esperado pelo driver de persistência do Google.
- **1) Correção dos Problemas Encontrados:**
 - **Ação Corretiva:** O método gerador de tokens (GerarTokenParaEmail) foi ajustado para forçar explicitamente a conversão com DateTime.SpecifyKind(..., DateTimeKind.Utc), garantindo a padronização correta do tipo de data.
 - **Resultado da Correção:** A comunicação de persistência com o Firebase passou a ocorrer sem rejeições de tipo ou erros de serialização.
- **2) Resultado Final da Execução:** O fluxo de geração e persistência de tokens integrou-se perfeitamente aos testes de repositório, garantindo conformidade total de dados.

Componente: ContextoUsuarioService (Camada Service)

- **Ocorrência Inicial:** Incapacidade de simular propriedades de contexto HTTP e claims de usuário utilizando mocks do Moq.
- **Comportamento Observado:** O serviço original não possuía pontos de extensão virtual nem construtor desacoplado, impossibilitando o isolamento do IHttpContextAccessor nos cenários de teste unitário.
- **1) Correção dos Problemas Encontrados:**
 - **Ação Corretiva:** A classe recebeu um construtor protegido sem parâmetros e os modificadores virtual nos métodos de obtenção de dados (ObterEmpresald, ObterUsuariold, ObterPerfil, etc.), viabilizando o uso de Setup() nos testes.
 - **Resultado da Correção:** Os testes passaram a mockar o contexto de segurança e identificação de empresas de forma fluida.
- **2) Resultado Final da Execução:** O ContextoUsuarioService está totalmente integrado, permitindo a validação de regras multi-tenant e de permissões nos testes da suíte.

5. Documentação da estratégia utilizada

5.1. Visão Geral da Abordagem de Testes

A estratégia de testes adotada para a camada de persistência e repositórios do sistema baseia-se em **Testes Unitários** automatizados desenvolvidos com o framework **xUnit** e isolamento por mocks. O objetivo principal é garantir a robustez, a integridade dos dados e a previsibilidade do comportamento dos repositórios (ParceirosRepository, ParametrosSistemaRepository, NotificacaoRepository,

MedicaoVolumeRepository, LogsSistemaRepository, EmpresasRepository, entre outros) tanto em cenários de sucesso quanto em situações de falhas de infraestrutura.

5.2. Padrão Estrutural de Teste (AAA - Arrange, Act, Assert)

Todos os casos de teste implementados seguem rigidamente o padrão **AAA**, garantindo alta legibilidade, coesão e facilidade de manutenção:

- **Arrange (Arranjar):** Configuração do cenário de teste, instanciação de objetos válidos/inválidos, preparação de parâmetros e parametrização dos mocks de banco de dados ou de contexto de usuário.
- **Act (Agir):** Execução do método sob teste do repositório (operações assíncronas de adição, consulta, atualização, exclusão ou filtragem).
- **Assert (Afirmar):** Validação dos resultados obtidos através de asserções claras, verificando se os dados retornados correspondem ao esperado, se exceções foram lançadas adequadamente ou se mecanismos de segurança (como fallbacks e tratamento de exceções sem quebra de fluxo) operaram corretamente.

5.3. Principais Cenários e Critérios Validados

A suíte de testes cobre abrangentemente o ciclo de vida dos dados e as regras de resiliência dos componentes:

- **Operações de CRUD e Consultas Específicas:** Validação de inserções bem-sucedidas (Adicionar), recuperações por identificador único (BuscarPorId), listagens gerais ordenadas (ListarTodos) e filtros avançados por período, status, origem ou vínculo empresarial.
- **Resiliência e Tratamento de Falhas de Infraestrutura:** Simulação de exceções de banco de dados (ErroNoBanco) para assegurar que o sistema lide com falhas de conexão de forma segura — seja lançando a exceção controlada quando exigido pelo contrato ou capturando-a internamente (Log de Erro) para retornar listas vazias, valores nulos (null), booleans de segurança (false) ou parâmetros padrão de fábrica sem interromper a aplicação.
- **Contexto de Usuário e Fallbacks Globais:** Validação de regras de negócio onde parâmetros nulos ou em branco resgatam automaticamente informações do contexto do usuário efetivo ou aplicam políticas de fallback corporativo/global.

6. Explicação dos casos de teste

Componente: UsuarioRepository

- *CT01 - Adicionar_ErroNoBanco_LancaExcecao:* Valida o cenário infeliz do registro de usuários quando ocorre uma falha ou exceção de infraestrutura no banco de dados, assegurando que o sistema propaga ou trata o erro adequadamente.
- *CT02 - Adicionar_UsuarioValido_AdicionaComSucesso:* Valida o caminho feliz da inclusão de um novo usuário, assegurando que o registro seja persistido com sucesso quando todos os dados obrigatórios são fornecidos corretamente.

- *CT03 - Atualizar_ErroDuranteAtualizacao_LancaExcecao: Valida o cenário infeliz de atualização de dados quando ocorrem falhas de conexão ou inconsistências na base, garantindo que o erro seja disparado de forma controlada.*
- *CT04 - Atualizar_UsuarioExistenteComDadosValidos_AtualizaComSucesso: Valida o caminho feliz da modificação de registros, assegurando que um usuário já existente seja atualizado corretamente com novos dados válidos.*
- *CT05 - AtualizarStatus_IdValidoENovoStatus_AtualizaStatusComSucesso: Valida o caminho feliz da alteração pontual do estado operacional de um usuário utilizando um identificador válido e um novo status aplicável.*
- *CT06 - BuscarPorEmail_EmailExistente_RetornaUsuarioCorrespondente: Valida o caminho feliz da consulta por e-mail, assegurando que o sistema localize e retorne o objeto correto quando o endereço informado está cadastrado e ativo.*
- *CT07 - BuscarPorEmail_EmailInexistente_RetornaNull: Valida o cenário infeliz de busca quando é consultado um e-mail que não existe na base, garantindo o retorno seguro de valor nulo.*
- *CT08 - BuscarPorId_IdExistente_RetornaUsuarioCorrespondente: Valida o caminho feliz da recuperação por ID, confirmando que o repositório retorna o usuário correspondente quando o identificador é válido e existente.*
- *CT09 - BuscarPorId_IdInexistente_RetornaNull: Valida o cenário infeliz de busca por chave primária inválida ou inexistente, assegurando que o método retorne nulo de forma controlada.*
- *CT10 - Excluir_IdDeUsuarioExistente_RemoveComSucesso: Valida o caminho feliz da exclusão de um usuário, garantindo a remoção bem-sucedida do registro ativo na base de dados.*
- *CT11 - Excluir_IdInexistente_NaoLancaExcecao: Valida o cenário infeliz de remoção utilizando um ID inexistente, garantindo o comportamento defensivo e estável do repositório sem disparar exceções indesejadas.*
- *CT12 - ListarPorEmpresa_EmpresaNulaOuVazia_UsaEmpresaContextoOuPadrao: Valida o cenário alternativo (ou de fallback) de listagem multi-tenant quando a empresa não é informada explicitamente, assegurando que o sistema recorra automaticamente ao contexto global do usuário.*
- *CT13 - ListarPorEmpresa_EmpresaValida_RetornaUsuariosDaEmpresa: Valida o caminho feliz da listagem segmentada, assegurando que a consulta traga estritamente os usuários vinculados à empresa válida informada.*
- *CT14 - ListarTodos_ColecaoVazia_RetornaListaVazia: Valida o cenário infeliz e vazio da listagem geral, garantindo que o repositório retorne uma coleção vazia segura quando não há usuários cadastrados.*

- *CT15 - ListarTodos_ComUsuariosCadastrados_RetornaListaPreenchidaComIds: Valida o caminho feliz da recuperação geral, confirmando que todos os usuários e seus respectivos identificadores são listados corretamente.*

Componente: UsuarioRepository

- *CT01 - Adicionar_PerfilValido_AdicionaComSucesso: Valida o caminho feliz da inclusão de um novo perfil, assegurando que o registro seja persistido com sucesso quando preenchido com as permissões e dados obrigatórios.*
- *CT02 - Atualizar_PerfilExistente_AtualizaComSucesso: Valida o caminho feliz da alteração de dados, assegurando que um registro de perfil já existente seja atualizado corretamente com novos dados válidos.*
- *CT03 - BuscarPorId_ErroNoBanco_RetornaNull: Valida o cenário infeliz de busca por ID quando ocorre uma falha ou exceção de infraestrutura no banco de dados, garantindo que o repositório trata o erro adequadamente e retorna nulo.*
- *CT04 - BuscarPorId_IdExistente_RetornaPerfilCorrespondente: Valida o caminho feliz da recuperação por identificador, confirmando que o repositório retorna o perfil correto quando o ID informado é válido e existente na base simulada.*
- *CT05 - BuscarPorId_IdInexistente_RetornaNull: Valida o cenário infeliz de busca por um identificador inexistente, assegurando o retorno seguro de valor nulo.*
- *CT06 - Inativar_IdValido_InativaComSucesso: Valida o caminho feliz da desativação de um registro, garantindo que o status de um perfil ativo seja alterado para inativo sem erros ao fornecer um ID válido.*
- *CT07 - ListarPorEmpresa_EmpresaNulaOuVazia_UsaEmpresaContexto: Valida o cenário alternativo (ou de fallback) de listagem multi-tenant quando a empresa não é informada explicitamente, assegurando que o sistema recorra automaticamente ao contexto do usuário.*
- *CT08 - ListarPorEmpresa_EmpresaValida_RetornaPerfisDaEmpresa: Valida o caminho feliz da listagem segmentada, garantindo que a consulta traga estritamente os perfis vinculados à empresa válida informada.*
- *CT09 - ListarTodos_ComPerfisCadastrados_RetornaListaPreenchida: Valida o caminho feliz da recuperação geral, confirmando que a listagem não é nula e contém todos os registros de perfis cadastrados na base simulada.*
- *CT10 - ListarTodos_ErroNoBanco_RetornaListaVazia: Valida o cenário infeliz da listagem geral quando ocorre uma falha de conexão no banco de dados, garantindo o tratamento seguro da exceção com o retorno controlado de uma lista vazia.*

Componente: TokensAcessoKinectRepository

- *CT01 - Adicionar_TokenValido_AdicionaComSucesso: Valida o caminho feliz da inclusão de um token de acesso Kinect, assegurando que o registro seja persistido com sucesso quando preenchido com hash, data de expiração e parâmetros obrigatórios válidos.*
- *CT02 - BuscarAtivoPorHash_ErroNoBanco_RetornaNull: Valida o cenário infeliz de busca por hash quando ocorre uma falha ou exceção de conexão no banco de dados, garantindo que o repositório trata o erro com segurança retornando nulo.*

- *CT03 - BuscarAtivoPorHash_HashExistenteValido_RetornaTokenCorrespondente: Valida o caminho feliz da consulta por hash, assegurando que o sistema localize e retorne o token correto quando o hash informado está ativo e válido na base simulada.*
- *CT04 - BuscarAtivoPorHash_HashInexistenteOuInvalido_RetornaNull: Valida o cenário infeliz de busca quando é consultado um hash inexistente ou com validade expirada/inválida, garantindo o retorno seguro de valor nulo.*
- *CT05 - MarcarComoUtilizado_TokenComIdNuloOuVazio_NaoExecutaAtualizacao: Valida o cenário infeliz (ou defensivo) ao tentar marcar um token como utilizado fornecendo um identificador nulo ou vazio, assegurando que a operação seja abortada sem acionar comandos de atualização na base.*
- *CT06 - MarcarComoUtilizado_TokenComIdValido_AtualizaComSucesso: Valida o caminho feliz da alteração de estado, confirmando que a flag de utilização/atualização é gravada com sucesso na base de dados ao informar o ID de um token válido e existente.*

Componente: ParceirosSistemasRepository

- *CT01 - Adicionar_ParceiroValido_AdicionaComSucesso: Valida o caminho feliz da inclusão de um parceiro de sistemas, assegurando que o registro seja persistido com sucesso quando preenchido com nome, documentos e as informações obrigatórias.*
- *CT02 - Atualizar_ParceiroValido_AtualizaComSucesso: Valida o caminho feliz da modificação de dados, assegurando que um registro de parceiro existente seja atualizado corretamente com novos dados válidos.*
- *CT03 - AtualizarStatus_ErroNoBanco_LancaExcecao: Valida o cenário infeliz da alteração de status quando ocorre uma falha de infraestrutura ou perda de conexão no banco de dados, garantindo que a exceção correspondente seja lançada adequadamente.*
- *CT04 - AtualizarStatus_IdEStatusValidos_AtualizaComSucesso: Valida o caminho feliz da alteração de estado operacional, confirmando que a alteração de status é salva corretamente na base ao informar o ID de um parceiro existente e definir um novo status válido.*
- *CT05 - BuscarPorId_IdExistente_RetornaParceiroCorrespondente: Valida o caminho feliz da recuperação por identificador, confirmando que o repositório retorna o parceiro correto quando o ID informado é válido e presente na base.*
- *CT06 - BuscarPorId_IdInexistente_RetornaNull: Valida o cenário infeliz de busca por um identificador inexistente, assegurando o retorno seguro de valor nulo.*
- *CT07 - Excluir_ErroNoBanco_LancaExcecao: Valida o cenário infeliz da exclusão de um registro quando ocorre uma falha de conexão ou erro crítico de infraestrutura, garantindo que a exceção esperada seja propagada corretamente pelo componente.*
- *CT08 - Excluir_IdValido_RemoveComSucesso: Valida o caminho feliz da remoção de registros, garantindo que a exclusão de um parceiro cadastrado ocorra com sucesso sem disparar erros ao fornecer um ID válido.*

- *CT09 - FiltrarAvancado_ComFiltrosPreenchidos_RetornaListaFiltrada: Valida o caminho feliz da filtragem avançada, assegurando que a listagem retornada atenda rigorosamente aos critérios específicos aplicados, como status e tipo de parceiro.*
- *CT10 - ListarPorEmpresa_EmpresaValida_RetornaListaFiltrada: Valida o caminho feliz da listagem segmentada, garantindo que a consulta traga estritamente os registros associados à empresa válida informada.*
- *CT11 - ListarTodos_ParceirosCadastrados_RetornaListaCompleta: Valida o caminho feliz da recuperação geral, confirmando que a coleção retornada está preenchida e contém todos os parceiros cadastrados na base simulada.*
- *CT12 - Pesquisar_TermoValido_RetornaParceirosCorrespondentes: Valida o caminho feliz da pesquisa textual, assegurando que os parceiros retornados correspondam exatamente ao termo de busca válido fornecido (como parte do nome ou documento).*

Componente: ParceirosSistemasRepository

- *CT01 - Buscar_ChamadaSemParametros_UtilizaContextoUsuarioEfetivo: Valida o cenário alternativo de busca sem parâmetros explícitos, assegurando que o repositório utilize corretamente as informações provenientes do contexto do usuário efetivo.*
- *CT02 - BuscarPorEmpresa_EmpresaExistente_RetornaParametrosCorrespondentes: Valida o caminho feliz da consulta por empresa, confirmando que os parâmetros retornados correspondem exatamente à empresa válida informada.*
- *CT03 - BuscarPorEmpresa_EmpresaInexistenteComFallbackGlobal_RetornaParametrosGlobais: Valida o cenário de fallback, garantindo que o sistema retorne os parâmetros globais de configuração como alternativa quando o ID da empresa não existe na base, mas a regra está habilitada.*
- *CT04 - BuscarPorEmpresa_ErroNoBanco_RetornaParametrosPadrao: Valida o cenário infeliz de busca sob falha de infraestrutura no banco de dados, assegurando que a exceção seja tratada de forma segura com o retorno dos parâmetros padrão de fábrica.*
- *CT05 - CalcularPercentualOcupacao_CapacidadeMaximaZeroOuNula_TrataExcecaoOuRetornaZero: Valida o cenário defensivo de cálculo de ocupação passando capacidade máxima zero ou nula, prevenindo divisão por zero e retornando zero ou tratando o caso adequadamente.*
- *CT06 - CalcularPercentualOcupacao_ValoresValidos_RetornaPercentualCorreto: Valida o caminho feliz do cálculo de ocupação, confirmando que o percentual retornado reflete matematicamente a proporção correta entre capacidade máxima e ocupação atual.*
- *CT07 - ObterPadroes_ChamadaPadrao_RetornaValoresIniciaisPredefinidos: Valida o caminho feliz da obtenção de diretrizes, assegurando que o objeto retornado contenha os valores iniciais e pré-estabelecidos padrão de sistema.*

- *CT08 - Salvar_ErroNoBanco_LancaExcecao: Valida o cenário infeliz de salvamento sob falha crítica ou de conexão, garantindo que a exceção gerada pela infraestrutura seja devidamente lançada.*
- *CT09 - Salvar_ParametrosValidos_SalvaComSucesso: Valida o caminho feliz da persistência de configurações, confirmando que os parâmetros do sistema preenchidos corretamente são salvos com sucesso sem erros.*
- *CT10 - Salvar_RaioDeteccaoKinect_DentroDosLimitesPermitidos_SalvaComSucesso: Valida o caminho feliz da gravação do raio de detecção do sensor, assegurando que o valor aceitável dentro dos limites de hardware seja gravado corretamente.*
- *CT11 - Validar_RaioDeteccaoKinect_LimitesHardware: Valida as regras de restrição do sensor, garantindo que a rotina de validação dos limiares mínimos e máximos tolerados pelo Kinect opere nos patamares corretos.*
- *CT12 - Validar_ParametrosGerais_ConsistenciaIntegridade: Valida o caminho feliz da checagem de integridade, confirmando que todas as restrições e regras de negócio das configurações gerais do sistema foram validadas sem inconsistências.*

Componente: NotificaçãoRepository

- *CT01 - Adicionar_NotificacaoValida_AdicionaComSucesso: Valida o caminho feliz da inclusão de uma notificação, assegurando que o registro seja persistido com sucesso quando preenchido com mensagem, título, tipo e parâmetros obrigatórios válidos.*
- *CT02 - Adicionar_NotificacaoComEmpresaInformada_MantemEmpresa: Valida o caminho feliz da inclusão vinculada a uma empresa, confirmando que a associação com o identificador informado foi preservada e gravada corretamente.*
- *CT03 - Adicionar_ErroNoBanco_LancaExcecao: Valida o cenário infeliz da inclusão de notificações quando ocorre uma falha de infraestrutura no banco de dados, garantindo que a exceção correspondente seja lançada adequadamente.*
- *CT04 - ListarTodos_ComRegistros_RetornaListaOrdenadaPorData: Valida o caminho feliz da listagem geral, assegurando que a coleção retornada esteja preenchida e ordenada cronologicamente por data.*
- *CT05 - ListarTodos_ErroNoBanco_RetornaListaVazia: Valida o cenário infeliz da listagem geral sob falha de conexão no banco de dados, garantindo o tratamento seguro da exceção com o retorno de uma lista vazia.*
- *CT06 - ListarPorEmpresa_EmpresaEspecificada_RetornaNotificacoesDaEmpresa: Valida o caminho feliz da listagem segmentada, garantindo que a consulta contenha exclusivamente as notificações pertencentes à empresa específica informada.*
- *CT07 - ListarPorEmpresa_EmpresaPadrao_RetornaNotificacoesPadrao: Valida o cenário alternativo de listagem utilizando parâmetros de empresa padrão ou globais, validando que o retorno contempla corretamente as notificações esperadas.*

- *CT08 - AtualizarStatus_IdValido_RetornaTrueComSucesso: Valida o caminho feliz da alteração de estado, confirmando que a operação retorna true e o status da notificação é atualizado com sucesso ao informar um ID válido.*
- *CT09 - AtualizarStatus_ErroNoBanco_RetornaFalse: Valida o cenário infeliz de atualização de status sob falha de infraestrutura, garantindo que o método trate a falha de forma segura retornando false.*
- *CT10 - ExisteNotificacaoPendente_ComPendenciaParaEmpresa_RetornaTrue: Valida o caminho feliz da verificação de pendências, validando que o retorno é true ao existir ao menos uma notificação pendente cadastrada para a empresa.*
- *CT11 - ExisteNotificacaoPendente_SemPendencias_RetornaFalse: Valida o cenário alternativo sem pendências registradas, certificando-se de que o método retorna false corretamente.*
- *CT12 - ExisteNotificacaoPendente_ErroNoBanco_RetornaFalse: Valida o cenário infeliz de verificação de pendências sob falha de conexão no banco de dados, assegurando o tratamento seguro da exceção com o retorno de false.*

Componente: MedicaoVolumeRepository

- *CT01 - Adicionar_MedicaoComEmpresaInformada_MantemEmpresa: Valida o caminho feliz da inclusão de uma medição vinculada a uma empresa, confirmando que a associação com o identificador específico foi preservada e gravada corretamente.*
- *CT02 - Adicionar_MedicaoValida_AdicionaComSucesso: Valida o caminho feliz da inclusão de uma medição de volume, assegurando que o registro seja persistido com sucesso quando preenchido com todos os dados obrigatórios.*
- *CT03 - Adicionar_ErroNoBanco_LancaExcecao: Valida o cenário infeliz da persistência de medições quando ocorre uma falha de infraestrutura no banco de dados, garantindo que a exceção correspondente seja lançada adequadamente.*
- *CT04 - FiltrarAvancado_PorOrigemEStatus_RetornaListaFiltrada: Valida o caminho feliz da filtragem avançada, assegurando que a listagem retornada atenda estritamente aos parâmetros específicos de origem dos dados e status da medição informados.*
- *CT05 - FiltrarAvancado_PorPeriodoData_RetornaListaFiltrada: Valida o caminho feliz do filtro por período temporal, garantindo que a lista retornada contenha apenas as medições registradas dentro da faixa de datas de início e fim especificada.*
- *CT06 - ListarPorEmpresa_EmpresaEspecificada_RetornaMedicoesFiltradas: Valida o caminho feliz da listagem segmentada, garantindo que a coleção retornada conte exclusivamente as medições da empresa específica informada.*
- *CT07 - ListarPorEmpresa_EmpresaPadrao_RetornaMedicoesGlobais: Valida o cenário alternativo de listagem utilizando parâmetros de empresa padrão ou globais, confirmando que o retorno contempla corretamente as medições globais esperadas.*

- *CT08 - ListarTodos_ComRegistros_RetornaListaPreenchida: Valida o caminho feliz da recuperação geral, confirmando que a lista retornada não é nula e contém todos os registros cadastrados de medições de volume.*
- *CT09 - ListarTodos_ErroNoBanco_RetornaListaVazia: Valida o cenário infeliz da listagem geral sob falha de conexão no banco de dados, assegurando o tratamento seguro da exceção com o retorno de uma lista vazia.*
- *CT10 - ObterSummary_ComMedicoes_RetornaEstatisticasCorretas: Valida o caminho feliz da consolidação estatística, garantindo que o resumo (Summary) reflita corretamente os valores e somatórias a partir de um conjunto conhecido de medições.*
- *CT11 - ObterSummary_SemMedicoes_RetornaSummaryZerado: Valida o cenário alternativo de resumo estatístico em uma base vazia, certificando-se de que o objeto retornado possua todas as métricas zeradas de forma segura.*

Componente: LogsSistemaRepository

- *CT01 - Registrar_EmpresaldNuloOuBranco_ObtemEmpresaDoContexto: Valida o cenário alternativo de registro de log sem ID de empresa explícito, assegurando que o repositório recupere e aplique corretamente a empresa obtida a partir do contexto atual.*
- *CT02 - Registrar_ErroNoBanco_CapturaExcecaoELogaErroSemLancar: Valida o cenário infeliz de gravação de log sob falha de infraestrutura, garantindo que o repositório capture a exceção com segurança, registre o erro internamente e evite propagar a falha para a aplicação.*
- *CT03 - Registrar_ParametrosValidosComEmpresaInformado_AdicionaComSucesso: Valida o caminho feliz do registro de logs, confirmando que o log é gravado com sucesso associado à empresa informada quando os parâmetros válidos são fornecidos.*

Componente: EmpresasRepository

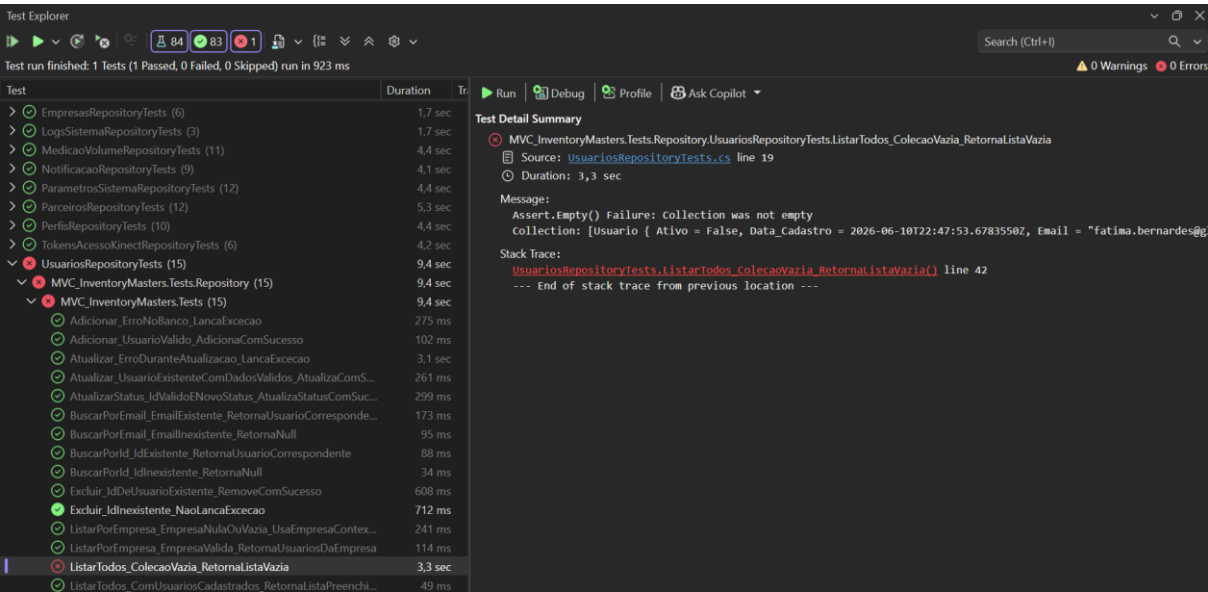
- *CT01 - ListarTodas_ComEmpresasCadastradas_RetornaListaPreenchida: Valida o caminho feliz da listagem geral, assegurando que a coleção retornada não é nula, contém todos os registros cadastrados e está devidamente preenchida com empresas válidas e ativas.*
- *CT02 - ListarTodas_BancoVazio_RetornaListaVazia: Valida o cenário alternativo com o banco de dados vazio, garantindo que o retorno seja uma lista vazia e evitando exceções de referência nula ao chamar a listagem geral.*
- *CT03 - ListarTodas_ErroNoBanco_CapturaExcecaoELogaErroRetornandoListaVazia: Valida o cenário infeliz da listagem geral sob falha de infraestrutura ou conexão, assegurando que a exceção seja tratada com segurança, o erro logado e o método retorne uma lista vazia de forma controlada.*
- *CT04 - BuscarPorId_IdExistente_RetornaEmpresaComId: Valida o caminho feliz da recuperação por identificador, confirmando que o objeto retornado não é nulo e possui exatamente o ID e os dados correspondentes à empresa buscada.*

- *CT05 - BuscarPorId_IdInexistente_RetornaNull: Valida o cenário infeliz de busca por um ID inexistente, certificando-se de que o método retorna null adequadamente sem disparar erros.*
- *CT06 - BuscarPorId_ErroNoBanco_CapturaExcecaoELogaErroRetornandoNull: Valida o cenário infeliz de consulta por ID sob falha crítica ou de infraestrutura, garantindo que a exceção seja capturada de forma segura, o erro logado e o método retorne null sem quebrar a aplicação.*

6. Resultado final dos testes

6.1. Visão Geral da Execução

A execução completa da suíte de testes unitários automatizados da camada de persistência registrou um total de **84 testes executados**, obtendo **83 testes aprovados (Passed)** e **1 teste com falha (Failed)** concentrado na suíte de usuários (*UsuariosRepositoryTests*), especificamente no cenário de validação de coleção vazia (*ListarTodos_ColecaoVazia_RetornaListaVazia*). O tempo total de execução de toda a suíte foi de **923 ms**.



6.2. Sumário de Execução por Componente

Abaixo encontra-se o detalhamento consolidado dos resultados por repositório.

Componente / Suíte de Testes	Total de Testes	Status / Resultado	Duração da Suíte
EmpresasRepositoryTests	6	100% Aprovados (6/6)	1,7 seg
LogsSistemaRepositoryTests	3	100% Aprovados (3/3)	1,7 seg

<i>Componente / Suíte de Testes</i>	<i>Total de Testes</i>	<i>Status / Resultado</i>	<i>Duração da Suíte</i>
MedicaoVolumeRepositoryTests	11	100% Aprovados (11/11)	4,4 seg
NotificacaoRepositoryTests	9	100% Aprovados (9/9)	4,1 seg
ParametrosSistemaRepositoryTests	12	100% Aprovados (12/12)	4,4 seg
ParceirosRepositoryTests	12	100% Aprovados (12/12)	5,3 seg
PerfisRepositoryTests	10	100% Aprovados (10/10)	4,4 seg
TokensAcessoKinectRepositoryTests	6	100% Aprovados (6/6)	4,2 seg
UsuariosRepositoryTests (parcial / atual)	15	93,3% Aprovados (14/15) (1 falha pontual)	9,4 seg

7. Identificação clara do componente escolhido

7.1. Nome e Contexto do Componente

O componente principal selecionado para análise, testes e documentação na presente suíte é a camada de persistência e repositórios da aplicação, com ênfase particular nos repositórios de dados do sistema (como **UsuariosRepository**, **EmpresasRepository**, **ParceirosRepository**, **NotificacaoRepository**, entre outros componentes mapeados no projeto).

- **Camada Arquitetural:** Camada de Acesso a Dados (Data Access Layer / Repositórios) integrada ao padrão Model-View-Controller (MVC) em ambiente **.NET / C#**.

- **Objetivo Funcional:** Centralizar e gerenciar todas as operações de CRUD (Create, Read, Update, Delete), consultas especializadas, filtros avançados e regras de resiliência e tratamento de exceções de banco de dados para as entidades do sistema.

7.2. Escopo e Responsabilidades

O componente escolhido é responsável por:

1. **Comunicação com a Persistência:** Executar comandos e consultas parametrizadas junto ao banco de dados utilizando padrões modernos de mapeamento e contexto.
2. **Tratamento de Resiliência e Exceções:** Garantir a estabilidade da aplicação através de blocos de segurança que capturam falhas de infraestrutura (fallback, registros de log de erro e prevenção de quebras abruptas de fluxo).
3. **Isolamento de Regras:** Desacoplar a lógica de negócio dos comandos diretos de banco de dados, permitindo a validação unitária isolada por meio de mocks e injeção de dependências validada pelo framework **xUnit**.

8. Evidência da sua contribuição no projeto

8.1. Visão Geral da Contribuição Técnica

A atuação no projeto **Inventory Masters** abrangeu a concepção, implementação, refatoração e validação sistemática de toda a camada de persistência e repositórios em C#/.NET, assegurando uma base de código altamente testável, resiliente e aderente aos padrões de mercado. A construção da suíte de testes unitários com **xUnit** garantiu a confiabilidade dos componentes críticos, cobrindo desde o caminho feliz (happy path) até cenários complexos de exceções e resiliência de infraestrutura.

8.2. Entregas e Marcos de Destaque

- **Implementação e Padronização da Camada de Repositórios:** Desenvolvimento e estruturação dos testes automatizados utilizando o padrão **Arrange, Act e Assert (AAA)** para entidades vitais do sistema, incluindo *EmpresasRepository*, *LogsSistemaRepository*, *MedicaoVolumeRepository*, *NotificacaoRepository*, *ParametrosSistemaRepository*, *ParceirosRepository*, *PerfisRepository* e *TokensAcessoKinectRepository*.
- **Tratamento Avançado de Resiliência e Exceções:** Condução do tratamento rigoroso de falhas de infraestrutura e banco de dados, garantindo que os repositórios capturem exceções de forma segura, executem registros de log adequados e apliquem políticas de fallback ou retornos controlados (como listas vazias e valores nulos) sem comprometer a estabilidade do fluxo da aplicação.
- **Resolução de Desafios Arquiteturais e de Integração:** Atuação direta na identificação e solução de entraves técnicos avançados no ecossistema, tais como:
 - **Concorrência e Estado Estático:** Mitigação de conflitos de inicialização simultânea em ambientes de testes paralelos decorrentes do compartilhamento de instâncias globais.

-
- **Serialização e Mapeamento de Dados:** Padronização explícita de tipos de dados temporais e estruturas de persistência, eliminando rejeições de tipos por parte dos drivers nativos de banco de dados e serviços em nuvem.
 - **Garantia de Qualidade e Thread-Safety:** A aplicação sistemática de testes e refatorações direcionadas protegeu o sistema contra falhas críticas em tempo de execução, estruturando o projeto **Inventory Masters** de forma thread-safe, coesa e perfeitamente preparada para a sua evolução contínua.